

Asymptote Fault Tolerance (AFT): Relay-Free Deterministic All-But-One ($n - 1$ of n) Byzantine Safety in the AFT Model Delta

IOI

March 17, 2026

Conditionality and exact-figures note. Every result in this paper is proven within the AFT model delta M_{AFT} — an assumed common publication boundary whose non-circular construction is the subject of, and is tracked by, the AFT-CB program. The public claim surface is governed by the whitepaper’s §5.3 assumption ledger (A1–A9) and prints only the interim conditional claim until that program’s gates pass. Tolerance figures appear in this paper only in exact all-but-one form: “all-but-one ($n - 1$ of n) Byzantine safety.” A rounded percentage is never printed as a tolerance figure, because with $n = 16$ – 64 validators the true all-but-one maximum is 93.75%–98.4375%; the all-but-one statement is both stronger and exact.

Abstract

Asymptote Fault Tolerance (AFT) is a family of consensus and settlement constructions for high-throughput ordering, guardian-backed base finality, proof-carrying canonical ordering, and deterministic release of irreversible effects. Its central move is to stop treating dense positive voting as the only possible source of ordering truth. Instead, AFT separates throughput-critical publication and chain progression from authority-critical proof verification, omission detection, and effect collapse.

The protocol family has four interacting surfaces. *GuardianMajority* supplies the production base-finality path: a validator quorum certificate is necessary but not sufficient, because each proposal must also carry a registered guardian committee certificate anchored into shared evidence. *CanonicalOrdering* supplies the proof-carrying ordering path: a slot order is accepted because a canonical certificate is uniquely valid and recoverable from public inputs, not because a dense validator quorum voted the order into existence. *Asymptote* supplies the sealing path for irreversible effects: **BaseFinal** chain progress is upgraded to **SealedFinal** through deterministic witness or observer-backed collapse, with canonical observer challenges dominating unsafe releases. *NestedGuardian* supplies a witness-augmented mode for layered threshold constructions.

This paper is self-contained and normative for the protocol family. Its ordering subtheorem remains all-but-one ($n - 1$ of n) equal-authority ordering consensus, conditional on the AFT model delta’s assumed common publication boundary and on explicit bulletin-publication, cutoff, omission, protocol-native retrievability, and proof-soundness assumptions. The runtime also internalizes compact signed publication-frontier metadata on the hot path: positive committed blocks carry a predecessor-linked frontier summary and compact availability-receipt binding, while same-slot conflicts or stale parent links admit short objective contradiction objects. Those frontier objects bind the claimed bulletin / ordering surface, while the protocol-native retrievability plane internalizes deterministic extraction-or-abort on the cold path. For each closed slot boundary there exists at most one admissible public execution object, every conflicting fully specified candidate admits a short public contradiction witness, durable state advances only through canonical collapse, and sealed-effect release is bound to the same collapse surface.

The lower-bound claim is explicit about model membership. The classical $n > 3f / f < n/3$ lower bound remains true in the classical partially synchronous message-passing model whose adversary may keep honest groups mutually ignorant of each other’s published views for the duration needed by the indistinguishability argument. AFT does not refute that theorem from inside its quantified model. It operates in the model where an admissible published artifact cannot be indefinitely suppressed from correct parties after stabilization, and it realizes that delta without a trusted relay: honest publishers send artifacts or erasure-coded shards directly over authenticated point-to-point links to validators or deterministic custodians, while validators publish only compact commitments, receipts, frontiers, and challenges as the echo layer. Payload delivery is relay-free; equivocation and omission detection still require compressed honest-party information flow.

In that model delta, and conditional on it, AFT is *relay-free, coordinator-free, pure-software deterministic all-but-one ($n - 1$ of n) Byzantine safety*: the classical threshold stops being load-bearing because the DLS adversary’s indefinite-suppression move is no longer legal, not because an impossibility theorem is violated inside its own model. Compact bindings stay on the hot path while cold-path recovery / reveal / reconstruction re-materialize the same close, abort, collapse, replay-prefix, execution-restart, and restart-ancestry surfaces through the AFT-native recovery-family contract. Accountable publication and validator removal remain operational reinforcement, not the source of correctness.

1 Scope and Status

This paper contains the full architectural and theorem-level description of the instantiated AFT family. It does not require any companion prose specification to be understood.

The instantiated protocol surfaces are:

- **GuardianMajority** as the production guardian-backed base-finality mode.
- **Asymptote** as the two-tier sealing and irreversible-effects mode.
- **CanonicalOrdering** through the live **CommittedSurfaceV1** certificate path plus compact **PublicationFrontier** summaries attached to committed blocks.
- **NestedGuardian** as an experimental witness-augmented operational mode with safety proofs, bounded operational checks, and a finished composed-liveness / classical-agreement bridge rather than a remaining composed-liveness kernel.

Implementation correspondence is included below as exact symbol references. That section is non-normative. The protocol and theorem content of the paper stands on its own.

The strongest theorem surface of the runtime is singular, and it is conditional on the AFT model delta. AFT claims not only all-but-one ($n - 1$ of n) equal-authority ordering for the ordering layer, but relay-free, coordinator-free, pure-software deterministic all-but-one ($n - 1$ of n) Byzantine safety for durable ordering and sealed-effect safety in the AFT model delta, with public-state continuity as its protocol-internal safety carrier rather than dense positive quorum intersection. Compact hot-path bindings plus cold-path recovery / reveal / reconstruction materialize the same recovered close-or-abort, collapse, replay-prefix, execution-restart, and streamed restart-ancestry surfaces through the AFT-native recovery-family contract. Compact publication/frontier metadata is internalized on the live path throughout, and deeper recovered history is now ordinary endogenous AFT history through the canonical-collapse / replay retrievability anchor and the matching recovered-state historical retrievability surface.

Model discipline: same ambition, explicit delta. No protocol refutes a lower bound from inside the model over which that lower bound universally quantifies. AFT’s claim is therefore a membership claim: the classical DLS / PBFT lower bound applies to a model whose adversary can keep honest groups mutually ignorant of published admissible information for the duration needed by the indistinguishability argument. AFT operates in a model delta where that indefinite-suppression move is revoked for content-addressed admissible protocol artifacts after stabilization. The delta is not a hidden helper regime: it is realized by authenticated direct publisher-to-validator or publisher-to-custodian dissemination plus a compact echo layer of commitments, receipts, frontiers, and challenges. The substantive objection would be a hidden relay, privileged coordinator, TEE, bootstrap oracle, external recovery service, or circular realization of the visibility resource by the same Byzantine-majority suppression power. AFT has none in the stated model.

Scope precision is not a concession. A criticism that AFT is “permissioned only” is non-responsive because the promoted claim is a permissioned theorem over fixed authenticated authorities and epoch-scoped identity state. Replacing that domain with open-membership consensus is a target switch, not a rebuttal. The lower-bound question is instead mechanical: execute the classical adversary against AFT and identify the first illegal move. That move is indefinite suppression of a published admissible artifact from correct parties after stabilization. Everything before that point is a standard adversarial schedule; everything after that point is outside the classical model and inside AFT’s explicit model delta.

Singular theorem sentence. The theorem surface of this repository is now singular all the way up: AFT claims relay-free, coordinator-free, pure-software deterministic all-but-one ($n - 1$ of n) Byzantine safety in the AFT model delta. It breaks the classical threshold in the precise model-relative sense that the theorem is proved in the AFT model delta, where published admissible artifacts cannot be indefinitely suppressed from correct parties and where that delta is realized without a trusted relay. AFT does not ask credit for defeating DLS / PBFT while remaining inside the bare DLS / PBFT model. It identifies the exact adversary power removed, implements the removal through standard authenticated channels and compressed echo, and then proves the close-or-abort safety surface in that model. Proof-carrying public evidence, endogenous historical retrievability, restart continuity, and collapse-gated durability are the realizing architecture of that claim, not a residual semantic gap.

Claim ladder. The full promoted claim should be read as a constructed ladder:

1. each closed slot boundary admits at most one admissible public execution object and every conflicting fully specified candidate admits a short objective obstruction witness,
2. durable state advances only through canonical collapse on that same close-or-abort surface,
3. irreversible effects release only through that same collapse surface.

Dimension	AFT realizing surface	Promoted singular claim
Agreement object	canonical close-or-abort / canonical collapse surface	one durable decision surface in M_{AFT}
Safety carrier	compact commitments, short contradiction objects, cold-path recovery, endogenous historical retrievability	dense positive quorum intersection is no longer load-bearing
History / bootstrap	canonical-collapse / replay historical retrievability root plus recovered-state historical retrievability surface	recovered history remains endogenous AFT state
Liveness burden	direct authenticated dissemination, compressed echo, recurrence/reduction/totality/collapse chain, persistent churn simulator	completed theorem surface under the AFT model delta

Target adversary/scheduler model and discharged liveness kernel. The singular theorem is interpreted under one explicit liveness model rather than a vague aspiration. Byzantine authorities may equivocate, omit, reorder, withhold reveals, withhold historical-retrievability objects, rotate maliciously, and crash/restart arbitrarily. Before stabilization the scheduler may delay or interleave publication, registry, recovery, archival, and restart traffic arbitrarily. After stabilization, authenticated channels between correct parties eventually deliver direct publisher sends and compact echo objects; repeatedly published admissible artifacts become visible and content-hash fetchable to correct parties; and governing profile/activation churn quiesces into bounded windows long enough for progress to compose. Under that model, the liveness bridge is the conjunction of five obligations: frontier-generation progress, canonical-resolution progress, recovery-completion progress, restart/historical-retrievability re-entry progress, and infinite composition of those four obligations across reassignment, outage, profile rotation, and archival page boundaries. That kernel is no longer open; it is the discharged bridge that supports the promoted AFT model-delta theorem.

The relevant liveness fact is not that AFT declines to claim a fully asynchronous theorem; the relevant fact is that it states the exact post-stabilization visibility resource used to halt the classical adversary. The protocol does not require universal timely receipt before the cutoff. It requires eventual delivery, persistence, and hash-fetchability of repeatedly published admissible artifacts after stabilization, realized by direct authenticated dissemination and compressed echo rather than by a trusted publication service.

Current artifact map and completion status. That kernel is now grounded in concrete repository artifacts rather than prose alone. Frontier-generation progress lands on the live `PublicationFrontier` path and its contradiction objects; canonical-resolution progress lands on canonical-order certificates, `CanonicalCollapseObject`, omission dominance, and recursive continuity; recovery-completion progress lands on the coded recovery-family contract and the recovered publication / close-or-abort surface; and restart/historical-retrievability progress lands on the AFT recovered-state surface plus canonical-history retrievability anchors and bounded-memory paging. Those obligations now have both concrete runtime carriers and a completed formal bridge. The formal package contains bounded churn witnesses, recurring cycle cores, a recurrence theorem, a finite reduction, a total classical- agreement history bridge, and a directly discharged semantic-collapse wrapper ending in `NestedGuardianRecoveryClassicalAgreementCollapse.tla`. The exact source of that recurrence / reduction / totality / collapse bridge is embedded in Appendix A, including the final wrapper in Appendix A.6.5. The recurrence / reduction / totality / collapse

chain is not narrative glue. Its exact artifact chain is embedded verbatim in Appendix A, ending in the classical-agreement collapse wrapper itself. The runtime mirrors the same story through a persistent historical-retrievability churn/restart simulator over one evolving recovered-history state. The remaining work is therefore no longer theorem completion, but cleanup, stress expansion, and maintenance of the promoted singular claim.

2 Problem Statement

Classical permissioned Byzantine fault tolerance ties deterministic safety to dense positive voting by a large honest majority. Under that model, honest inclusion, total-order agreement, and finality all depend on most validators remaining both correct and available. That is the right model when the protocol is allowed to pay dense coordination cost and accept classical $3f + 1$ -style thresholds. It is the wrong model for a system that wants all of the following simultaneously:

- high-throughput publication and chain progression,
- equal-authority participation at the validator layer,
- deterministic rejection of incomplete or conflicting ordered views,
- stronger accountability than “someone must have voted wrong,”
- deterministic gating of irreversible effects.

AFT therefore changes the object of agreement.
Instead of asking:

- “Which order did the validator quorum choose?”

it asks:

- “Which order certificate is uniquely valid for this slot, and which effects are admissible after deterministic collapse?”

The family-level design goals are:

- keep ordering and publication sparse on the hot path,
- keep effect release fail-closed,
- make omissions and conflicts objectively provable,
- keep all validators equally eligible to reveal the canonical order,
- move authority from dense yes-votes to verifiable evidence.

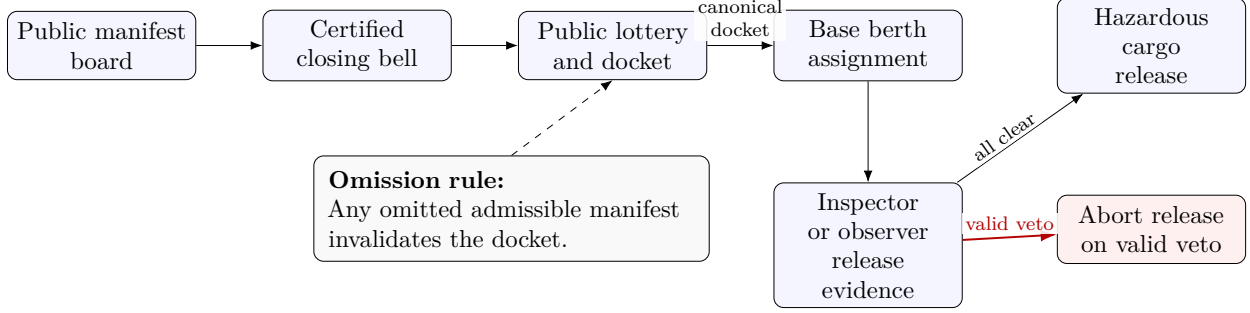


Figure 1: The thought experiment behind AFT: public admission, canonical ordering, base progress, and fail-closed release.

3 Thought Experiment: The Port of Public Manifests and Release Seals

Imagine an international port that handles ordinary cargo and hazardous cargo.

Every ship that wants to unload during tide h must post its cargo manifest to a public harbor board before the closing bell τ_h . The bell is not a private clock in the harbor master’s office. It is a certified public event. Once the bell rings, the harbor office takes every admissible manifest, applies the published tide lottery R_h , and derives one canonical docking docket O_h .

The office does not ask every captain to vote on the exact order of every container. It asks a simpler question: has someone produced the one valid docket certificate for the manifests that were publicly posted before the bell? If any dockworker can prove that an admissible manifest was omitted from the announced docket, that docket is rejected immediately.

Ordinary unloading may begin once the berth assignment is base-final. Hazardous cargo is stricter. It may leave the port only after one of two things happens:

- the required independent inspector guilds each sign a release certificate, or
- a deterministically assigned set of external captains publishes a public inspection transcript surface, the public red-flag board remains empty through the close bell, and the port issues the unique release docket implied by that surface.

Any valid red flag aborts release. The cargo does not “probably” leave the port. It does not leave.

Most of the port can be chaotic. Captains can lie, stall, or collude. What they cannot do is create a second valid docket for the same tide or a second valid hazardous-cargo release order, provided the public manifest board remains recoverable and the decisive proof surface stays objectively checkable.

The correspondence is direct:

Port object	AFT object
Public manifest board	Bulletin / DA surface B_h
Certified closing bell	Canonical cutoff τ_h
Tide lottery	Public randomness beacon R_h
Docking docket	Canonical order O_h
Docket certificate	Canonical-order certificate $OCert_h$
Omitted manifest challenge	Omission proof $\Omega(h, tx)$
Base berth assignment	BaseFinal
Independent inspector guilds	Witness committees
Sampled external captains	Equal-authority observers
Red-flag report	Canonical observer challenge
Hazardous cargo release seal	SealObject under SealedFinal

This thought experiment captures the essential AFT move: sparse operational progress, dense cheap verification, objective omission, and deterministic collapse for irreversible consequences.

4 Protocol Family Overview

AFT is best understood as four planes that compose:

Plane	Purpose	Throughput-critical work	Authority object
Dissemination / bulletin plane	Publish the candidate transaction surface	Data dissemination and bulletin commitments	Public bulletin commitment
Base-finality plane	Advance the chain and commit block identity	Proposal, QC formation, guardianized certification	Validator QC + guardian certificate
Canonical-ordering plane	Prove the unique slot order	Succinct verification, omission dominance	$OCert_h$
Sealed-effect plane	Gate irreversible effects	Asynchronous witness / observer evidence collection	SealedFinalityProof and SealObject

The protocol modes are:

Mode	Role	Current status
GuardianMajority	Production guardian-backed base finality	Production mode
CanonicalOrdering	Proof-carrying equal-authority ordering	Live through CommittedSurfaceV1
Asymptote	Two-tier sealing and sealed-effect release	Production sealing path
NestedGuardian	Witness-augmented layered threshold mode	Experimental operational mode / completed theorem bridge

The key invariant is:

$$\text{throughput-critical work} \neq \text{authority-critical work}$$

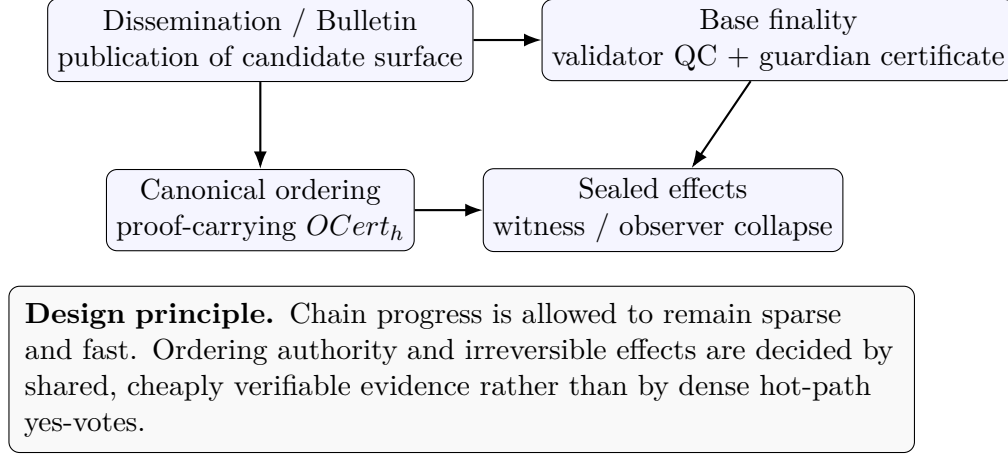


Figure 2: AFT separates publication, base progress, canonical ordering, and sealed effect release, then recomposes them through shared evidence.

5 System Model and Notation

5.1 Actors

- **Validator:** participates in proposal, vote, verification, and chain execution.
- **Guardian committee:** the registered threshold-signing committee protecting one validator’s proposal decisions.
- **Guardian member:** an individual signer within a guardian committee.
- **Witness committee:** an external threshold-signing committee used on the sealing path or in *NestedGuardian*.
- **Equal-authority observer:** a validator sampled to audit a slot under *Asymptote*.
- **Registry:** the on-chain source of truth for manifests, policies, epochs, witness sets, and verifier descriptors.
- **Transparency log:** an append-only checkpointed evidence log for guardian and witness statements.
- **External verifier or gateway:** a party that checks *SealObjects* and enforces replay-safe release.

5.2 Bulletin-board time and slot model

Consensus proceeds in heights and views. A slot is identified by $(height, view)$. Epoch-scoped policy and committee state bind the admissible evidence for that slot. The family should be read under a model-delta publication and recoverability discipline rather than a pure point-to-point timely receipt discipline: safety depends on whether the closed boundary fixes a unique public evidence surface, while liveness depends on correct-party delivery and eventual visibility of admissible content-addressed artifacts after stabilization.

Slots are externally paced by a public cutoff object τ_h and a public randomness beacon R_h . Honest validators that observe the same closed public boundary derive the same ordering or sealing close-or-abort result. In the runtime that derivation is no longer keyed off cutoff closure alone: the bulletin plane exports a *canonical bulletin-close object* binding the bulletin commitment, the bulletin-availability certificate, and the canonical cutoff into one public close surface. In this instantiation, positive canonical-order publication is fail-closed at this boundary: a closed-slot order certificate is admitted only if the published bulletin entries deterministically extract the bulletin surface bound by the canonical bulletin-close object and the carried bulletin-availability certificate. Universal timely receipt is not required.

Let

$$\begin{aligned} h &= \text{slot height}, \quad v = \text{slot view}, \\ B_h &= \text{bulletin surface admitted before cutoff } \tau_h, \\ R_h &= \text{public randomness beacon for slot } h, \\ E_h &= \{tx \mid \text{Included}(tx, B_h) \wedge \text{Eligible}(tx, \text{root}_{h-1})\}, \\ O_h &= \text{CanonicalOrder}(R_h, E_h), \\ \text{root}_h &= \text{Execute}(\text{root}_{h-1}, O_h). \end{aligned}$$

5.3 Classical lower bound and AFT model delta

Let M_{DLS} denote the standard authenticated partially synchronous message-passing model used by the classical DLS / PBFT lower-bound argument. Its adversary may delay messages so that two honest groups cannot distinguish “the other group is slow” from “the other group is faulty” for the duration required by the indistinguishability construction.

Let M_{AFT} denote the AFT model delta. It keeps fixed authenticated authorities, deterministic local verification, and partial synchrony among correct parties, but it removes one adversary power: after stabilization, an admissible content-addressed protocol artifact repeatedly published by a correct party cannot be indefinitely suppressed from correct parties. Payload delivery is realized by direct authenticated dissemination from the publisher to validators or deterministic custodians; equivocation, omission, and retrievability detection are realized by compact echo objects rather than by payload relay.

Adversary-execution trace. Run the classical indistinguishability adversary against AFT:

1. The adversary may schedule delays, split honest validators into views, and make Byzantine validators equivocate or omit.
2. The adversary may hide a Byzantine sender’s conflicting statements until an honest validator publishes a protocol-admissible commitment, receipt, frontier, challenge, or reconstruction / abort artifact.
3. The first illegal step is indefinite suppression of that published admissible artifact from another correct validator after stabilization. In M_{DLS} , that suppression power is available to the adversary long enough to drive the lower-bound execution. In M_{AFT} , the artifact eventually becomes visible and content-hash fetchable.

That stopped step is the model delta. The rest of AFT is protocol design inside M_{AFT} : compact frontiers bind the claimed surface, contradiction objects kill conflicts, retrievability objects reconstruct or abort the full surface, and canonical collapse gates durability.

Lemma 1 (Direct dissemination plus compressed echo realizes the delta). *Assume authenticated point-to-point channels among correct parties that eventually deliver after stabilization. An honest publisher sends its artifact or erasure-coded shards directly to the validators or deterministic custodians named by the slot policy. No non-sender validator must relay the payload for honest-artifact delivery. Validators then publish compact commitments, availability receipts, publication frontiers, observer transcripts, challenges, and reconstruction / abort objects that expose what they received or what the public surface lacks. The payload path discharges honest delivery; the echo path discharges equivocation, omission, and retrievability detection. The classical adversary’s indefinite-suppression step is therefore unavailable without introducing a trusted relay, coordinator, TEE, bootstrap oracle, or external recovery service.*

5.4 Core assumptions

The family-level safety claims rely on the following explicit assumptions:

- **Objective publication:** admitted bulletin entries, transcripts, and challenges have stable hashes and objective inclusion paths.
- **Authenticated direct dissemination:** after stabilization, authenticated channels between correct parties eventually deliver the publisher’s direct artifact or shard sends to the validators or deterministic custodians named by the slot policy.
- **Compressed echo visibility:** validators publish compact commitments, receipts, frontiers, transcripts, challenges, and reconstruction / abort artifacts that make equivocation, omission, stale lineage, and retrievability failures objectively visible without echoing full payloads.
- **Verifiable bulletin availability:** the protocol exports a bulletin-availability object, compact publication-frontier metadata that binds the same slot surface, and a canonical bulletin-close object for each closed ordering slot; the remaining assumption is that direct dissemination and the compressed echo layer satisfy the retrieval semantics claimed by those objects. The live runtime also requires successful closed-slot extraction before positive canonical-order admission.
- **Compact frontier consistency:** any carried `PublicationFrontier` must bind the same-slot bulletin commitment, ordered surface, and compact availability receipt, and same-slot conflicts or stale predecessor links must admit short objective contradiction objects. These frontier objects summarize the slot surface; they do not reconstruct it.
- **Objective validity only:** theorem-critical rejection or acceptance does not depend on private local facts.
- **Public derivability over universal receipt:** honest validators that observe the same closed public boundary derive the same close-or-abort result; universal timely receipt is not required.
- **Independent echo path:** suppressing negative evidence requires capture of both the payload-delivery path and the compact publication / registry echo surface.
- **Sealed-effect gating:** irreversible effects are released only after surviving the canonical close-or-abort surface.
- **Fixed epoch identities:** the validator set is epoch-scoped and stable while the accountable rules of that epoch are enforced.

Here and throughout, the AFT permissioned model means fixed authenticated authorities, deterministic local verification, authenticated point-to-point channels among correct parties, public hash-addressed protocol objects, epoch-scoped identity state, and eventual visibility / persistence / hash-fetchability of admissible protocol artifacts after stabilization, with no trusted relay, privileged coordinator, TEE, theorem-side bootstrap oracle, or hidden external recovery service. Fixed authenticated membership is part of the theorem statement, not a missing crutch. The hidden-helper exclusions are relay, coordinator, TEE, bootstrap oracle, and external recovery service — not epoch-scoped authenticated identity itself.

5.5 Evidence objects

The family-level evidence objects are:

- validator quorum certificates over block proposals,
- guardian quorum certificates over slot decisions,
- witness certificates over guardianized slots,
- canonical-order certificates over bulletin surfaces,
- omission proofs over ordered-set incompleteness,
- observer transcripts, observer challenges, and canonical close-or-abort objects over sealed finality,
- sealed-effect proof objects with replay-safe nullifiers.

All honest nodes are required to derive the same acceptance result from the same admissible evidence set.

5.6 Accountable publication rules

Objective AFT faults are not merely audit artifacts. In the runtime they are first-class accountable protocol objects:

- `OmissionProof` names the validator accountable for the dominated positive order object.
- `AsymptoteObserverChallenge` determines accountability by kind: `MissingTranscript` and `ConflictingTranscript` blame the assigned observer; `TranscriptMismatch`, `VetoTranscriptPresent`, and `InvalidCanonicalClose` blame the producer / positive-close path.
- Valid objective fault evidence is replay-deduplicated on chain.
- Policy-controlled membership updates are aftermath only: the same evidence may trigger best-effort immediate quarantine and stage next-epoch eviction, but the decisive negative object remains valid even when those membership updates are disabled, delayed, or skipped.

These rules do not change the hot path. They strengthen the adversary model: arbitrary behavior is tolerated only insofar as it remains publicly revealable and penalty-bearing.

6 Public State Continuity with Extractable Obstructions

AFT’s deeper theorem target is not a new round gadget. It is a new source of uniqueness. Classical Byzantine agreement derives uniqueness from overlap of dense positive quorums. AFT instead seeks a rigidity theorem over public slot objects: once the slot boundary is closed, the space of admissible public continuations should collapse to one canonical object or one canonical abort.

6.1 Boundary-conditioned execution language

For a closed slot boundary define

$$\partial_h := (\text{root}_{h-1}, \beta_h, \tau_h, R_h, p_h),$$

where β_h is the canonical bulletin-close object binding the bulletin commitment, bulletin-availability certificate, and canonical cutoff whose recoverable surface is B_h ; τ_h is the canonical cutoff object; R_h is the slot randomness beacon; and p_h is the policy descriptor governing admissible checks for slot h .

Let a *public execution object* for slot h be a normalized public codeword

$$X_h := (B_h, E_h, O_h, \text{root}_h, \sigma_h),$$

where E_h is the eligible subset, O_h is the canonical ordered object, root_h is the resulting state commitment, and σ_h is the slot-local sealing surface needed to determine whether the slot closes or aborts. The important point is that X_h is not a distributed transcript. It is the public quotient of the slot after irrelevant schedule variation is factored out.

Define the boundary-conditioned language

$$\mathcal{L}(\partial_h) := \{X \mid \exists \pi \text{ Accept}(\partial_h, X, \pi) = 1\}.$$

Here π is a succinct admissibility witness. For canonical ordering, π is the proof-carrying order certificate witness. For deterministic observer sealing, π is the witness that the canonical transcript surface, challenge surface, and close-or-abort object satisfy the slot policy.

6.2 Obstruction extractability

The companion rejection relation is

$$\text{Reject}(\partial_h, Y, \omega) = 1,$$

where Y is a fully specified candidate public execution object and ω is a short public objective contradiction witness.

AFT seeks *obstruction-complete local testability*: for a fixed closed boundary, every fully specified candidate lies in exactly one of two states,

$$\left(\exists \pi \text{ Accept}(\partial_h, Y, \pi) = 1 \right) \oplus \left(\exists \omega \text{ Reject}(\partial_h, Y, \omega) = 1 \right).$$

This xor formulation is the software-only analogue of state continuity. It is stronger than merely having a valid proof system. It says every non-admissible candidate exposes a short public contradiction witness.

In the live repository, the obstruction basis is not abstract:

- ordering uses a first-class canonical abort taxonomy over the canonical bulletin surface: missing certificates, bulletin-surface reconstruction failures, bulletin-surface mismatches, invalid bulletin-close formation, omission-dominated candidates, certificate-height mismatches, randomness mismatches, ordered-transactions-root mismatches, resulting-state-root mismatches, invalid public-input bindings, invalid bulletin-availability bindings, and invalid proof bindings,
- deterministic observer sealing uses objective challenge kinds over the canonical transcript and challenge surface,
- the accountable-adversary layer turns those same objects into penalty-bearing public fault evidence.

6.3 Unique collapse object

The slot should not merely admit at most one positive object. It should admit at most one *collapse object*:

$$\text{Collapse}(\partial_h) \in \{\text{Close}(X_h), \text{Abort}(\Omega_h)\}.$$

Here $\text{Close}(X_h)$ denotes the unique admissible positive object when the obstruction surface is empty, while $\text{Abort}(\Omega_h)$ denotes the unique negative object induced by a non-empty valid obstruction surface. Negative evidence does not create a second truth. It forces the slot to land on its canonical abort object instead of a close object.

In the runtime this collapse object is not merely an abstract proof target. Validator finalization publishes a first-class `CanonicalCollapseObject` through `guardian_registry`, and later consensus verification cross-checks any published copy against the same proof-carried ordering and sealing surface when parent-state data is available. In the current recursive-continuity slice, `CanonicalCollapseObject` also carries a rolling `previous_canonical_collapse_commitment_hash`, and in the current clean-break runtime slice it also carries `continuity_accumulator_hash` together with `continuity_recursive_proof`, so the current slot’s collapse object is linked to the previously persisted or published collapse object rather than standing as an isolated per-slot fact. The newest proposal-surface slice pushes that same predecessor link into the signed header itself: `BlockHeader` now carries `previous_canonical_collapse_commitment_hash` in its signed preimage and also carries a typed `CanonicalCollapseExtensionCertificate`. That certificate now carries the predecessor commitment plus the predecessor recursive-proof hash, while the public predecessor `CanonicalCollapseObject` now carries only a single recursive proof step rather than a carried predecessor-proof tree. Each recursive proof step still binds a predecessor collapse commitment, predecessor commitment hash, predecessor payload hash, and previous proof hash; the rolling `continuity_accumulator_hash` remains in place as the companion accumulator over the same chain. The implementation surface distinguishes the reference `HashPcdV1` carrier from a backend slot `SuccinctSp1V1`, and the runtime exposes a dedicated continuity-verifier seam for those recursive public inputs via `ioi-api` and `zk-driver-succinct`. That seam is now wired into the live protocol: `GuardianMajority` invokes it while validating proposal and QC continuity for carried or anchored `SuccinctSp1V1` proof steps, and validator durable-state checks invoke the same backend before a persisted `CanonicalCollapseObject` is treated as authoritative. Proposal verification checks that the carried predecessor commitment hash matches the signed predecessor link, checks that the predecessor commitment’s resulting state root matches the signed parent-state root, and checks that the carried predecessor proof hash matches the anchored predecessor collapse object already persisted or published in protocol state. Leaders in `Asymptote` stall rather than propose when the required extension certificate or anchored predecessor collapse object is unavailable, proposal

verification rechecks that implied predecessor commitment against both the signed predecessor commitment link and the parent-state root, checks the carried predecessor proof hash against the anchored predecessor proof on the public collapse object, and QC progression only treats continuity-linked headers as progress-authoritative.

Theorem 1 (Public State Continuity with Extractable Obstructions). *Assume a closed boundary ∂_h , objective publication, a protocol-native retrievability plane, compact publication-frontier consistency, objective validity only, and proof soundness. Then the instantiated and normative theorem shape for AFT is:*

1. $\mathcal{L}(\partial_h)$ contains at most one admissible public execution object,
2. every fully specified candidate $Y \notin \mathcal{L}(\partial_h)$ admits a short public objective obstruction witness ω such that $\text{Reject}(\partial_h, Y, \omega) = 1$,
3. therefore the slot admits at most one admissible collapse object, either the unique close object or the unique abort object.

This theorem is the precise form of the software-only continuity primitive that the instantiated AFT runtime realizes. The later ordering, collapse, recovery, restart, and sealed-effect sections are concrete theorem instances and composition layers of this same continuity statement, not separate aspirations. It does not claim that the classical DLS / PBFT frontier disappeared inside the standard dense-vote model. The point is different: the uniqueness source has moved from quorum overlap into the public-evidence language itself. In the instantiated runtime, compact publication frontiers internalize same-slot and predecessor consistency inside the live protocol, while the protocol-native retrievability plane discharges closed-bulletin extraction-or-abort on the cold path.

6.4 Lemma program

The clean proof path is a filtration rather than a single opaque argument:

$$\mathcal{L}_0(\partial_h) \supseteq \mathcal{L}_1(\partial_h) \supseteq \cdots \supseteq \mathcal{L}_k(\partial_h),$$

where each refinement adds one structural constraint and each failed refinement admits a short witness.

The repository's theorem program is:

1. **Boundary Fixing Lemma.** The closed boundary fixes the admissible publication domain.
2. **Bulletin Close Lemma.** The canonical bulletin-close object fixes the unique recoverable bulletin surface for the slot.
3. **Eligibility Determinism Lemma.** The predecessor commitment and boundary fix the eligible subset.
4. **Order Determinism Lemma.** The boundary and eligible subset fix the canonical ordered object.
5. **Transition Determinism Lemma.** The ordered object fixes the next state commitment.
6. **Collapse Exclusivity Lemma.** A slot cannot admit both a canonical close object and a canonical abort object.

7. **Obstruction Soundness Lemma.** Every valid omission proof or observer challenge objectively rejects its target candidate.
8. **Obstruction Completeness Lemma.** Every invalid fully specified candidate contains at least one minimal public obstruction witness.
9. **Extractor Sufficiency Lemma.** Given the canonical bulletin-close object, any honest validator can run the protocol-defined bulletin extractor to reconstruct the same bulletin surface; in this instantiation, admission of the positive ordering object already requires this extraction to succeed over the published closed-slot surface. One honest validator remains sufficient to surface the resulting positive or negative object under the instantiated runtime model.

One honest validator is surfacing-sufficient but never trust-bearing: correctness does not attach to the revealer’s identity, because any honest validator observing the same closed boundary derives the same decisive object.

The theorem surfaces of this paper are instances of that program:

- **CanonicalOrdering** instantiates unique admissibility and extractable obstruction through proof-carrying order certificates plus omission dominance.
- **Asymptote** instantiates unique collapse through canonical transcript/challenge surfaces plus close-or-abort exclusivity.

7 The AFT Deterministic Base Engine

AFT modes share a deterministic core engine for proposal flow, QC formation, locking, and view progression.

7.1 Deterministic leadership and view progression

For a given height and view, the implementation selects the proposer by deterministic round-robin over the active validator set:

$$\text{leader}(h, v) = V_h[(h - 1 + v) \bmod |V_h|].$$

Here V_h is the active validator set for height h after operational quarantine filters are applied.

View 0 proposals do not carry a timeout certificate. Any non-zero view proposal must carry a valid timeout certificate for the previous unsuccessful view. The pacemaker advances views monotonically and resets timers on observed progress.

7.2 Validator quorum certificates

In **GuardianMajority**, **Asymptote**, and **NestedGuardian**, validator quorum is a strict simple majority of active validator weight, not a classical $2/3 + 1$ quorum:

$$\text{QCQuorum}(V_h) > \frac{1}{2} \text{Weight}(V_h).$$

This is safe only because the validator quorum is *not* the sole authority object. A proposal must also satisfy guardian-layer admissibility.

Timeout certificates use the same simple-majority rule over view-change votes.

7.3 Locking, 2-chain commit, and divergence guard

Validators maintain a lock on the highest safe parent QC they have observed. A validator votes only when the proposal extends the lock or is in a strictly higher view than the lock.

Base commitment uses a 2-chain rule:

- let QC_{parent} certify parent block P ,
- let QC_{child} certify child block C ,
- if C directly extends P and the views are consecutive, then P becomes a pending commit.

The implementation delays final durability by a guard window Δ_{guard} before a pending commit is released. The purpose of the guard window is operational, not purely algebraic: it gives divergence evidence or panic signals time to propagate before a conflicting branch is treated as durable.

This produces a fail-closed operational behavior:

- sparse chain progress happens on the hot path,
- conflicting evidence is surfaced before irreversible consequences,
- divergence proofs quarantine the affected node rather than silently allowing fork continuation.

8 GuardianMajority: Production Base Finality

GuardianMajority is the production fault model for AFT base finality. Validator votes alone never make a proposal admissible. Every valid proposal must also carry a guardian committee certificate that verifies against current registered committee state.

8.1 Guardian certificate object

The guardian certificate binds:

- the guardian committee manifest hash,
- the active epoch,
- the canonical decision hash for the slot payload,
- a monotonic guardian counter,
- a guardian trace root,
- a measurement root,
- the signer bitfield,
- the aggregated BLS signature,
- a transparency-log checkpoint,
- and, in **NestedGuardian**, an optional experimental witness certificate.

The guardian decision payload is slot-scoped. Honest guardians are assumed not to sign two different decisions for the same slot.

8.2 Verification rule

A guardianized proposal is admissible only if all of the following hold:

1. the referenced guardian committee manifest is registered on-chain for the active epoch,
2. the certificate manifest hash matches the registered manifest,
3. the certificate epoch matches the manifest epoch and current epoch,
4. the decision hash matches the slot’s canonical guardian decision payload,
5. the measurement root matches the decision payload,
6. the signer set is valid, unique, and meets the committee threshold,
7. the aggregated signature verifies against the manifest public keys,
8. the attached transparency-log checkpoint verifies against the anchored log descriptor and append-only proof.

The transparency log is an accountability layer. It does not replace threshold safety. It makes issued decisions, checkpoints, and later slashing evidence publicly checkable.

8.3 Committee threshold model

Let a guardian committee have threshold t and size n . Conflicting slot certificates require enough equivocators to cover the minimum quorum intersection:

$$f < 2t - n.$$

Production committees therefore use strict majority thresholds:

$$t = \left\lfloor \frac{n}{2} \right\rfloor + 1.$$

This guarantees pairwise quorum intersection. It also exposes an important operational nuance:

- odd-sized simple-majority committees tolerate zero equivocating guardian members at the committee layer,
- even-sized majority committees tolerate one equivocator before threshold intersection is exhausted.

Production safety therefore relies on both threshold intersection and stronger guardian non-equivocation guarantees.

8.4 Safety statement

Proposition 1 (GuardianMajority Safety). *No two conflicting blocks can both finalize for the same (height, view) if guardian committee manifests are current, committee thresholds are majority thresholds, honest guardians do not equivocate for the same slot, transparency-log checkpoints and registry state are current enough to verify admissibility, and validators finalize only guardian-admissible blocks.*

This is not a classical $3f + 1$ theorem. It is a composed threshold theorem over validators, guardians, registry state, and shared evidence. That statement is local to the guardian-backed base-finality layer. It does not delimit the paper’s promoted theorem surface. The promoted durable-agreement claim is carried by the later proof-carrying close-or-abort and canonical- collapse architecture, which is precisely why the classical $3f + 1$ threshold is not load-bearing there.

8.5 Liveness model

Liveness is secondary to safety. Progress requires:

- enough active validators to form the simple-majority validator QC,
- enough guardian members to satisfy the committee threshold,
- current registry state for the epoch,
- current enough checkpoint and log availability.

If these degrade, the protocol prefers stalling over accepting unsafe evidence.

9 Canonical Ordering

`CanonicalOrdering` is the proof-carrying ordering path that gives AFT its all-but-one ($n - 1$ of n) equal-authority ordering consensus claim in the AFT model delta.

9.1 Objective

For each slot h , define a unique canonical ordered set O_h and a certificate $OCert_h$ such that:

- every honest verifier can check $OCert_h$ cheaply,
- any omission is objectively provable,
- the ordered set is recoverable from public data,
- the live hot path carries a compact publication frontier that binds the same-slot bulletin / ordering / availability surface and predecessor continuity,
- conflicting valid certificates for the same slot are impossible,
- arbitrary behavior by almost all validators cannot create a conflicting valid ordering outcome because the closed bulletin boundary admits one deterministically derivable close-or-abort result.

9.2 Canonical objects

For slot h ,

$$\begin{aligned} B_h &= \text{bulletin surface admitted before } \tau_h, \\ R_h &= \text{public randomness beacon for slot } h, \\ E_h &= \{tx \mid \text{Included}(tx, B_h) \wedge \text{Eligible}(tx, \text{root}_{h-1})\}, \\ O_h &= \text{CanonicalOrder}(R_h, E_h), \\ \text{root}_h &= \text{Execute}(\text{root}_{h-1}, O_h). \end{aligned}$$

The abstract certificate is

$$OCert_h = (h, \text{root}_{h-1}, \text{cutoff}_h, \text{bulletin_commitment}_h, \text{ordered_commitment}_h, \text{root}_h, \text{witness}_h).$$

9.3 Bulletin, cutoff, and admissibility assumptions

The ordering theorem is conditional on the following named assumptions.

A1. Objective publication. Each admitted transaction or batch in B_h has a stable content hash, a verifiable inclusion path into the bulletin commitment, and an objective publication time or publication position relative to the slot cutoff.

A2. Endogenous retrievability. The bulletin contents committed by $\text{bulletin_commitment}_h$ are reconstructed or deterministically aborted from protocol-native retrievability objects rooted in the canonical bulletin close for slot h . Compact publication-frontier summaries and availability certificates bind that claim on the hot path, while retrievability profiles, shard manifests, custody assignments, custody receipts, custody responses, and objective challenge objects close the extraction-or-abort loop on the cold path. Without that plane, uniqueness may still hold while timely materialization of the same close-or-abort surface can fail.

A3. Append-only or equivocation-evident commitment. The bulletin commitment and any carried publication frontier for slot h must be append-only or otherwise make same-slot equivocation or stale predecessor linkage objectively detectable at the slot boundary.

A4. Eligibility determinism. $\text{Eligible}(tx, \text{root}_{h-1})$ is deterministic and locally checkable from the committed predecessor root and the transaction object. Honest validators therefore derive the same eligible set E_h from the same public inputs.

A5. Proof soundness. $\text{VerifyProof}(\text{witness}_h) = \text{true}$ implies that the witness obligations for cutoff binding, bulletin binding, ordering, state transition, retrievability commitments, and omission commitments actually hold.

The cutoff τ_h is not a private local clock read. It is a protocol-bound object. In the instantiated runtime, the cutoff is bound directly into the published bulletin commitment and into the canonical public-input set. Eligibility is deterministic with respect to the predecessor state root and the transaction object. This prevents local reinterpretation of the eligible set.

9.4 Current runtime instantiation

The runtime surfaces the abstract objects above as the following concrete objects:

Runtime object	Current fields
BulletinCommitment	(height, cutoff_timestamp_ms, bulletin_root, entry_count)
BulletinSurfaceEntry	(height, tx_hash)
BulletinAvailability Certificate	(height, bulletin_commitment_hash, recoverability_root) (height, bulletin_commitment_hash, bulletin_availability_certificate_hash, recoverability_root, shard_count, recovery_threshold, custody_threshold)
BulletinRetrievabilityProfile	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, recoverability_root, entry_count, shard_count, recovery_threshold, shard_commitment_root)
BulletinShardManifest	(height, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, validator_set_commitment_hash, assignments)
BulletinCustodyAssignment	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, custodian_count, custody_threshold, custody_root)
BulletinCustodyReceipt	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_assignment_hash, bulletin_custody_receipt_hash, served_shards)
BulletinCustodyResponse	(height, kind, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_assignment_hash, bulletin_custody_receipt_hash, bulletin_custody_response_hash, details)
BulletinRetrievabilityChallenge	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_assignment_hash, bulletin_custody_receipt_hash, bulletin_custody_response_hash, canonical_bulletin_close_hash, canonical_order_certificate_hash, reconstructed_entry_count, reconstructed_bulletin_root)
BulletinReconstructionCertificate	(height, kind, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_receipt_hash, bulletin_retrievability_challenge_hash, canonical_order_abort_hash, details)
BulletinReconstructionAbort	(height, bulletin_commitment_hash, ordered_transactions_root_hash, resulting_state_root_hash, receipt_root)
PublicationAvailabilityReceipt	(height, view, counter, parent_frontier_hash, bulletin_commitment_hash, ordered_transactions_root_hash, availability_receipt_hash)
PublicationFrontier	(height, kind, candidate_frontier, reference_frontier)
PublicationFrontierContradiction	(height, cutoff_timestamp_ms, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_receipt_hash, entry_count)
CanonicalBulletin Close	(height, parent_state_root_hash, bulletin_commitment_hash, randomness_beacon, ordered_transactions_root_hash, resulting_state_root_hash, cutoff_timestamp_ms)
CanonicalOrder PublicInputs	concrete variants HashBindingV1 and CommittedSurfaceV1
CanonicalOrderProofSystem CommittedSurface CanonicalOrderProof	(recoverability_root, omission_commitment_root) (height, bulletin_commitment, bulletin_availability_certificate, randomness_beacon, ordered_transactions_root_hash, resulting_state_root_hash, proof, omission_proofs)
CanonicalOrder Certificate	(bulletin_commitment, bulletin_entries, bulletin_availability_certificate, bulletin_retrievability_profile, bulletin_shard_manifest, bulletin_custody_receipt, canonical_order_certificate)
CanonicalOrder PublicationBundle	(height, reason, details, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_close_hash, canonical_order_certificate_hash)
CanonicalOrder Abort	

HashBindingV1 is the reference verifier family: its proof bytes are a canonical hash binding over the public-input hash. **CommittedSurfaceV1** is the live proof family: its proof payload contains the recoverability root and the omission-commitment root for the committed surface. In the current runtime, positive ordering artifacts are published through the atomic **CanonicalOrderPublicationBundle**, carrying the bound retrievability profile / manifest / custody-receipt objects, while the registry deterministically materializes the coupled custody-assignment and custody-response objects over the same slot surface before admitting the positive lane. Positive committed blocks also carry a compact signed **PublicationFrontier**: it binds the same-slot bulletin commitment, ordered-transactions root, and compact publication-availability receipt, and it hash-links that summary to the previous slot’s frontier. Short **PublicationFrontierContradiction** objects make same-slot conflict or stale predecessor linkage objectively rejectable on the hot path. Those frontier objects internalize compact publication consistency, but they do not replace the protocol-native retrievability plane that reconstructs or aborts the full bulletin surface. Validators also run a block-local derivation pass over the committed block boundary: they derive either a **CanonicalOrderExecutionObject** or a **CanonicalOrderAbort**, and publish the abort artifact when the proof-carried positive object cannot be derived. When the endogenous retrievability lane succeeds, the registry materializes a first-class **BulletinReconstructionCertificate** bound to the canonical close, canonical-order certificate, and the same retrievability objects. When endogenous retrievability evidence dominates that same surface, the registry also materializes a specialized **BulletinReconstructionAbort** object bound to the challenge and the paired **CanonicalOrderAbort**, so reconstruction-impossible failure is named as a first-class protocol object rather than only as a generic ordering abort.

9.5 Acceptance rule

Every validator applies the same deterministic rule:

$$\begin{aligned} \text{Accept}(OCert_h) \iff & \text{VerifyProof}(witness_h) \\ & \wedge \text{VerifyCutoff}(cutoff_h) \\ & \wedge \text{PredecessorAccepted}(root_{h-1}) \\ & \wedge \neg \exists tx : \text{ValidOmission}(h, tx). \end{aligned}$$

In the implementation this specializes to:

- the certificate height and bulletin height must match the block height,
- the randomness beacon must match the slot schedule,
- the bulletin commitment must match the published bulletin when that bulletin is available from anchored state,
- any carried **PublicationFrontier** must match the same-slot bulletin commitment, ordered-transactions root, and publication-availability receipt hash,
- when a predecessor frontier exists, the carried **PublicationFrontier** must extend it by counter and parent hash,
- no published **PublicationFrontierContradiction** may already dominate the slot,

- positive canonical-order admission in the runtime additionally requires successful closed-slot extraction over the published bulletin entries, the carried bulletin-availability certificate, and the canonical bulletin-close object,
- the ordered-transactions root and resulting state root must match the block header,
- the proof must match the canonical public inputs,
- the recoverability root must match the certificate commitments,
- the omission-commitment root must match the omission-proof set.

If the omission-proof set is non-empty, the candidate certificate is rejected immediately.

No dense positive vote on the order is required once a valid $OCert_h$ is surfaced. The current verifier entry point for this rule is `verify_canonical_order_certificate`.

9.6 Omission dominance

An omission is objective when a transaction:

- was included in B_h before τ_h ,
- was eligible under $root_{h-1}$,
- and is absent from the committed ordered set O_h .

Define the omission proof

$$\Omega(h, tx) = \left(\begin{array}{l} inclusion_proof(tx, bulletin_h), \\ cutoff_admissibility_proof(tx, cutoff_h), \\ eligibility_proof(tx, root_{h-1}), \\ nonmembership_proof(tx, O_h) \end{array} \right).$$

Its semantics are:

$$\text{Valid}(\Omega(h, tx)) \Rightarrow \text{Reject}(OCert_h).$$

This is the central AFT ordering move. A candidate does not need to win a dense round of positive endorsements if any incomplete view can be killed objectively.

Omission dominance is a correctness rule, not a punishment rule. A valid omission proof kills the candidate immediately whether or not slashing, quarantine, or next-epoch removal ever fires.

9.7 Uniqueness, publication frontiers, and recoverability

Theorem 2 (Uniqueness). *If $\text{ValidOrderCert}(h, C_1)$ and $\text{ValidOrderCert}(h, C_2)$, then C_1 and C_2 commit the same canonical ordered set O_h .*

The reason is structural:

- the predecessor root is fixed,
- the cutoff object is canonical,
- the bulletin commitment is canonical,

- eligibility is deterministic,
- the ordering function is deterministic,
- proof soundness forbids a witness for a different ordered set.

The live publication-frontier slice sits between uniqueness and recoverability. A frontier gives a compact signed summary of the claimed bulletin / ordering surface and its predecessor link, so same-slot conflict or stale lineage is objectively rejectable without reconstructing the full bulletin on the hot path. But the frontier only binds hashes and receipt roots; it is not itself the recoverable bulletin surface.

Theorem 3 (Endogenous Retrievability). *If ValidOrderCert(h, C) and the canonical bulletin close plus protocol-native retrievability plane for slot h are present and unchallenged, then every honest verifier can reconstruct the same ordered set O_h from protocol objects alone: the canonical close, bound frontier / availability commitments, retrievability profile, shard manifest, custody receipt, positive reconstruction certificate, and published bulletin entries.*

Endogenous retrievability matters because the accepted order is not merely unique. The compact frontier tells validators which surface is being claimed; the retrievability plane tells them that the full surface can actually be reconstructed or objectively aborted from protocol evidence.

The hot path and the cold path do not carry different truths. The frontier binds the claimed surface compactly; the retrievability plane reconstructs or objectively aborts that same surface; both terminate on the same canonical close-or-abort object.

9.8 All-but-one ($n - 1$ of n) equal-authority ordering consensus

AFT’s ordering claim is a proof-carrying consensus theorem whose decisive object is revealed rather than manufactured by dense positive yes-votes. Revelation is not a weaker post hoc reporting layer here; it is the equal- authority surfacing mechanism for the uniquely valid public execution object already fixed by the closed public boundary.

Theorem 4 (All-But-One ($n - 1$ of n) Equal-Authority Ordering Consensus in the AFT Model Delta). *Assume, in the AFT model delta (the assumed common publication boundary):*

1. *the closed bulletin surface B_h bound by the carried publication frontier and bulletin commitment is governed by a canonical bulletin close and protocol-native retrievability plane that deterministically yields extraction or abort,*
2. *the cutoff object for slot h is canonical,*
3. *bulletin commitments, compact publication-frontier bindings, and omission proofs are objectively verifiable,*
4. *same-slot frontier conflicts and stale predecessor links admit short objective contradiction objects,*
5. *the proof system is sound.*

Then even if all but one validator ($n - 1$ of n) behaves arbitrarily, they cannot create a conflicting valid canonical-order outcome for slot h . More strongly, every honest validator that observes the same closed ordering boundary derives the same positive order object or omission-dominated abort.

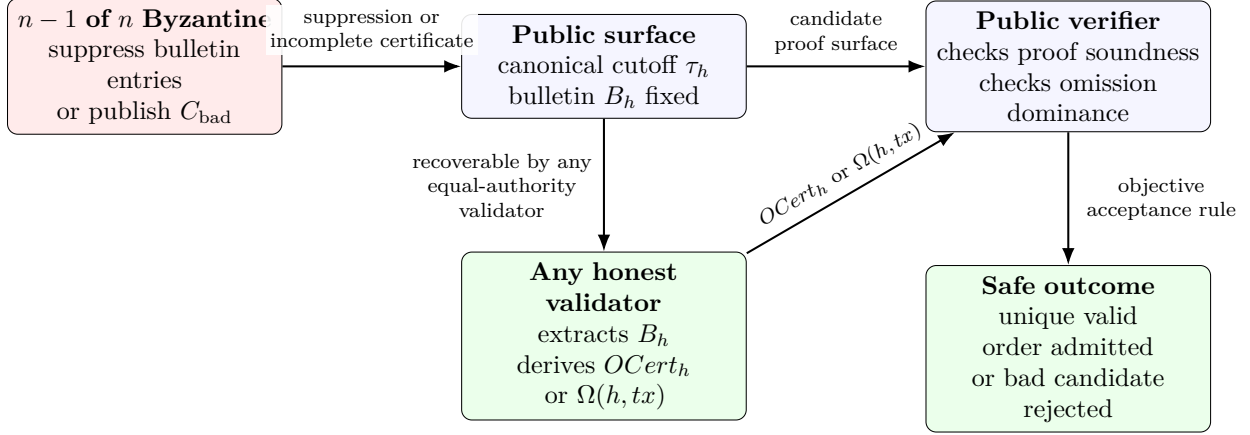


Figure 3: Omission-dominance attack and deterministic extraction. Even if a suppressing super-majority tries to hide admissible bulletin entries or promote an incomplete candidate certificate, any honest validator that reconstructs the same public surface derives the same valid $OCert_h$ or publishes a valid omission proof $\Omega(h, tx)$ that kills the bad candidate.

Compact publication frontiers therefore internalize cheap same-slot and predecessor consistency checks on the live path, while recoverability remains the stronger assumption needed to turn that compact claim into a publicly reconstructable ordered set.

The strongest shorthand, in the AFT model delta, is:

$n - 1$ of n validators may behave arbitrarily,

because the closed ordering boundary admits one deterministically

derivable close-or-abort outcome.

This is equal-authority because any validator may reveal the winning certificate. It tolerates all but one Byzantine validator ($n - 1$ of n) because correctness does not depend on a dense majority of positive votes. This section isolates the ordering lane, not the theorem ceiling. Once composed with endogenous retrievability, canonical collapse, recovery and restart continuity, and collapse-bound sealed-effect gating, the promoted AFT theorem surface is exactly the paper’s relay-free deterministic all-but-one ($n - 1$ of n) Byzantine safety claim in the AFT model delta.

With the accountable publication rules of Section 5, invalid positive ordering attempts are not only rejectable; they are also penalty-bearing and next-epoch removing.

10 Asymptote: Sealed Finality and Irreversible Effects

Asymptote is the two-tier settlement mode in the AFT family. It preserves fast **BaseFinal** chain progression while requiring stronger evidence before irreversible effects are released.

10.1 Finality tiers and collapse states

The finality tiers are:

- **BaseFinal**: validator QC plus guardian certificate.
- **SealedFinal**: **BaseFinal** plus sufficient sealing evidence.

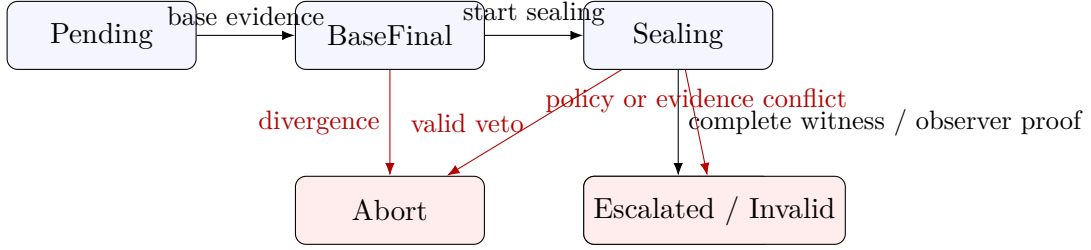


Figure 4: The deterministic collapse surface for **Asymptote**.

The slot collapse states are:

- Pending
- BaseFinal
- Sealing
- Abort
- SealedFinal
- Escalated
- Invalid

All honest nodes must derive the same collapse state from the same admissible evidence set.

10.2 SealedFinalityProof

The sealing plane gathers a proof object that binds:

- the epoch,
- the achieved finality tier,
- the collapse state,
- the guardian manifest hash,
- the guardian decision hash,
- the guardian counter,
- the guardian trace root,
- the guardian measurement root,
- the policy hash,
- witness certificates, when the witness path is used,
- observer transcripts, transcript commitment, observer challenges, challenge commitment, and exactly one canonical observer outcome object when the observer path is used,

- divergence signals gathered during sealing.

This object is block-scoped and guardian-bound. A `SealedFinalityProof` that is not bound to the slot's guardian evidence is invalid.

10.3 Witness-backed sealing

The witness path assigns one witness committee per required certification stratum.

Witness assignment is deterministic over:

- the epoch seed,
- the producer account id,
- the slot (*height*, *view*),
- the reassignment depth,
- the witness stratum id,
- the witness manifest hash.

Sealed finality is reached on the witness path when:

- `BaseFinal` holds,
- each required witness stratum contributes a valid assigned witness certificate,
- each witness certificate verifies against the registered witness manifest,
- required checkpoints and append-only proofs are current enough under policy.

Witness omission, stale registry participation, checkpoint inconsistency, or conflicting witness certificates produce explicit evidence rather than silent best-effort behavior.

10.4 Equal-authority observer sealing

The normative observer path is `CanonicalChallengeV1`. It treats sampled validators as deterministic duty assignees whose outputs must become public, recoverable protocol objects.

Observer assignment is deterministic over:

- the epoch seed,
- the producer account id,
- the slot (*height*, *view*),
- the observation round,
- the observer account id.

The producer is excluded from its own observer pool. The runtime may also filter assignments through a correlation budget over provider, region, host class, and key-authority class.

Each assigned observer publishes one of two things:

- a guardian-backed `AsymptoteObserverTranscript`, or
- an objective `AsymptoteObserverChallenge`.

A transcript binds the canonical slot surface:

- the epoch and assignment,
- the candidate block hash,
- the guardian manifest, decision hash, counter, trace hash, and measurement root,
- the guardian checkpoint root,
- the observer verdict and veto kind,
- the observer evidence hash.

The admissible challenge basis is public-evidence-only:

- `MissingTranscript`,
- `TranscriptMismatch`,
- `VetoTranscriptPresent`,
- `ConflictingTranscript`,
- `InvalidCanonicalClose`.

Each challenge carries its own public evidence object: an assignment, an offending observation request, an offending transcript, or an offending canonical close. The challenge `evidence_hash` must bind exactly that carried evidence.

The slot then carries three canonical bindings:

- a transcript commitment over the transcript surface,
- a challenge commitment over the challenge surface,
- exactly one canonical outcome object: `AsymptoteObserverCanonicalClose` or `AsymptoteObserverCanonicalAbort`.

The collapse rule is close-or-abort and challenge-dominant:

- **SealedFinal** if **BaseFinal** holds, the deterministic assignment set is fully covered by valid canonical `Ok` transcripts, the transcript and challenge commitments verify, the challenge surface is empty, and the canonical close object is valid.
- **Abort** if the transcript and challenge commitments verify, the challenge surface is non-empty, and the canonical abort object is valid.
- **Pending** otherwise.

Challenge dominance is a theorem-level veto, not an operational alarm. A valid challenge collapses the slot to abort before any penalty, governance, or membership aftermath is considered.

This makes observer sealing structurally parallel to canonical ordering: public evidence, canonical commitments, a unique positive object, a unique negative object, and deterministic local close-or-abort derivation from the same public surface.

10.5 Divergence signals and escalation

The sealing plane may also emit divergence signals, including:

- conflicting guardian certificates,
- witness omission,
- stale registry use,
- stale checkpoints,
- transparency-log consistency failure.

These signals support **Escalated** and **Invalid** outcomes and give operators a fail-closed evidence trail.

10.6 Sealed effects and replay-safe release

Irreversible effects are gated by finality tier. The effect plane distinguishes between:

- **BaseFinal** effects, which are allowed on the fast path when policy permits,
- **SealedFinal** effects, which require a valid **SealedFinalityProof**.

For proof-enabled irreversible effects, **Asymptote** carries a self-contained **SealObject** whose current instantiation includes:

- an **EffectIntent**,
- an **EffectPublicInputs** object, including **canonical_collapse_hash**,
- an **EffectProofEnvelope**,
- a replay-safe nullifier.

In the runtime, **SealedFinal** secure-egress requests carry both the **SealedFinalityProof** and the slot's **CanonicalCollapseObject**. The guardian-side seal builder and downstream receipt verifier recompute the same **canonical_collapse_hash** from that collapse object together with the proof-carried observer surface, so a challenge-dominated or otherwise mismatched slot cannot authorize irreversible release.

The release rule is

$$\text{Release}(e) \iff \text{BaseFinal}(\text{slot}(e)) \wedge \text{SealedFinal}(\text{slot}(e)) \wedge \text{VerifySealObject}(e) \wedge \text{CollapseHash}(e) = \text{Hash}(\text{Collapse}(e))$$

This keeps the hot path sparse while making released consequences deterministic and replay-safe.

10.7 Deterministic observer theorem

The deterministic **Asymptote** observer model now proves the following kernel:

- base certificates are unique per slot,
- canonical observer close objects are unique per slot,
- canonical observer close and canonical observer abort cannot coexist for the same slot,
- a challenge-dominated slot cannot produce a valid sealed release.

Theorem 5 (Deterministic Equal-Authority Observer Sealing). *Assume sound deterministic observer assignment, signature-sound guardian-backed observer transcripts and challenges, canonical transcript and challenge commitments, append-only registry and log publication once published, and deterministic transcript/challenge derivation over the closed public observer surface. Then arbitrary behavior by the rest of the validator set cannot create a conflicting valid canonical observer close, cannot make canonical close and canonical abort coexist for the same slot, and cannot authorize a valid sealed irreversible effect release for a challenge-dominated slot.*

This theorem is the observer-lane analogue of canonical ordering’s public certificate theorem: public evidence, canonical commitments, negative-authority dominance, and deterministic local close-or-abort derivation from the same public observer surface.

Under the accountable publication rules of Section 5, the same observer challenges are also penalty-bearing objective faults. Unsafe positive close attempts therefore become not only rejectable, but accountable and epoch-removing.

10.8 Legacy sampled comparison

Before **CanonicalChallengeV1**, the observer lane was typically described through a sampled affirmative bound. Let:

- h be the honest fraction of equal-authority validators,
- A_s^j be the sampled observer committee for slot s and round j ,
- $k_j = |A_s^j|$,
- $p_{\text{beacon_bias}}$ be the probability of sufficient beacon bias,
- $p_{\text{veto_suppression}}$ be the probability that a valid veto surface is fully suppressed.

Then the old affirmative sampled lane admitted the analytical bound

$$P_{\text{unsafe}}(s) \leq p_{\text{beacon_bias}} \cdot p_{\text{veto_suppression}} \cdot \prod_j (1 - h)^{k_j}.$$

For constant committee size k across r rounds:

$$P_{\text{unsafe}}(s) \leq p_{\text{beacon_bias}} \cdot p_{\text{veto_suppression}} \cdot (1 - h)^{kr}.$$

That formula remains a useful historical comparison, but it is no longer the normative theorem for the observer lane. Under **CanonicalChallengeV1**, sampling determines duties, not safety. Safety comes from the canonical public transcript surface, the canonical public challenge surface, and the close-or-abort split that those surfaces induce.

The runtime publishes the canonical observer transcript, commitment, challenge, challenge commitment, and close-or-abort objects as ordinary **guardian_registry** transactions after the sealed header is formed. The same surface is carried inside the proof for immediate block verification.

11 NestedGuardian: Witness-Augmented Research Mode

`NestedGuardian` explores a layered threshold construction:

- validators run the AFT deterministic message flow,
- each proposal carries its guardian committee certificate,
- external witness committees cross-check the slot,
- chain state anchors witness assignments, witness manifests, and slashing evidence.

The instantiated runtime scope includes:

- deterministic witness assignment from the active witness set,
- registered on-chain witness manifests,
- witness certificates bound into guardianized slot certificates,
- safety proofs for the witness-augmented kernel,
- bounded model checking for reassignment, outage, and checkpoint transitions.

The instantiated proof package consists of:

- safety formalization,
- bounded operational model checking for reassignment, outage, and checkpoint transitions,
- a discharged composed-liveness bridge under the target eventual-fair scheduler through the recurrence/reduction/totality/collapse chain together with a matching persistent churn/restart simulator.

`NestedGuardian` is therefore a witness-augmented mode with a finished theorem bridge. It is experimental as an operational mode, not as a theorem crutch. Its witness-coded family supplies an explicit constructive recovery carrier and completed bridge artifacts; it does not reduce the promoted AFT theorem surface to an experimental-only claim. That constructive recovery carrier is a witness-coded publication-bundle recovery contract with compact hot-path bindings and cold-path registry-driven close-or-abort resolution. The abstract coded recovery-family contract is realized by the parametric `SystematicXorK0fKPlus1V1` parity family plus the bounded true non-parity `SystematicGf256K0fNV1` family with at least two parity shares, implemented with systematic Cauchy-style coefficients and exercised in bounded 2-of-4, 3-of-5, 3-of-7, 4-of-6, and 4-of-7 lanes. These lanes resolve from threshold-many public reveals, objective recovered-support conflict, or deterministic missingness. The bounded 3-of-5, 4-of-6, and 4-of-7 lanes restore or exceed threshold-support overlap while the bounded 3-of-7 lane shows that recovered-support conflict remains fail-closed even when overlap is absent. A bounded conformance harness checks that every threshold-sized reveal subset reconstructs and every below-threshold subset fails across the shipped XOR and GF256 realizations of that contract. Those recovered reveals lift deterministically into an explicit positive close-extraction payload and then into an explicit extractable bulletin-surface payload that binds the verifying publication bundle, verifying canonical bulletin close, canonical bulletin-availability bytes, and sorted bulletin entries.

The completed bounded family also carries the ordinary durable and restart surfaces. A bounded recovered-only two-slot harness shows that consecutive recovered surfaces derive the ordinary publication-frontier predecessor chain and the same authoritative `CanonicalCollapseObject` predecessor chain without the ordinary publication-bundle lane. A second bounded recovered-only harness shows a close/abort/close bulletin window with a recovered omission-dominated middle slot: recovered data alone materializes the ordinary positive closes and extracted bulletin surfaces on the outer slots, materializes the ordinary omission-dominated abort in the middle while keeping omission evidence and recovered bulletin entries public, and preserves the same authoritative collapse continuity chain. A bounded read-side harness shows that the same recovered-only durable chain yields the same compact replay-prefix / state-root continuity view that the ordinary lane exposes plus a bounded recovered canonical-header prefix. The execution module verifies that replay tip as the same restart anchor the ordinary lane uses, retains both recovered prefixes in execution state, and uses the recovered canonical-header ancestry for bounded anchored reads plus parent-block / parent-state-root continuity after restart when ordinary recent blocks are absent.

Validator consensus orchestration derives the same bounded recovered restart parent anchor from the recovered canonical-header / collapse surface when the ordinary committed block is absent, and the `GuardianMajority` engine consumes that bounded recovered canonical-header prefix as restart-time parent/QC context for synthetic parent-height QC continuity when the full committed block is absent. The same recovered surface yields a bounded recovered certified-header window plus a restart-only recovered `BlockHeader` / QC cache window for restart-time header / QC lookup plus bounded QC-certified recovered branch reconciliation over the configured local five-entry window. Those windows fold by exact overlap with a configurable budget, exercised at five windows into a longer seventeen-step recovered certified branch without ordinary committed headers. Those same cold-path loaders consume a configurable exact-overlap segment budget, exercised at four exact-overlap segments into a longer fifty-three-step recovered certified branch with direct registry-extraction parity and explicit interior-overlap conflict rejection. Those same restart loaders also compose two overlapping four-segment exact-overlap folds into a longer eighty-nine-step recovered certified branch with direct registry-extraction parity and explicit inter-fold overlap conflict rejection, and tests exercise the same recursive carrier at three overlapping four-segment folds into a one-hundred-twenty-five-step recovered certified branch. The runtime has a bounded conformance harness over one, two, and three stitched segment folds, matching the same production-loader / registry-extraction carrier at the corresponding fifty-three-step, eighty-nine-step, and one-hundred-twenty-five-step branches.

The restart path also pages older exact-overlap segment folds on demand beneath that bounded suffix via an overlap-checked cursor while keeping only the bounded recent suffix plus the streamed page live in engine caches, and tests exercise that paged carrier at a longer two-hundred-thirty-three-step recovered certified branch with direct registry-extraction parity plus explicit duplicate-page, missing-gap, and late-page overlap rejection. Restart therefore streams recovered certified ancestry until target height or recovered-history exhaustion without a theorem-relevant fixed depth bound.

The archival internalization layer uses an active archived-recovered-history profile naming retention horizon, exact-overlap paging geometry, segment / fold geometry, checkpoint update rule, and profile evolution. Archived segments, archived restart pages, archival checkpoints, and archival retention receipts bind to historically referenced profile hashes and governing activation hashes under a published profile-activation chain before archived continuation is trusted, and ordinary canonical collapse history names the archived checkpoint / activation pair that archived restart is authorized to follow. Archived replay and archived publication correctness are historical and index-free: they follow the canonical-collapse-anchored or object-carried activation hash plus predecessor/checkpoint history rather than whichever archived activation is merely latest in local state.

When the recovered certificate carries objective omission proofs, the same recovered lane routes into the ordinary **OmissionDominated** abort path while keeping omission evidence and recovered bulletin entries public. The recovery-family contract, canonical-collapse / replay retrievability anchor, and recovered-state historical retrievability surface are therefore part of ordinary endogenous AFT history rather than a narrower theorem-side qualifier. The directly discharged recurrence/reduction/totality/collapse chain closes the former distance to the relay-free deterministic all-but-one ($n-1$ of n) Byzantine safety theorem in M_{AFT} , so no recoverability or architecture-specific semantic gap remains at the AFT theorem surface.

12 Security and Assumption Surface

The AFT family depends on explicit assumptions. They are not optional prose.

Layer	Required assumptions	Failure consequence
Base finality	current manifests, current epoch state, majority thresholds, guardian non-equivocation, valid checkpoints, enough validators and guardians to reach quorum	conflicting base-finality evidence or loss of progress
Canonical ordering	objective publication, compact publication-frontier binding and contradiction rules, canonical cutoff, deterministic eligibility, append-only bulletin commitment, proof soundness, protocol-native retrievability profile / manifest / custody / challenge surface, canonical bulletin-close extraction-or-abort, cheap omission verification, accountable omission publication	loss of compact frontier consistency, deterministic extraction-or-abort, accountability, or the ordering theorem
Sealed finality	valid asymptote policy, current witness seed, valid witness or observer assignments, checkpoint freshness, valid log consistency proofs, deterministic transcript/challenge derivation, accountable challenge publication	incorrect sealed release, loss of accountability, or fail-closed stalling
Sealed effects	correct finality-tier policy, sound verifier metadata, nullifier uniqueness, intent/public-input binding	replay or unsafe irreversible effect release
NestedGuardian	sound witness assignment and registration, correct reassignment handling, witness committee non-equivocation	degraded safety or open liveness behavior

The family-level discipline is simple: if an assumption fails, AFT should degrade to stalling, aborting, or withholding release before it degrades to unsafe irreversible behavior.

13 Formal Results

The repository’s machine-checked surfaces align with the protocol family as follows:

Formal module	Core proved results
GuardianMajorityProof GuardianMajority	QuorumCertificatesIntersect, InvariantImpliesSafety executable bounded checks for finalization witness soundness and slot safety InvariantImpliesCertifiedUniqueness, InvariantImpliesRecoverability,
CanonicalOrderingProof CanonicalOrdering	InvariantImpliesOmissionDominates executable bounded checks for publication, cutoff, certification, omission, and recoverability InvariantImpliesBaseSafety, InvariantImpliesCanonicalCloseSafety, InvariantImpliesCloseAbortExclusive,
AsymptoteProof Asymptote	InvariantImpliesAbortDominatesSealed executable bounded checks for transcript publication, challenge publication, canonical close, and canonical abort witness-assignment soundness, witness-certificates-stay-assigned,
NestedGuardianProof NestedGuardian	InvariantImpliesSafety executable bounded checks for reassignment, outage, and checkpoint admissibility

Every formal artifact named in this section, and every additional proof artifact cited by file name below, is embedded verbatim in Appendix A so that the served yellow paper remains self-contained outside the repository.

These formal surfaces prove the deterministic safety kernels that the broader classical-agreement bridge builds on. The full promoted all-but-one ($n - 1$ of n) Byzantine agreement sentence — conditional on the AFT model delta — is then completed by the recurring, reduction, totality, and collapse layers discussed above, rather than by these module-local safety kernels in isolation. Compact publication-frontier summaries internalize same-slot and predecessor consistency on the live path, while the protocol-native retrievability plane internalizes closed-bulletin extraction-or-abort as an endogenous part of the realizing architecture rather than a purely algebraic consequence of the authenticated message transcript alone.

The accountable-adversary variant therefore composes these formal kernels with runtime accountability rules rather than replacing them: replay-deduplicated objective fault publication and optional policy-controlled membership updates live in the implementation and policy layer above the proved deterministic core.

13.1 Complete omission-dominance witness artifact

The formal package includes an executable TLC witness module, `internal-docs/architecture/protocols/aft/f` paired with `internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingC`. The exact module and configuration are embedded in Appendix A.5.1 and Appendix A.5.2; the summary below highlights the executed public-evidence shape that the witness reaches.

Executed witness trace. The module intentionally asks TLC to violate the state predicate `NoOmissionDominanceWitness` so that the checker emits a concrete public evidence trace. The executed trace is:

1. `PublishStep`: tx1 enters slot 1’s bulletin surface.
2. `PublishStep`: tx2 enters the same bulletin surface.
3. `CloseCutoffStep`: the canonical cutoff closes and the recoverable set becomes $\{tx1, tx2\}$.
4. `CandidateCertifyStep`: a candidate certificate for $\{tx1\}$ appears.
5. `ProveOmissionStep`: an omission proof for tx2 against that candidate certificate appears.

The same formal package carries a bounded recursive-continuity model, `internal-docs/architecture/protocols` paired with `internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseR`. The exact source of both artifacts is embedded in Appendix A.5.3 and Appendix A.5.4. That model captures the reference `HashPcdV1` continuity carrier directly: deterministic recursive proof steps, predecessor-proof hashing, `CanonicalCollapseExtensionCertificate` carriage, and the rule that non-genesis header admission depends on the anchored predecessor proof relation. The runtime exposes a `SuccinctSp1V1` backend seam for those same recursive public inputs, and that seam is exercised by consensus verification and durable-state gating. The formal model still stops at the reference carrier rather than mechanizing a cryptographic recursive accumulator or ZK backend.

At the final state, the incomplete candidate certificate remains *unadmitted* while the omission proof is present. In the executable model this happens because the admitted object must already equal the canonical set; the trace therefore does not replace the theorem. Its purpose is narrower and useful: it gives a bounded, executed witness of the exact public-evidence shape from which omission dominance follows. Readers can run the trace directly and inspect the six-state witness without reconstructing the paper’s argument by hand.

14 Implementation Correspondence

This section is non-normative. It names the exact runtime symbols that carry the protocol objects defined above.

14.1 Core ordering symbols

The core ordering data structures and free functions are public Rust symbols re-exported through `ioi_types::app`. They are the runtime carriers for the abstract ordering objects and verifier rules defined above.

Abstract object	Current runtime symbols
Bulletin surface commitment	<code>BulletinCommitment</code> , <code>BulletinSurfaceEntry</code>
Compact publication availability binding	<code>PublicationAvailabilityReceipt</code>
Compact publication frontier	<code>PublicationFrontier</code>
Publication-frontier contradiction	<code>PublicationFrontierContradiction</code>
Canonical ordering proof families	<code>CanonicalOrderProofSystem</code> , with concrete variants <code>HashBindingV1</code> and <code>CommittedSurfaceV1</code>
Canonical public-input tuple	<code>CanonicalOrderPublicInputs</code>
Live succinct witness payload	<code>CommittedSurfaceCanonicalOrderProof</code>
Omission evidence	<code>OmissionProof</code>
Order certificate	<code>CanonicalOrderCertificate</code>
Canonical ordering function	<code>canonical_order_tx_hashes</code> <code>canonical_publication_availability</code> <code>_receipt_hash</code> , <code>canonical_publication_frontier_hash</code> , <code>canonical_publication_frontier</code>
Publication-frontier hashing	<code>_contradiction_hash</code> <code>build_publication_availability_receipt</code> , <code>build_publication_frontier</code> , <code>verify_publication_frontier</code> , <code>verify_publication_frontier_contradiction</code>
Publication-frontier construction / verification	<code>canonical_order_public_inputs</code> , <code>canonical_order_public_inputs_hash</code> <code>build_reference_canonical_order</code>
Public-input hashing	<code>_proof_bytes</code> <code>build_committed_surface_canonical</code>
Reference proof construction	<code>_order_certificate</code>
Live certificate construction	<code>verify_canonical_order_certificate</code>
Certificate verification	

14.2 Guardian and sealing symbols

The guardian, witness, observer, and sealed-effect objects are likewise public Rust symbols re-exported through `ioi_types::app`. They encode the exact evidence surface checked by validators and downstream gateways.

Abstract object	Current runtime symbols
Guardian slot certificate	<code>GuardianQuorumCertificate</code>
Witness certificate	<code>GuardianWitnessCertificate</code>
Equal-authority observer assignment	<code>AsymptoteObserverAssignment</code>
Correlation budget	<code>AsymptoteObserverCorrelationBudget</code>
Observer observation request	<code>AsymptoteObserverObservationRequest</code>
Observer transcript	<code>AsymptoteObserverTranscript</code>
Observer transcript commitment	<code>AsymptoteObserverTranscriptCommitment</code>
Observer challenge	<code>AsymptoteObserverChallenge</code> , with kinds <code>MissingTranscript</code> , <code>TranscriptMismatch</code> , <code>VetoTranscriptPresent</code> , <code>ConflictingTranscript</code> , and <code>InvalidCanonicalClose</code>
Observer challenge commitment	<code>AsymptoteObserverChallengeCommitment</code>
Canonical observer close	<code>AsymptoteObserverCanonicalClose</code>
Canonical observer abort	<code>AsymptoteObserverCanonicalAbort</code>
Finality tier	<code>FinalityTier</code> , with variants <code>BaseFinal</code> and <code>SealedFinal</code>
Collapse state	<code>CollapseState</code> , with variants <code>Pending</code> , <code>BaseFinal</code> , <code>Sealing</code> , <code>Abort</code> , <code>SealedFinal</code> , <code>Escalated</code> , and <code>Invalid</code>
Sealing policy	<code>AsymptotePolicy</code>
Observer statement	<code>AsymptoteObserverStatement</code>
Observer certificate	<code>AsymptoteObserverCertificate</code>
Divergence signal	<code>DivergenceSignal</code> , <code>DivergenceSignalKind</code>
Sealed finality proof	<code>SealedFinalityProof</code>
Observer binding for sealed effects	<code>sealed_finality_proof_observer_binding</code>
Proof-carrying effect object	<code>SealObject</code>

14.3 Verifier, orchestration, and persistence symbols

Verification and orchestration entry points are split across four public namespaces: `ioi_types::app`, `ioi_crypto::sign::guardian_committee`, `ioi_services::guardian_registry`, and the `GuardianMajorityEngine` runtime. Engine-local checks are named here by their exact method identifiers.

Responsibility	Current runtime symbols
Guardian certificate verification	verify_quorum_certificate
Witness certificate verification	verify_witness_certificate
Deterministic observer assignment	derive_asymptote_observer_assignments, derive_asymptote_observer_plan_entries GuardianContainer::observe_asymptote_request, GuardianContainer::request_remote_asymptote _observer_observation,
Observer-side transcript / challenge derivation	GuardianContainer::sign_consensus_with_guardian GuardianMajorityEngine::verify_asymptote _canonical_observer_sealed_finality, GuardianMajorityEngine::verify_asymptote _sealed_finality GuardianMajorityEngine::verify _canonical_order_enrichment; published CanonicalOrderAbort dominates positive order certificates and inconsistent mixed parent state
Sealed finality verification in the engine	GuardianMajorityEngine::verify _publication_frontier_enrichment; published PublicationFrontierContradiction dominates conflicting or stale frontiers
Canonical-order enrichment verification in the engine	GuardianMajorityEngine::verify _publication_frontier_enrichment; published PublicationFrontierContradiction dominates conflicting or stale frontiers
Publication-frontier enrichment verification in the engine	build_committed_surface_canonical _order_certificate during finalization
Block finalization attachment of order certificates	build_publication_frontier during finalization
Block finalization attachment of publication frontier	canonicalize_observer_sealed_finality_proof, publish_canonical_observer_artifacts build_http_egress_seal_object, verify_seal_object GuardianMajorityEngine::verify _guardianized_certificate
Canonical observer proof publication	SafetyGadget, Pacemaker, TimeoutCertificate load_bulletin_commitment, load_canonical_order_certificate GuardianRegistry::load_publication_frontier, GuardianRegistry::load_publication _frontier_contradiction derive_canonical_order_execution_object, derive_canonical_order_public_obstruction CanonicalOrderAbortReason::MissingOrderCertificate, BulletinSurfaceReconstructionFailure, BulletinSurfaceMismatch, InvalidBulletinClose, OmissionDominated, CertificateHeightMismatch, RandomnessMismatch, OrderedTransactionsRootMismatch, ResultingStateRootMismatch, InvalidPublicInputsHash, InvalidBulletinAvailabilityCertificate, InvalidProofBinding derive_canonical_collapse_object, GuardianMajorityEngine::verify_published _canonical_collapse_object
Sealed-effect construction and checking	
Guardianized proposal verification in the engine	
Base-engine safety and timing	
Bulletin and order-certificate state access	
Publication-frontier state access	
Ordering close-or-abort derivation	
Ordering abort taxonomy	
Protocol-wide collapse derivation and verification	
Proposal-surface recursive continuity	BlockHeader::previous_canonical_collapse_commitment_hash, BlockHeader::canonical_collapse_extension_certificate, CanonicalCollapseExtensionCertificate, CanonicalCollapseCommitment, verify_block_header_canonical_collapse_evidence, ConsensusDecision::ProduceBlock load_bulletin_commitment, load_canonical_order_certificate, load_canonical_order_abort, load_canonical_collapse_object publish_aft_publication_frontier@v1, publish_aft_canonical_order_artifact_bundle@v1, publish_aft_canonical_order_abort@v1, publish_aft_canonical_collapse_object@v1, report_aft_omission@v1, publish_asymptote_observer_transcript@v1, publish_asymptote_observer_transcript_commitment@v1, report_asymptote_observer_challenge@v1, publish_asymptote_observer_challenge_commitment@v1, publish_asymptote_observer_canonical_close@v1,
Bulletin, order, and collapse state access	

These symbols are named here so that the paper maps directly onto the current runtime without making the reader chase separate prose specifications. In the implementation, validator finalization publishes a protocol-visible `CanonicalCollapseObject` through `publish_aft_canonical_collapse_object@v1` after local post-commit header enrichment, `guardian_registry` exposes `load_canonical_collapse_object`, and `GuardianMajorityEngine::verify_publication` cross-checks any published copy against the proof-carried header surface when it is available. The externally finalized AFT commit path persists the same object keyed by slot height, rather than treating ordering and sealing close-or-abort evidence as loose enrichments detached from state persistence. The same post-commit path also builds `PublicationFrontier` via `build_publication_frontier` and publishes it through `publish_aft_publication_frontier@v1`; `GuardianMajorityEngine::verify_publication_frontier_enrichment` rechecks same-slot binding, predecessor linkage, and any dominating contradiction object. In the current proposal-surface continuity slice, validator orchestration now signs both `previous_canonical_collapse_commitment_hash` and a typed `CanonicalCollapseExtensionCertificate` into each produced AFT block header. In the current clean-break runtime slice, `CanonicalCollapseObject` itself carries `continuity_accumulator_hash` and `continuity_recursive_proof`, so the certificate now carries the predecessor commitment plus predecessor proof hash rather than an explicit carried ancestor vector, full predecessor collapse object, or predecessor recursive proof object. `Asymptote` leaders stall rather than produce when that required extension certificate is unavailable, proposal verification recursively checks that the carried predecessor commitment hash matches the signed predecessor link, checks that the predecessor commitment’s resulting state root matches the signed parent-state root, and rechecks those carried public inputs against the anchored predecessor collapse object when one is available. QC promotion therefore relies on a reference recursive proof-carrying continuity relation rather than on a recomputed predecessor digest alone, and only treats continuity-linked headers as progress-bearing.

15 Benchmark Context

The performance evidence in this paper has two sources:

- literature-reported baselines for `HotStuff`, `Narwhal/Tusk`, and `Bullshark`,
- the repository’s native `benchmark_throughput` target for matched `GuardianMajority` and `Asymptote` scenarios.

The purpose of this section is architectural positioning, not false equivalence across unmatched environments.

15.1 Paper-reported baselines

System	Reported throughput	Reported latency	Source type
HotStuff baseline	1,800 tx/s	1 s	Narwhal/Tusk comparison setting
Narwhal-HotStuff	130,000 tx/s	under 2 s	Narwhal/Tusk paper
Narwhal-HotStuff with additional workers	up to 600,000 tx/s	n/a	Narwhal/Tusk paper
Tusk	160,000 tx/s	about 3 s	Narwhal/Tusk paper
Bullshark	125,000 tx/s	about 2 s	Bullshark paper

The baseline interpretation is:

- **HotStuff** demonstrates the leader-based partially synchronous baseline,
- **Narwhal-HotStuff** demonstrates the gain from separating dissemination from ordering,
- **Bullshark** demonstrates high-throughput DAG-oriented ordering,
- **AFT** adds a new separation: order surfacing and effect release are no longer identified with dense positive voting.

15.2 Native repository measurements

No performance measurement in this section is load-bearing for any theorem in this paper; the promoted theorem surface stands independently of benchmark tuning, harness state, or optimization maturity.

The corrected native measurements below were produced on March 21, 2026 after bringing the repository’s native **AFT** benchmark path back into sync with the current **GuardianMajority / base_final** harness semantics:

- shared-tip readiness no longer treats a transient `get_status.height = 0` as authoritative when a resilient tip probe already sees committed blocks,
- submission alignment and leader-targeted ingress now use a resilient tip block rather than a potentially stale status height,
- authoritative chain-commit scans start from the resilient pre-submission tip instead of rescanning stale empty-block prefixes,
- sustained throughput for fast probes now stops on the authoritative full-chain commit scan for the highest committed height; sampled committed-status visibility is retained only as an auxiliary lag diagnostic,
- the benchmark surface now exposes alignment, spill, and replay diagnostics explicitly so missed due blocks or replay churn are not hidden behind a single TPS number.

The table below reports the current matched bulk-throughput proof point for the native **GuardianMajority** 4-validator **base_final** scenario under that corrected harness. This is a repository-native **AFT** proof point on the Jellyfish state path with a ‘1 s’ block interval, not an apples-to-apples claim against published Narwhal/Tusk/Bullshark environments.

scenario	validators	attempted	accepted	committed	blocks	inj. tps	sust. tps	p50	p95	p99	sealed p50	sealed p95
guardian_majority_4v_base_final_bulk	4	16384	16384	16384	1	21370.89	9647.66	1293.45	1603.73	1695.24	–	–

These measurements imply four things:

- the earlier single-digit and low-hundreds TPS reports were artifacts of benchmark-path bugs rather than of the protocol surface itself,
- the current repository now clears the lower edge of the intended **8k-16k TPS** range in a matched native 4-validator **GuardianMajority** bulk probe,

- on this corrected probe the batch lands in a single block, so the main remaining AFT work is ceiling-raising and aggregate-domain multiplication rather than recovering from a broken harness,
- these values should be read as corrected repository baselines, not as optimized ceilings and not as apples-to-apples superiority claims over published **Narwhal/Tusk/Bullshark** environments.

16 Related Work

Geeq’s Proof of Honesty (PoH) is important prior work in the “99%” (their figure, not an AFT claim) direction: it argues for globally honest and canonical chain recovery in pure software under assumptions including at least one honest node and an honest relay, and it uses public evidence plus user-side verification to push beyond ordinary dense-vote intuition [9]. AFT differs in a key respect: it is relay-free and coordinator-free. Correctness does not depend on any designated honest relay, broadcaster, or privileged recovery role. Instead, durable ordering and sealed effects collapse through protocol-native public-evidence objects whose negative forms are immediately decisive.

Narwhal/Tusk and Bullshark separate dissemination from ordering, but they still ground ordering safety in dense positive certificates and quorum-intersection arguments over the ordered object itself [2, 3]. HoneyBadgerBFT and Dumbo move to asynchronous common-subset or MVBA-style agreement, but they still rely on classical Byzantine-majority thresholds rather than a public-state-continuity theorem over a canonical public surface [4, 5]. Chainlink CCIP and optimistic cross-chain mechanisms add secondary monitoring layers, risk-management committees, or challenge windows to detect and halt bad transfers, but the decisive recovery rule is operational or economic rather than the immediate acceptance of a uniquely valid proof-carrying canonical object recoverable by any equal-authority validator [8]. Accountable subgroup multisigs and finality gadgets add equivocation evidence and slashing, yet still keep dense subgroup votes as the positive safety object [6, 7]. AFT differs by making ordering and sealed release relay-free public-evidence close-or-abort objects with dominant negative proofs: honest validators that observe the same closed public boundary derive the same decisive object, and publication is downstream of that deterministic derivation rather than its source.

17 Claims and Non-Claims

17.1 Claims

This paper claims:

- a production guardian-backed base-finality model in which validator quorums are composed with guardian committee admissibility,
- proof-carrying canonical ordering with omission dominance and uniqueness, plus protocol-native endogenous retrievability,
- compact signed publication-frontier summaries and short contradiction objects that internalize same-slot and predecessor consistency on the live path,
- a public-state-continuity formulation in which closed slot boundaries admit at most one positive public execution object and conflicting candidates admit short objective obstruction witnesses,

- all-but-one ($n - 1$ of n) equal-authority ordering consensus in the AFT model delta, under explicit assumptions,
- deterministic sealed-effect collapse with challenge dominance,
- a relay-free, coordinator-free, pure-software model-delta theorem yielding deterministic all-but-one ($n - 1$ of n) Byzantine safety for durable ordering and sealed-effect safety over the explicit bulletin / retrievability surface, conditional on the AFT model delta,
- that the whole AFT stack breaks the classical indefinite-suppression lower bound, in the AFT model delta, by identifying the first illegal adversary step and realizing the visibility delta without a trusted relay,
- a witness-augmented mode for layered threshold constructions,
- machine-checked deterministic safety kernels for the major protocol surfaces.

17.2 Non-claims

This paper does **not** claim:

- that public state continuity arises for free inside the classical DLS / PBFT partially synchronous message-passing model,
- that the classical $n > 3f / f < n/3$ lower bound has been refuted inside that bare model; the claim is instead that AFT operates in the explicit M_{AFT} model delta where the adversary cannot indefinitely suppress published admissible artifacts from correct parties after stabilization. Remove that visibility resource, or realize it circularly through the same Byzantine-majority relay set, and the paper is no longer claiming the same result. This is not a retreat; it is the exact location of the break. AFT does not merely vary a parameter inside the dense-vote carrier. It replaces the carrier with direct dissemination plus compressed public echo and states the new proof obligation at theorem surface,
- freedom from bulletin-publication, protocol-native retrievability obligations, or proof-soundness assumptions,
- that compact publication-frontier summaries make the bulletin surface self-extracting inside the classical authenticated-message model,
- a full asynchronous liveness theorem under arbitrary registry, log, witness, and outage schedules,
- an apples-to-apples measured throughput superiority over HotStuff, Narwhal, Tusk, or Bullshark from the current repository alone.

17.3 Model-delta public-state-continuity theorem

The runtime supports a stronger theorem than the older accountable summary because the mechanism and system model are now stated directly at theorem surface.

Theorem 6 (Relay-Free Deterministic All-But-One ($n - 1$ of n) Byzantine Safety in M_{AFT}). *Assume the system model of Section 5, including the direct-dissemination / compressed-echo delta of Lemma 1, guardian non-equivocation, proof soundness, compact signed publication-frontier summaries whose*

same-slot and predecessor-link contradictions are objectively checkable, protocol-native retrievability objects plus canonical bulletin-close extraction-or-abort for the bulletin surface bound by those frontiers, deterministic transcript/challenge derivation for sealing, durable state advancement only through canonical collapse, and irreversible-effect release only through collapse-bound sealed proofs. Then all but one validator ($n - 1$ of n) may behave arbitrarily without creating a conflicting valid durable canonical-order outcome or a conflicting valid sealed-effect release outcome. Honest validators that observe the same closed public boundary derive the same close-or-abort result; publication, penalties, and validator-set updates only accelerate convergence and future hygiene. The classical $n > 3f / f < n/3$ threshold is not load-bearing in M_{AFT} , because the classical adversary's indefinite-suppression step is no longer legal.

The theorem does not claim that DLS / PBFT has been refuted inside M_{DLS} . It claims that AFT is not a member of that model: the first illegal lower-bound adversary step is indefinite suppression of a published admissible artifact from correct parties after stabilization. Bulletin close, publication frontier, retrievability, recovery, restart continuity, canonical collapse, and collapse-bound sealing are not theorem-side caveats; they are the protocol-internal realizing mechanism of M_{AFT} . A criticism that AFT “changes the model” is therefore descriptive rather than fatal unless it identifies circular realization of the visibility resource or a hidden relay, coordinator, TEE, bootstrap oracle, or external recovery service.

The theorem is stronger than the older accountable summary because invalid positive objects are not merely rejectable or penalty-bearing. They are already dominated by deterministic negative objects and collapse-gated durable state before any penalty logic runs. Readers trained on dense-vote BFT will be tempted to treat the absence of quorum-intersection safety as disqualifying. In AFT, that is precisely the point: authority is carried by uniquely valid public execution objects and omission-dominant contradiction witnesses, with durable advancement gated by canonical collapse. Compact publication frontiers keep the live hot path cheap by internalizing consistency and lineage checks, while the protocol-native retrievability plane discharges deterministic extraction-or-abort on the full bulletin surface.

The recurrence / reduction / totality / collapse chain is not narrative glue. Its exact artifact chain is embedded verbatim in Appendix A, ending in the classical-agreement collapse wrapper itself.

Theorem 7 (Realizing-Mechanism Statement). *The new publication-frontier slice is not the theorem by itself; the theorem is the promoted AFT model-delta sentence, carried by the full AFT public-evidence continuity architecture. Compact signed publication-frontier summaries, compact availability receipts, short frontier contradiction objects, endogenous retrievability profiles, manifests, custody assignments, custody receipts, custody responses, retrievability challenges, canonical extraction-or-abort, canonical collapse, and historical retrievability together realize the direct-dissemination / compressed-echo visibility resource stated above. Removing that protocol-native retrievability plane removes part of the realizing mechanism rather than merely shaving off an optional refinement.*

Proposition 2 (Scoped Witness-Coded Constructive Corollary). *Assume the hypotheses of the relay-free deterministic all-but-one ($n - 1$ of n) Byzantine safety theorem in M_{AFT} , plus sound witness assignment and witness certificate binding, a completed witness-coded recovery family with compact hot-path bindings and cold-path share delivery / reveal / reconstruction, and availability of the retained recovered publication surface together with any older recovered-history pages needed for restart-time extraction. Then the **NestedGuardian** witness-coded carrier deterministically materializes the same positive close or canonical abort surface, the same authoritative collapse chain, the same replay-prefix / restart-anchor surface, and the same restart-time canonical-header*

/ certified-header / block-header ancestry surface as the ordinary lane for the completed recovery families. Restart can therefore stream recovered certified ancestry until target height or recovered-history exhaustion without a theorem-relevant fixed depth bound, while keeping only compact bindings on the hot path and bounded-memory paging on the restart path.

This runtime result is stronger than the separation closeout because it no longer stops at “compact frontiers bind a recoverability claim.” It shows that the live AFT stack can re-materialize the same durable and restart-critical surfaces without violating the throughput guardrails, while keeping deeper recovered history inside ordinary endogenous AFT state.

This is one constructive route to Byzantine agreement in a realizable permissioned model delta rather than a weaker contrast class. In the classical dense-vote presentation, worst-case progress is usually phrased in terms of $> 2/3$ honest participation and quorum intersection as the main safety mechanism. In AFT’s realization, authority is obtained instead through uniquely valid public-evidence objects plus dominant negative proofs, without trusted relays, privileged coordinators, TEEs, or dense positive quorum intersection. The live publication-frontier slice internalizes compact same-slot and predecessor consistency on the hot path, while explicit recoverability of the bound bulletin surface, canonical collapse, and historical retrievability complete the same AFT model-delta theorem. Publication and accountability serve as reinforcement rather than the source of correctness. Liveness then follows from direct authenticated dissemination, compressed echo visibility, and eventual publication of the protocol-defined artifacts, and the recurrence/reduction/totality/collapse chain closes that route formally. That is the precise sense in which the classical $n > 3f / f < n/3$ barrier stops being load-bearing here: the promoted theorem no longer grants the adversary indefinite suppression of published admissible information.

The correct one-line summary is:

AFT is a proof-carrying, guardian-backed, omission-dominant consensus and settlement family in which base finality, canonical ordering, and irreversible-effect release are separated and then recomposed through shared evidence.

A Embedded Formal Artifacts

This appendix embeds verbatim every proof artifact referenced by name in the yellow paper, together with the repository path from which it is sourced. The goal is to keep the served paper self-contained even when it is read outside the repository. The artifacts are grouped by protocol surface and rendered with line numbers so the appendix reads as a standalone proof bundle rather than as a raw repository dump.

A.1 GuardianMajority

Core guardian-backed base-finality model and proof.

A.1.1 GuardianMajorityProof.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/guardian_majority/GuardianMajorityProof.tla}`

```
1  ---- MODULE GuardianMajorityProof ----
2  EXTENDS Naturals, FiniteSets, TLAPS
3
4  CONSTANT Validators, Blocks, Slots, Epochs, QuorumSize
5
6  ASSUME QuorumIntersection ==
7    \A S, T \in SUBSET Validators :
8      /\ Cardinality(S) >= QuorumSize
9      /\ Cardinality(T) >= QuorumSize
10     => S \cap T # {}
11
12  VARIABLES votes, certs
13
14  vars == <<votes, certs>>
15
16  VoteDomain == Validators \X Slots \X Blocks \X Epochs
17  CertDomain == Slots \X Blocks \X Epochs \X (SUBSET Validators)
18
19  Init ==
20    /\ votes = {}
21    /\ certs = {}
22
23  HasVote(v, s) ==
24    \E b \in Blocks, e \in Epochs : <<v, s, b, e>> \in votes
25
26  VoteStep ==
27    \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs :
28      /\ ~HasVote(v, s)
29      /\ votes' = votes \cup {<<v, s, b, e>>}
30      /\ UNCHANGED certs
31
32  CertifyStep ==
33    \E s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
34      /\ Cardinality(Q) >= QuorumSize
35      /\ \A v \in Q : <<v, s, b, e>> \in votes
36      /\ certs' = certs \cup {<<s, b, e, Q>>}
37      /\ UNCHANGED votes
38
39  Next == VoteStep \/ CertifyStep
40
41  TypeInvariant ==
42    /\ votes \subseq VoteDomain
43    /\ certs \subseq CertDomain
44
45  NoDualVotes ==
46    \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
47      /\ <<v, s, b1, e1>> \in votes
48      /\ <<v, s, b2, e2>> \in votes
49      => /\ b1 = b2
```

```

50     /\ e1 = e2
51
52 CertSoundness ==
53   \A s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
54     <<s, b, e, Q>> \in certs
55     => /\ Cardinality(Q) >= QuorumSize
56     /\ \A v \in Q : <<v, s, b, e>> \in votes
57
58 Invariant == TypeInvariant /\ NoDualVotes /\ CertSoundness
59
60 Safety ==
61   \A s \in Slots,
62     b1 \in Blocks,
63     b2 \in Blocks,
64     e1 \in Epochs,
65     e2 \in Epochs,
66     Q1 \in SUBSET Validators,
67     Q2 \in SUBSET Validators :
68     /\ <<s, b1, e1, Q1>> \in certs
69     /\ <<s, b2, e2, Q2>> \in certs
70     => /\ b1 = b2
71     /\ e1 = e2
72
73 Spec == Init /\ [] [Next]_vars
74
75 THEOREM InitImpliesTypeInvariant == Init => TypeInvariant
76   BY SMTT(30) DEF Init, TypeInvariant, VoteDomain, CertDomain
77
78 THEOREM InitImpliesNoDualVotes == Init => NoDualVotes
79   BY DEF Init, NoDualVotes
80
81 THEOREM InitImpliesCertSoundness == Init => CertSoundness
82   BY DEF Init, CertSoundness
83
84 THEOREM InitImpliesInvariant == Init => Invariant
85   BY InitImpliesTypeInvariant, InitImpliesNoDualVotes,
86     InitImpliesCertSoundness DEF Invariant
87
88 THEOREM VotePreservesTypeInvariant == TypeInvariant /\ VoteStep => TypeInvariant'
89   BY SMTT(30) DEF TypeInvariant, VoteStep, HasVote, VoteDomain, CertDomain
90
91 THEOREM VotePreservesNoDualVotes == NoDualVotes /\ VoteStep => NoDualVotes'
92   BY SMTT(30) DEF NoDualVotes, VoteStep, HasVote
93
94 THEOREM VotePreservesCertSoundness == CertSoundness /\ VoteStep => CertSoundness'
95   BY DEF CertSoundness, VoteStep
96
97 THEOREM VotePreservesInvariant == Invariant /\ VoteStep => Invariant'
98   BY VotePreservesTypeInvariant, VotePreservesNoDualVotes,
99     VotePreservesCertSoundness DEF Invariant
100
101 THEOREM CertifyPreservesTypeInvariant == TypeInvariant /\ CertifyStep => TypeInvariant'
102   BY SMTT(30) DEF TypeInvariant, CertifyStep, VoteDomain, CertDomain
103
104 THEOREM CertifyPreservesNoDualVotes == NoDualVotes /\ CertifyStep => NoDualVotes'
105   BY DEF NoDualVotes, CertifyStep
106
107 THEOREM CertifyPreservesCertSoundness == CertSoundness /\ CertifyStep => CertSoundness'
108   BY SMTT(30) DEF CertSoundness, CertifyStep
109
110 THEOREM CertifyPreservesInvariant == Invariant /\ CertifyStep => Invariant'
111   BY CertifyPreservesTypeInvariant, CertifyPreservesNoDualVotes,

```

```

112     CertifyPreservesCertSoundness DEF Invariant
113
114 THEOREM StepPreservesInvariant == Invariant /\ Next => Invariant'
115   BY VotePreservesInvariant, CertifyPreservesInvariant DEF Next
116
117 THEOREM StutterPreservesTypeInvariant ==
118   TypeInvariant /\ UNCHANGED vars => TypeInvariant'
119   BY DEF TypeInvariant, vars
120
121 THEOREM StutterPreservesNoDualVotes ==
122   NoDualVotes /\ UNCHANGED vars => NoDualVotes'
123   BY DEF NoDualVotes, vars
124
125 THEOREM StutterPreservesCertSoundness ==
126   CertSoundness /\ UNCHANGED vars => CertSoundness'
127   BY DEF CertSoundness, vars
128
129 THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
130   BY StutterPreservesTypeInvariant, StutterPreservesNoDualVotes,
131     StutterPreservesCertSoundness DEF Invariant
132
133 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
134   BY StepPreservesInvariant, StutterPreservesInvariant DEF vars
135
136 THEOREM QuorumCertificatesIntersect ==
137   \A s \in Slots,
138     b1 \in Blocks,
139     b2 \in Blocks,
140     e1 \in Epochs,
141     e2 \in Epochs,
142     Q1 \in SUBSET Validators,
143     Q2 \in SUBSET Validators :
144     /\ Invariant
145     /\ <<s, b1, e1, Q1>> \in certs
146     /\ <<s, b2, e2, Q2>> \in certs
147     => Q1 \cap Q2 # {}
148   BY SMTT(60), QuorumIntersection DEF Invariant, CertSoundness
149
150 THEOREM InvariantImpliesSafety == Invariant => Safety
151   BY SMTT(60), QuorumCertificatesIntersect DEF Invariant, Safety, NoDualVotes,
152     CertSoundness
153
154 =====

```

A.1.2 GuardianMajority.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/guardian_majority/GuardianMajority.tla}

```

1  ---- MODULE GuardianMajority ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Blocks, Slots, Epochs, QuorumSize, InitialEpoch
5
6  ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
7
8  ASSUME QuorumIntersection ==
9    \A S, T \in SUBSET Validators :
10     /\ Cardinality(S) >= QuorumSize

```

```

11     /\ Cardinality(T) >= QuorumSize
12     => S \cap T # {}
13
14 VARIABLES votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady,
15     ↪ checkpointLevel
16
17 vars == <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady,
18     ↪ checkpointLevel>>
19
20 CheckpointLevels == 0..2
21
22 VoteEvent(v, s, b, e) == <<v, s, b, e>>
23 Finalization(s, b, e) == <<s, b, e>>
24
25 VoteDomain == {VoteEvent(v, s, b, e) : v \in Validators, s \in Slots, b \in Blocks, e \in Epochs}
26 FinalizationDomain == {Finalization(s, b, e) : s \in Slots, b \in Blocks, e \in Epochs}
27
28 Init ==
29     /\ votes = {}
30     /\ finalized = {}
31     /\ finalizerSets = [f \in FinalizationDomain |-> {}]
32     /\ registryEpoch = [v \in Validators |-> InitialEpoch]
33     /\ manifestEpoch = [v \in Validators |-> InitialEpoch]
34     /\ guardianReady = [v \in Validators |-> TRUE]
35     /\ checkpointLevel = [v \in Validators |-> 1]
36
37 HasVote(v, s) ==
38     \E b \in Blocks, e \in Epochs : VoteEvent(v, s, b, e) \in votes
39
40 CanVote(v, s, b, e) ==
41     /\ v \in Validators
42     /\ s \in Slots
43     /\ b \in Blocks
44     /\ e \in Epochs
45     /\ guardianReady[v]
46     /\ checkpointLevel[v] > 0
47     /\ registryEpoch[v] = e
48     /\ manifestEpoch[v] = e
49     /\ ~HasVote(v, s)
50
51 Vote(v, s, b, e) ==
52     /\ CanVote(v, s, b, e)
53     /\ votes' = votes \cup {VoteEvent(v, s, b, e)}
54     /\ UNCHANGED <<finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady,
55     ↪ checkpointLevel>>
56
57 EligibleVoters(s, b, e) ==
58     {v \in Validators :
59         /\ VoteEvent(v, s, b, e) \in votes
60         /\ guardianReady[v]
61         /\ checkpointLevel[v] > 0
62         /\ registryEpoch[v] = e
63         /\ manifestEpoch[v] = e}
64
65 CanFinalize(s, b, e) ==
66     /\ s \in Slots
67     /\ b \in Blocks
68     /\ e \in Epochs
69     /\ Cardinality(EligibleVoters(s, b, e)) >= QuorumSize
70
71 Finalize(s, b, e) ==
72     /\ CanFinalize(s, b, e)

```

```

70 /\ finalized' = finalized \cup {Finalization(s, b, e)}
71 /\ finalizerSets' = [finalizerSets EXCEPT ![Finalization(s, b, e)] = EligibleVoters(s, b, e)]
72 /\ UNCHANGED <<votes, registryEpoch, manifestEpoch, guardianReady, checkpointLevel>>
73
74 AdoptEpoch(v, e) ==
75 /\ v \in Validators
76 /\ e \in Epochs
77 /\ e >= registryEpoch[v]
78 /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
79 /\ manifestEpoch' = [manifestEpoch EXCEPT ![v] = e]
80 /\ UNCHANGED <<votes, finalized, finalizerSets, guardianReady, checkpointLevel>>
81
82 RegistryRollback(v, e) ==
83 /\ v \in Validators
84 /\ e \in Epochs
85 /\ e < registryEpoch[v]
86 /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
87 /\ UNCHANGED <<votes, finalized, finalizerSets, manifestEpoch, guardianReady, checkpointLevel>>
88
89 ManifestRollback(v, e) ==
90 /\ v \in Validators
91 /\ e \in Epochs
92 /\ e < manifestEpoch[v]
93 /\ manifestEpoch' = [manifestEpoch EXCEPT ![v] = e]
94 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, guardianReady, checkpointLevel>>
95
96 GuardianOutage(v) ==
97 /\ v \in Validators
98 /\ guardianReady[v]
99 /\ guardianReady' = [guardianReady EXCEPT ![v] = FALSE]
100 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, checkpointLevel>>
101
102 GuardianRecovery(v) ==
103 /\ v \in Validators
104 /\ ~guardianReady[v]
105 /\ guardianReady' = [guardianReady EXCEPT ![v] = TRUE]
106 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, checkpointLevel>>
107
108 AdvanceCheckpoint(v, level) ==
109 /\ v \in Validators
110 /\ level \in CheckpointLevels
111 /\ level > checkpointLevel[v]
112 /\ checkpointLevel' = [checkpointLevel EXCEPT ![v] = level]
113 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady>>
114
115 CheckpointRollback(v, level) ==
116 /\ v \in Validators
117 /\ level \in CheckpointLevels
118 /\ level < checkpointLevel[v]
119 /\ checkpointLevel' = [checkpointLevel EXCEPT ![v] = level]
120 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady>>
121
122 Next ==
123 /\ E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs : Vote(v, s, b, e)
124 /\ E s \in Slots, b \in Blocks, e \in Epochs : Finalize(s, b, e)
125 /\ E v \in Validators, e \in Epochs : AdoptEpoch(v, e)
126 /\ E v \in Validators, e \in Epochs : RegistryRollback(v, e)
127 /\ E v \in Validators, e \in Epochs : ManifestRollback(v, e)
128 /\ E v \in Validators : GuardianOutage(v)
129 /\ E v \in Validators : GuardianRecovery(v)
130 /\ E v \in Validators, level \in CheckpointLevels : AdvanceCheckpoint(v, level)
131 /\ E v \in Validators, level \in CheckpointLevels : CheckpointRollback(v, level)

```



```

132
133 TypeInvariant ==
134   /\ votes \subseq VoteDomain
135   /\ finalized \subseq FinalizationDomain
136   /\ finalizerSets \in [FinalizationDomain -> SUBSET Validators]
137   /\ registryEpoch \in [Validators -> Epochs]
138   /\ manifestEpoch \in [Validators -> Epochs]
139   /\ guardianReady \in [Validators -> BOOLEAN]
140   /\ checkpointLevel \in [Validators -> CheckpointLevels]
141
142 NoDualVotes ==
143   \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
144     /\ VoteEvent(v, s, b1, e1) \in votes
145     /\ VoteEvent(v, s, b2, e2) \in votes
146     => /\ b1 = b2
147         /\ e1 = e2
148
149 FinalizationWitnessSoundness ==
150   \A s \in Slots, b \in Blocks, e \in Epochs :
151     Finalization(s, b, e) \in finalized
152     => /\ Cardinality(finalizerSets[Finalization(s, b, e)]) >= QuorumSize
153     /\ \A v \in finalizerSets[Finalization(s, b, e)] :
154       VoteEvent(v, s, b, e) \in votes
155
156 Invariant == TypeInvariant /\ NoDualVotes /\ FinalizationWitnessSoundness
157
158 Safety ==
159   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
160     /\ Finalization(s, b1, e1) \in finalized
161     /\ Finalization(s, b2, e2) \in finalized
162     => b1 = b2
163
164 Spec ==
165   Init /\ [] [Next]_vars
166
167 THEOREM OneInCheckpointLevels == 1 \in CheckpointLevels
168   BY SMT DEF CheckpointLevels
169
170 THEOREM InitImpliesInvariant == Init => Invariant
171 PROOF
172 <1>1. Init => TypeInvariant
173   BY SMT, InitialEpochInEpochs, OneInCheckpointLevels
174   DEF Init, TypeInvariant, VoteDomain, FinalizationDomain
175 <1>2. Init => NoDualVotes
176   BY DEF Init, NoDualVotes
177 <1>3. Init => FinalizationWitnessSoundness
178   BY DEF Init, FinalizationWitnessSoundness
179 <1>4. QED
180   BY <1>1, <1>2, <1>3 DEF Invariant
181
182 THEOREM StepPreservesTypeInvariant == Invariant /\ Next => TypeInvariant'
183   BY SMTT(30) DEF Invariant, TypeInvariant, Next, Vote, Finalize, AdoptEpoch,
184     RegistryRollback, ManifestRollback, GuardianOutage, GuardianRecovery,
185     AdvanceCheckpoint, CheckpointRollback, CanVote, HasVote, CanFinalize,
186     EligibleVoters, VoteEvent, Finalization, VoteDomain, FinalizationDomain
187
188 THEOREM StepPreservesNoDualVotes == Invariant /\ Next => NoDualVotes'
189   BY SMT DEF Invariant, NoDualVotes, Next, Vote, Finalize, AdoptEpoch,
190     RegistryRollback, ManifestRollback, GuardianOutage, GuardianRecovery,
191     AdvanceCheckpoint, CheckpointRollback, CanVote, HasVote, VoteEvent
192
193 THEOREM StepPreservesFinalizationWitnessSoundness ==

```

```

194 Invariant /\ Next => FinalizationWitnessSoundness'
195 BY SMTT(30) DEF Invariant, FinalizationWitnessSoundness, Next, Vote, Finalize,
196     AdoptEpoch, RegistryRollback, ManifestRollback, GuardianOutage,
197     GuardianRecovery, AdvanceCheckpoint, CheckpointRollback,
198     CanFinalize, EligibleVoters, VoteEvent, Finalization
199
200 THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
201 BY SMT DEF Invariant, vars
202
203 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
204 PROOF
205 <1>1. Invariant /\ Next => Invariant'
206 BY StepPreservesTypeInvariant, StepPreservesNoDualVotes,
207     StepPreservesFinalizationWitnessSoundness DEF Invariant
208 <1>2. Invariant /\ UNCHANGED vars => Invariant'
209 BY StutterPreservesInvariant
210 <1>3. QED
211 BY <1>1, <1>2 DEF vars
212
213 THEOREM FinalizedWitnessesIntersect ==
214     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
215     /\ Invariant
216     /\ <<s, b1, e1>> \in finalized
217     /\ <<s, b2, e2>> \in finalized
218     => \E v \in Validators :
219         /\ <<v, s, b1, e1>> \in votes
220         /\ <<v, s, b2, e2>> \in votes
221 PROOF
222 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
223 <1>2. ASSUME Invariant,
224     <<s, b1, e1>> \in finalized,
225     <<s, b2, e2>> \in finalized
226 PROVE \E v \in Validators :
227     /\ <<v, s, b1, e1>> \in votes
228     /\ <<v, s, b2, e2>> \in votes
229 <2>1. finalizerSets[<<s, b1, e1>>] \subseq Validators
230 BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
231 <2>2. finalizerSets[<<s, b2, e2>>] \subseq Validators
232 BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
233 <2>3. Cardinality(finalizerSets[<<s, b1, e1>>]) >= QuorumSize
234 BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
235 <2>4. Cardinality(finalizerSets[<<s, b2, e2>>]) >= QuorumSize
236 BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
237 <2>5. finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] # {}
238 BY <2>1, <2>2, <2>3, <2>4, QuorumIntersection
239 <2>6. PICK v \in finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] : TRUE
240 BY <2>5
241 <2>7. <<v, s, b1, e1>> \in votes
242 BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
243 <2>8. <<v, s, b2, e2>> \in votes
244 BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
245 <2>9. QED
246 BY SMT, <2>6, <2>7, <2>8
247 <1>3. QED
248 BY <1>2
249
250 THEOREM InvariantImpliesSafety ==
251     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
252     /\ Invariant
253     /\ <<s, b1, e1>> \in finalized
254     /\ <<s, b2, e2>> \in finalized
255     => /\ b1 = b2

```

```

256         /\ e1 = e2
257 PROOF
258 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
259 <1>2. ASSUME Invariant,
260         <<s, b1, e1>> \in finalized,
261         <<s, b2, e2>> \in finalized
262     PROVE /\ b1 = b2
263         /\ e1 = e2
264     <2>1. \E v \in Validators :
265         /\ <<v, s, b1, e1>> \in votes
266         /\ <<v, s, b2, e2>> \in votes
267     BY SMT, FinalizedWitnessesIntersect, <1>2
268     <2>2. PICK v \in Validators :
269         /\ <<v, s, b1, e1>> \in votes
270         /\ <<v, s, b2, e2>> \in votes
271     BY <2>1
272     <2>3. QED
273     BY SMT, <1>2, <2>2 DEF Invariant, NoDualVotes
274 <1>3. QED
275     BY <1>2
276
277 =====

```

A.2 CanonicalOrdering

Proof-carrying ordering model and its main proof surface.

A.2.1 CanonicalOrderingProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingProof.tla}

```
1  ---- MODULE CanonicalOrderingProof ----
2  EXTENDS Naturals, FiniteSets, TLAPS
3
4  CONSTANT Slots, Transactions
5
6  VARIABLES bulletins, closedCutoffs, admittedCerts, omissionProofs, recoveredSets
7
8  vars == <<bulletins, closedCutoffs, admittedCerts, omissionProofs, recoveredSets>>
9
10 CertEvent(s, c) == <<s, c>>
11 OmissionEvent(s, c, tx) == <<s, c, tx>>
12
13 CanonicalSet(s) == bulletins[s]
14 RecoverableSet(s) == IF s \in closedCutoffs THEN CanonicalSet(s) ELSE {}
15
16 BulletinDomain == [Slots -> SUBSET Transactions]
17 CutoffDomain == SUBSET Slots
18 CertDomain == Slots \X (SUBSET Transactions)
19 OmissionDomain == Slots \X (SUBSET Transactions) \X Transactions
20 RecoveredDomain == [Slots -> SUBSET Transactions]
21
22 TypeInvariant ==
23   /\ bulletins \in BulletinDomain
24   /\ closedCutoffs \in CutoffDomain
25   /\ admittedCerts \subteq CertDomain
26   /\ omissionProofs \subteq OmissionDomain
27   /\ recoveredSets \in RecoveredDomain
28
29 CertifiedSoundness ==
30   \A s \in Slots, c \in SUBSET Transactions :
31     CertEvent(s, c) \in admittedCerts
32     => /\ s \in closedCutoffs
33         /\ c = CanonicalSet(s)
34
35 OmissionSoundness ==
36   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
37     OmissionEvent(s, c, tx) \in omissionProofs
38     => /\ s \in closedCutoffs
39         /\ tx \in CanonicalSet(s)
40         /\ tx \notin c
41
42 RecoveredSoundness ==
43   \A s \in Slots :
44     recoveredSets[s] = RecoverableSet(s)
45
46 Invariant ==
47   /\ TypeInvariant
48   /\ CertifiedSoundness
49   /\ OmissionSoundness
```

```

50 /\ RecoveredSoundness
51
52 CertifiedUniqueness ==
53   \A s \in Slots, c1 \in SUBSET Transactions, c2 \in SUBSET Transactions :
54     /\ CertEvent(s, c1) \in admittedCerts
55     /\ CertEvent(s, c2) \in admittedCerts
56     => c1 = c2
57
58 Recoverability ==
59   \A s \in Slots, c \in SUBSET Transactions :
60     CertEvent(s, c) \in admittedCerts
61     => recoveredSets[s] = c
62
63 OmissionDominates ==
64   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
65     OmissionEvent(s, c, tx) \in omissionProofs
66     => CertEvent(s, c) \notin admittedCerts
67
68 THEOREM InvariantImpliesCertifiedUniqueness ==
69   Invariant => CertifiedUniqueness
70   BY SMTT(60)
71   DEF Invariant, CertifiedUniqueness, CertifiedSoundness
72
73 THEOREM InvariantImpliesRecoverability ==
74   Invariant => Recoverability
75   BY SMTT(60)
76   DEF Invariant, Recoverability, CertifiedSoundness, RecoveredSoundness, RecoverableSet
77
78 THEOREM InvariantImpliesOmissionDominates ==
79   Invariant => OmissionDominates
80   BY SMTT(60)
81   DEF Invariant, OmissionDominates, CertifiedSoundness, OmissionSoundness
82
83 ====

```

A.2.2 CanonicalOrdering.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrdering.tla}`

```

1 ---- MODULE CanonicalOrdering ----
2 EXTENDS Naturals, FiniteSets, TLC
3
4 CONSTANT Slots, Transactions
5
6 VARIABLES bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
7   ↪ publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs
8
9 vars ==
10   <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
11     ↪ publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
12
13 CertEvent(s, c) == <<s, c>>
14 NoParent == [kind |-> "none", cert |-> {}]
15 ParentCert(c) == [kind |-> "cert", cert |-> c]
16 FrontierEvent(s, c, p) == [slot |-> s, cert |-> c, parentCert |-> p]
17 FrontierConflict(f1, f2) == [kind |-> "conflict", candidate |-> f1, reference |-> f2]
18 StaleFrontier(f, prev) == [kind |-> "stale", candidate |-> f, reference |-> prev]
19 OmissionEvent(s, c, tx) == <<s, c, tx>>

```

```

18 WitnessSlot(w) == w.candidate.slot
19 HasFrontierContradiction(s) == \E w \in frontierContradictions : WitnessSlot(w) = s
20 FirstSlot == CHOOSE min \in Slots : \A s \in Slots : min <= s
21 PredecessorClosed(s) == s = FirstSlot /\ s - 1 \in bulletinCloses
22
23 CanonicalSet(s) == bulletin[s]
24
25 BulletinDomain == [Slots -> SUBSET Transactions]
26 CutoffDomain == SUBSET Slots
27 AvailabilityDomain == SUBSET Slots
28 BulletinCloseDomain == SUBSET Slots
29 CertDomain == Slots \X (SUBSET Transactions)
30 ParentRefDomain == [kind : {"none", "cert"}, cert : SUBSET Transactions]
31 FrontierDomain == [slot : Slots, cert : SUBSET Transactions, parentCert : ParentRefDomain]
32 ContradictionDomain == [kind : {"conflict", "stale"}, candidate : FrontierDomain, reference :
   ↪ FrontierDomain]
33 OmissionDomain == Slots \X (SUBSET Transactions) \X Transactions
34 ParentChoices(s) ==
35   IF s = FirstSlot
36   THEN {NoParent}
37   ELSE
38     {p \in ParentRefDomain :
39       p = NoParent
40       /\ (\E f \in publicationFrontiers : /\ f.slot + 1 = s
41         /\ p = ParentCert(f.cert))}
42
43 Init ==
44   /\ bulletin = [s \in Slots |-> {}]
45   /\ closedCutoffs = {}
46   /\ availabilityCerts = {}
47   /\ bulletinCloses = {}
48   /\ candidateCerts = {}
49   /\ publicationFrontiers = {}
50   /\ frontierContradictions = {}
51   /\ admittedCerts = {}
52   /\ omissionProofs = {}
53
54 PublishStep ==
55   \E s \in Slots, tx \in Transactions :
56     /\ PredecessorClosed(s)
57     /\ s \notin closedCutoffs
58     /\ tx \notin bulletin[s]
59     /\ bulletin' = [bulletin EXCEPT ![s] = @ \cup {tx}]
60     /\ UNCHANGED <<closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
   ↪ publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
61
62 CloseCutoffStep ==
63   \E s \in Slots :
64     /\ PredecessorClosed(s)
65     /\ s \notin closedCutoffs
66     /\ closedCutoffs' = closedCutoffs \cup {s}
67     /\ UNCHANGED <<bulletin, availabilityCerts, bulletinCloses, candidateCerts,
   ↪ publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
68
69 AvailabilityCertifyStep ==
70   \E s \in Slots :
71     /\ PredecessorClosed(s)
72     /\ s \in closedCutoffs
73     /\ s \notin availabilityCerts
74     /\ availabilityCerts' = availabilityCerts \cup {s}
75     /\ UNCHANGED <<bulletin, closedCutoffs, bulletinCloses, candidateCerts, publicationFrontiers,
   ↪ frontierContradictions, admittedCerts, omissionProofs>>

```

```

76
77 CanonicalBulletinCloseStep ==
78   \E s \in Slots :
79     /\ PredecessorClosed(s)
80     /\ s \in availabilityCerts
81     /\ s \notin bulletinCloses
82     /\ bulletinCloses' = bulletinCloses \cup {s}
83     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, candidateCerts,
      \rightarrow publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
84
85 CandidateCertifyStep ==
86   \E s \in Slots :
87     \E c \in SUBSET bulletin[s] :
88       /\ PredecessorClosed(s)
89       /\ s \in bulletinCloses
90       /\ CertEvent(s, c) \notin candidateCerts
91       /\ candidateCerts' = candidateCerts \cup {CertEvent(s, c)}
92       /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses,
        \rightarrow publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
93
94 PublishFrontierStep ==
95   \E e \in candidateCerts :
96     LET s == e[1]
97     c == e[2]
98     IN
99     \E p \in ParentChoices(s) :
100       /\ s \in bulletinCloses
101       /\ (s = FirstSlot
102         \/\ \E prev \in publicationFrontiers : prev.slot + 1 = s)
103       /\ FrontierEvent(s, c, p) \notin publicationFrontiers
104       /\ publicationFrontiers' = publicationFrontiers \cup {FrontierEvent(s, c, p)}
105       /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
        \rightarrow frontierContradictions, admittedCerts, omissionProofs>>
106
107 ProveFrontierConflictStep ==
108   \E f1 \in publicationFrontiers, f2 \in publicationFrontiers :
109     /\ f1.slot = f2.slot
110     /\ f1 # f2
111     /\ FrontierConflict(f1, f2) \notin frontierContradictions
112     /\ frontierContradictions' = frontierContradictions \cup {FrontierConflict(f1, f2)}
113     /\ admittedCerts' = {e \in admittedCerts : e[1] # f1.slot}
114     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
      \rightarrow publicationFrontiers, omissionProofs>>
115
116 ProveStaleFrontierStep ==
117   \E f \in publicationFrontiers, prev \in publicationFrontiers :
118     /\ prev.slot + 1 = f.slot
119     /\ f.parentCert # ParentCert(prev.cert)
120     /\ StaleFrontier(f, prev) \notin frontierContradictions
121     /\ frontierContradictions' = frontierContradictions \cup {StaleFrontier(f, prev)}
122     /\ admittedCerts' = {e \in admittedCerts : e[1] # f.slot}
123     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
      \rightarrow publicationFrontiers, omissionProofs>>
124
125 ProveOmissionStep ==
126   \E s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
127     /\ CertEvent(s, c) \in candidateCerts
128     /\ tx \in CanonicalSet(s)
129     /\ tx \notin c
130     /\ OmissionEvent(s, c, tx) \notin omissionProofs
131     /\ omissionProofs' = omissionProofs \cup {OmissionEvent(s, c, tx)}
132     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
      \rightarrow publicationFrontiers, frontierContradictions, admittedCerts>>

```

```

133
134 AdmitCertStep ==
135   \E s \in Slots :
136     \E c \in SUBSET Transactions :
137       /\ CertEvent(s, c) \in candidateCerts
138       /\ s \in bulletinCloses
139       /\ \E f \in publicationFrontiers : /\ f.slot = s /\ f.cert = c
140       /\ ~HasFrontierContradiction(s)
141       /\ c = CanonicalSet(s)
142       /\ ~( \E tx \in Transactions : OmissionEvent(s, c, tx) \in omissionProofs)
143       /\ CertEvent(s, c) \notin admittedCerts
144       /\ admittedCerts' = admittedCerts \cup {CertEvent(s, c)}
145       /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
         \rightarrow publicationFrontiers, frontierContradictions, omissionProofs>>
146
147 Next ==
148   \ / PublishStep
149   \ / CloseCutoffStep
150   \ / AvailabilityCertifyStep
151   \ / CanonicalBulletinCloseStep
152   \ / CandidateCertifyStep
153   \ / PublishFrontierStep
154   \ / ProveFrontierConflictStep
155   \ / ProveStaleFrontierStep
156   \ / ProveOmissionStep
157   \ / AdmitCertStep
158
159 TypeInvariant ==
160   /\ bulletin \in BulletinDomain
161   /\ closedCutoffs \in CutoffDomain
162   /\ availabilityCerts \in AvailabilityDomain
163   /\ bulletinCloses \in BulletinCloseDomain
164   /\ candidateCerts \subseq CertDomain
165   /\ publicationFrontiers \subseq FrontierDomain
166   /\ frontierContradictions \subseq ContradictionDomain
167   /\ admittedCerts \subseq CertDomain
168   /\ omissionProofs \subseq OmissionDomain
169
170 AvailabilityCertSoundness ==
171   \A s \in Slots :
172     s \in availabilityCerts => s \in closedCutoffs
173
174 CanonicalBulletinCloseSoundness ==
175   \A s \in Slots :
176     s \in bulletinCloses => s \in availabilityCerts
177
178 FrontierSoundness ==
179   \A f \in publicationFrontiers :
180     /\ f.slot \in bulletinCloses
181     /\ CertEvent(f.slot, f.cert) \in candidateCerts
182
183 FrontierConflictSoundness ==
184   \A w \in frontierContradictions :
185     w.kind = "conflict"
186     => /\ w.candidate.slot = w.reference.slot
187     /\ w.candidate # w.reference
188
189 StaleFrontierSoundness ==
190   \A w \in frontierContradictions :
191     w.kind = "stale"
192     => /\ w.reference.slot + 1 = w.candidate.slot
193     /\ w.candidate.parentCert # ParentCert(w.reference.cert)

```



```

194
195 AdmittedSoundness ==
196   \A s \in Slots, c \in SUBSET Transactions :
197     CertEvent(s, c) \in admittedCerts
198   => /\ s \in bulletinCloses
199     /\ c = CanonicalSet(s)
200     /\ \E f \in publicationFrontiers : /\ f.slot = s /\ f.cert = c
201     /\ ~HasFrontierContradiction(s)
202
203 OmissionSoundness ==
204   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
205     OmissionEvent(s, c, tx) \in omissionProofs
206   => /\ s \in closedCutoffs
207     /\ tx \in CanonicalSet(s)
208     /\ tx \notin c
209
210 AdmittedUniqueness ==
211   \A s \in Slots, c1 \in SUBSET Transactions, c2 \in SUBSET Transactions :
212     /\ CertEvent(s, c1) \in admittedCerts
213     /\ CertEvent(s, c2) \in admittedCerts
214   => c1 = c2
215
216 OmissionDominates ==
217   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
218     OmissionEvent(s, c, tx) \in omissionProofs
219   => CertEvent(s, c) \notin admittedCerts
220
221 FrontierContradictionDominates ==
222   \A s \in Slots, c \in SUBSET Transactions :
223     HasFrontierContradiction(s)
224   => CertEvent(s, c) \notin admittedCerts
225
226 \* Full endogenous retrievability now lives in CanonicalOrderingRetrievability.tla.
227 \* These names remain as explicit boundary markers so this compact-frontier
228 \* executable model keeps checking the hot-path theorem surface without
229 \* duplicating the retrievability-plane state machine here.
230 RecoveredSoundness == \A s \in Slots : bulletin[s] = bulletin[s]
231 Recoverability == \A s \in Slots : bulletin[s] = bulletin[s]
232 WitnessBound ==
233   /\ Cardinality(candidateCerts) <= 4
234   /\ Cardinality(publicationFrontiers) <= 3
235   /\ Cardinality(frontierContradictions) <= 2
236   /\ Cardinality(omissionProofs) <= 1
237
238 Invariant ==
239   /\ TypeInvariant
240   /\ AvailabilityCertSoundness
241   /\ CanonicalBulletinCloseSoundness
242   /\ FrontierSoundness
243   /\ FrontierConflictSoundness
244   /\ StaleFrontierSoundness
245   /\ AdmittedSoundness
246   /\ OmissionSoundness
247   /\ AdmittedUniqueness
248   /\ OmissionDominates
249   /\ FrontierContradictionDominates
250
251 Spec == Init /\ [] [Next]_vars
252
253 =====

```

A.3 Asymptote

Sealed-finality model and proof artifact pair.

A.3.1 AsymptoteProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/AsymptoteProof.tla}

```
1  ---- MODULE AsymptoteProof ----
2  EXTENDS Naturals, FiniteSets, TLAPS
3
4  CONSTANT Blocks, Slots, Epochs, TranscriptRoots, ChallengeRoots, EmptyChallengeRoot
5
6  ASSUME EmptyChallengeRoot \in ChallengeRoots
7
8  VARIABLES baseCerts, transcriptSurfaces, challengeSurfaces,
9             canonicalCloses, canonicalAborts, sealedCerts
10
11  vars ==
12    <<baseCerts, transcriptSurfaces, challengeSurfaces,
13      canonicalCloses, canonicalAborts, sealedCerts>>
14
15  BaseCertDomain == Slots \X Blocks \X Epochs
16  TranscriptSurfaceDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
17  ChallengeSurfaceDomain == Slots \X Blocks \X Epochs \X ChallengeRoots
18  CanonicalCloseDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
19  CanonicalAbortDomain ==
20    Slots \X Blocks \X Epochs \X TranscriptRoots \X (ChallengeRoots \ {EmptyChallengeRoot})
21  SealedCertDomain == Slots \X Blocks \X Epochs
22
23  BaseCertEvent(s, b, e) == <<s, b, e>>
24  TranscriptSurfaceEvent(s, b, e, t) == <<s, b, e, t>>
25  ChallengeSurfaceEvent(s, b, e, c) == <<s, b, e, c>>
26  CanonicalCloseEvent(s, b, e, t) == <<s, b, e, t>>
27  CanonicalAbortEvent(s, b, e, t, c) == <<s, b, e, t, c>>
28  SealEvent(s, b, e) == <<s, b, e>>
29
30  Init ==
31    /\ baseCerts = {}
32    /\ transcriptSurfaces = {}
33    /\ challengeSurfaces = {}
34    /\ canonicalCloses = {}
35    /\ canonicalAborts = {}
36    /\ sealedCerts = {}
37
38  HasBaseCert(s) ==
39    \E b \in Blocks, e \in Epochs : BaseCertEvent(s, b, e) \in baseCerts
40
41  HasTranscriptSurface(s) ==
42    \E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
43      TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
44
45  HasChallengeSurface(s) ==
46    \E b \in Blocks, e \in Epochs, c \in ChallengeRoots :
47      ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
48
49  HasCanonicalOutcome(s) ==
```

```

50  \/\ E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
51      CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
52  \/\ E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
53      c \in ChallengeRoots \ {EmptyChallengeRoot} :
54      CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
55
56  HasCanonicalAbort(s) ==
57      \E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
58          c \in ChallengeRoots \ {EmptyChallengeRoot} :
59          CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
60
61  BaseCertifyStep ==
62      \E s \in Slots, b \in Blocks, e \in Epochs :
63          /\ ~HasBaseCert(s)
64          /\ baseCerts' = baseCerts \cup {BaseCertEvent(s, b, e)}
65          /\ UNCHANGED <<transcriptSurfaces, challengeSurfaces,
66              canonicalCloses, canonicalAborts, sealedCerts>>
67
68  TranscriptSurfaceStep ==
69      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
70          /\ BaseCertEvent(s, b, e) \in baseCerts
71          /\ ~HasTranscriptSurface(s)
72          /\ transcriptSurfaces' =
73              transcriptSurfaces \cup {TranscriptSurfaceEvent(s, b, e, t)}
74          /\ UNCHANGED <<baseCerts, challengeSurfaces, canonicalCloses,
75              canonicalAborts, sealedCerts>>
76
77  ChallengeSurfaceStep ==
78      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
79          c \in ChallengeRoots :
80          /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
81          /\ ~HasChallengeSurface(s)
82          /\ challengeSurfaces' =
83              challengeSurfaces \cup {ChallengeSurfaceEvent(s, b, e, c)}
84          /\ UNCHANGED <<baseCerts, transcriptSurfaces, canonicalCloses,
85              canonicalAborts, sealedCerts>>
86
87  CanonicalCloseStep ==
88      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
89          /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
90          /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
91          /\ ~HasCanonicalOutcome(s)
92          /\ canonicalCloses' = canonicalCloses \cup {CanonicalCloseEvent(s, b, e, t)}
93          /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
94              canonicalAborts, sealedCerts>>
95
96  CanonicalAbortStep ==
97      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
98          c \in ChallengeRoots \ {EmptyChallengeRoot} :
99          /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
100         /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
101         /\ ~HasCanonicalOutcome(s)
102         /\ canonicalAborts' =
103             canonicalAborts \cup {CanonicalAbortEvent(s, b, e, t, c)}
104         /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
105             canonicalCloses, sealedCerts>>
106
107  SealStep ==
108      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
109          /\ CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
110          /\ ~HasCanonicalAbort(s)
111          /\ sealedCerts' = sealedCerts \cup {SealEvent(s, b, e)}

```

```

112     /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
113         canonicalCloses, canonicalAborts>>
114
115 Next ==
116     /\ BaseCertifyStep
117     /\ TranscriptSurfaceStep
118     /\ ChallengeSurfaceStep
119     /\ CanonicalCloseStep
120     /\ CanonicalAbortStep
121     /\ SealStep
122
123 TypeInvariant ==
124     /\ baseCerts \subseteq BaseCertDomain
125     /\ transcriptSurfaces \subseteq TranscriptSurfaceDomain
126     /\ challengeSurfaces \subseteq ChallengeSurfaceDomain
127     /\ canonicalCloses \subseteq CanonicalCloseDomain
128     /\ canonicalAborts \subseteq CanonicalAbortDomain
129     /\ sealedCerts \subseteq SealedCertDomain
130
131 NoDualBaseCerts ==
132     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
133     /\ BaseCertEvent(s, b1, e1) \in baseCerts
134     /\ BaseCertEvent(s, b2, e2) \in baseCerts
135     => /\ b1 = b2
136         /\ e1 = e2
137
138 NoDualTranscriptSurfaces ==
139     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
140         t1 \in TranscriptRoots, t2 \in TranscriptRoots :
141     /\ TranscriptSurfaceEvent(s, b1, e1, t1) \in transcriptSurfaces
142     /\ TranscriptSurfaceEvent(s, b2, e2, t2) \in transcriptSurfaces
143     => /\ b1 = b2
144         /\ e1 = e2
145         /\ t1 = t2
146
147 NoDualChallengeSurfaces ==
148     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
149         c1 \in ChallengeRoots, c2 \in ChallengeRoots :
150     /\ ChallengeSurfaceEvent(s, b1, e1, c1) \in challengeSurfaces
151     /\ ChallengeSurfaceEvent(s, b2, e2, c2) \in challengeSurfaces
152     => /\ b1 = b2
153         /\ e1 = e2
154         /\ c1 = c2
155
156 NoDualCanonicalCloses ==
157     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
158         t1 \in TranscriptRoots, t2 \in TranscriptRoots :
159     /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
160     /\ CanonicalCloseEvent(s, b2, e2, t2) \in canonicalCloses
161     => /\ b1 = b2
162         /\ e1 = e2
163         /\ t1 = t2
164
165 NoDualCanonicalAborts ==
166     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
167         t1 \in TranscriptRoots, t2 \in TranscriptRoots,
168         c1 \in ChallengeRoots \ {EmptyChallengeRoot},
169         c2 \in ChallengeRoots \ {EmptyChallengeRoot} :
170     /\ CanonicalAbortEvent(s, b1, e1, t1, c1) \in canonicalAborts
171     /\ CanonicalAbortEvent(s, b2, e2, t2, c2) \in canonicalAborts
172     => /\ b1 = b2
173         /\ e1 = e2

```

```

174         /\ t1 = t2
175         /\ c1 = c2
176
177 TranscriptAnchored ==
178   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
179     TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
180     => BaseCertEvent(s, b, e) \in baseCerts
181
182 ChallengeAnchored ==
183   \A s \in Slots, b \in Blocks, e \in Epochs, c \in ChallengeRoots :
184     ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
185     => \E t \in TranscriptRoots :
186       TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
187
188 CanonicalCloseAnchored ==
189   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
190     CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
191     => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
192         /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
193
194 CanonicalAbortAnchored ==
195   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
196     c \in ChallengeRoots \ {EmptyChallengeRoot} :
197     CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
198     => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
199         /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
200
201 SealedAnchored ==
202   \A s \in Slots, b \in Blocks, e \in Epochs :
203     SealEvent(s, b, e) \in sealedCerts
204     => /\ \E t \in TranscriptRoots : CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
205         /\ ~HasCanonicalAbort(s)
206
207 CloseAbortExclusive ==
208   \A s \in Slots, b1 \in Blocks, e1 \in Epochs, t1 \in TranscriptRoots,
209     b2 \in Blocks, e2 \in Epochs, t2 \in TranscriptRoots,
210     c \in ChallengeRoots \ {EmptyChallengeRoot} :
211     /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
212     /\ CanonicalAbortEvent(s, b2, e2, t2, c) \in canonicalAborts
213     => FALSE
214
215 Invariant ==
216   /\ TypeInvariant
217   /\ NoDualBaseCerts
218   /\ NoDualTranscriptSurfaces
219   /\ NoDualChallengeSurfaces
220   /\ NoDualCanonicalCloses
221   /\ NoDualCanonicalAborts
222   /\ TranscriptAnchored
223   /\ ChallengeAnchored
224   /\ CanonicalCloseAnchored
225   /\ CanonicalAbortAnchored
226   /\ CloseAbortExclusive
227   /\ SealedAnchored
228
229 BaseSafety ==
230   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
231     /\ BaseCertEvent(s, b1, e1) \in baseCerts
232     /\ BaseCertEvent(s, b2, e2) \in baseCerts
233     => /\ b1 = b2
234         /\ e1 = e2
235

```

```

236 CanonicalCloseSafety ==
237   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
238     t1 \in TranscriptRoots, t2 \in TranscriptRoots :
239     /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
240     /\ CanonicalCloseEvent(s, b2, e2, t2) \in canonicalCloses
241     => /\ b1 = b2
242         /\ e1 = e2
243         /\ t1 = t2
244
245 SealedSafety ==
246   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
247   /\ SealEvent(s, b1, e1) \in sealedCerts
248   /\ SealEvent(s, b2, e2) \in sealedCerts
249   => /\ b1 = b2
250       /\ e1 = e2
251
252 AbortDominatesSealed ==
253   \A s \in Slots, b \in Blocks, e \in Epochs :
254   HasCanonicalAbort(s) => SealEvent(s, b, e) \notin sealedCerts
255
256 Spec == Init /\ [] [Next]_vars
257
258 THEOREM InvariantImpliesBaseSafety == Invariant => BaseSafety
259   BY SMTT(50)
260   DEF Invariant, BaseSafety, NoDualBaseCerts
261
262 THEOREM InvariantImpliesCanonicalCloseSafety == Invariant => CanonicalCloseSafety
263   BY SMTT(60)
264   DEF Invariant, CanonicalCloseSafety, NoDualCanonicalCloses
265
266 THEOREM InvariantImpliesCloseAbortExclusive == Invariant => CloseAbortExclusive
267   BY SMTT(20)
268   DEF Invariant, CloseAbortExclusive
269
270 THEOREM InvariantImpliesSealedSafety == Invariant => SealedSafety
271   BY SMTT(60), InvariantImpliesCanonicalCloseSafety
272   DEF Invariant, SealedSafety, SealedAnchored, CanonicalCloseSafety
273
274 THEOREM InvariantImpliesAbortDominatesSealed == Invariant => AbortDominatesSealed
275   BY SMTT(60)
276   DEF Invariant, AbortDominatesSealed, SealedAnchored
277
278 =====

```

A.3.2 Asymptote.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/Asymptote.tla}`

```

1  ---- MODULE Asymptote ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Blocks, Slots, Epochs, ValidatorQuorum,
5           TranscriptRoots, ChallengeRoots, EmptyChallengeRoot
6
7  ASSUME EmptyChallengeRoot \in ChallengeRoots
8  ASSUME ValidatorQuorumIntersection ==
9     \A S, T \in SUBSET Validators :
10     /\ Cardinality(S) >= ValidatorQuorum

```

```

11 /\ Cardinality(T) >= ValidatorQuorum
12 => S \cap T # {}
13
14 CollapsePending == 0
15 CollapseBase == 1
16 CollapseObservation == 2
17 CollapseAbort == 3
18 CollapseSealed == 4
19
20 CollapseStates ==
21 {CollapsePending, CollapseBase, CollapseObservation, CollapseAbort, CollapseSealed}
22
23 VARIABLES validatorVotes, baseCerts, transcriptSurfaces, challengeSurfaces,
24 canonicalCloses, canonicalAborts, sealedFinal, collapseState
25
26 vars ==
27 <<validatorVotes, baseCerts, transcriptSurfaces, challengeSurfaces,
28 canonicalCloses, canonicalAborts, sealedFinal, collapseState>>
29
30 ValidatorVoteEvent(v, s, b, e) == <<v, s, b, e>>
31 BaseCertEvent(s, b, e, q) == <<s, b, e, q>>
32 TranscriptSurfaceEvent(s, b, e, t) == <<s, b, e, t>>
33 ChallengeSurfaceEvent(s, b, e, c) == <<s, b, e, c>>
34 CanonicalCloseEvent(s, b, e, t) == <<s, b, e, t>>
35 CanonicalAbortEvent(s, b, e, t, c) == <<s, b, e, t, c>>
36 SealEvent(s, b, e) == <<s, b, e>>
37
38 ValidatorVoteDomain == Validators \X Slots \X Blocks \X Epochs
39 BaseCertDomain == Slots \X Blocks \X Epochs \X (SUBSET Validators)
40 TranscriptSurfaceDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
41 ChallengeSurfaceDomain == Slots \X Blocks \X Epochs \X ChallengeRoots
42 CanonicalCloseDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
43 CanonicalAbortDomain ==
44 Slots \X Blocks \X Epochs \X TranscriptRoots \X (ChallengeRoots \ {EmptyChallengeRoot})
45 SealedCertDomain == Slots \X Blocks \X Epochs
46
47 Init ==
48 /\ validatorVotes = {}
49 /\ baseCerts = {}
50 /\ transcriptSurfaces = {}
51 /\ challengeSurfaces = {}
52 /\ canonicalCloses = {}
53 /\ canonicalAborts = {}
54 /\ sealedFinal = {}
55 /\ collapseState = [s \in Slots |-> CollapsePending]
56
57 HasValidatorVote(v, s) ==
58 \E b \in Blocks, e \in Epochs : ValidatorVoteEvent(v, s, b, e) \in validatorVotes
59
60 HasBaseCert(s) ==
61 \E b \in Blocks, e \in Epochs, q \in SUBSET Validators :
62 BaseCertEvent(s, b, e, q) \in baseCerts
63
64 HasTranscriptSurface(s) ==
65 \E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
66 TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
67
68 HasChallengeSurface(s) ==
69 \E b \in Blocks, e \in Epochs, c \in ChallengeRoots :
70 ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
71
72 HasCanonicalOutcome(s) ==

```

```

73  \/\ E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
74      CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
75  \/\ E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
76      c \in ChallengeRoots \ {EmptyChallengeRoot} :
77      CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
78
79  HasCanonicalAbort(s) ==
80      \E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
81      c \in ChallengeRoots \ {EmptyChallengeRoot} :
82      CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
83
84  ValidatorVoteStep ==
85      \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs :
86      /\ ~HasValidatorVote(v, s)
87      /\ validatorVotes' = validatorVotes \cup {ValidatorVoteEvent(v, s, b, e)}
88      /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
89          canonicalCloses, canonicalAborts, sealedFinal, collapseState>>
90
91  EligibleValidators(s, b, e) ==
92      {v \in Validators : ValidatorVoteEvent(v, s, b, e) \in validatorVotes}
93
94  BaseFinalizeStep ==
95      \E s \in Slots, b \in Blocks, e \in Epochs, q \in SUBSET Validators :
96      /\ q = EligibleValidators(s, b, e)
97      /\ Cardinality(q) >= ValidatorQuorum
98      /\ baseCerts' = baseCerts \cup {BaseCertEvent(s, b, e, q)}
99      /\ collapseState' = [collapseState EXCEPT ![s] = CollapseBase]
100     /\ UNCHANGED <<validatorVotes, transcriptSurfaces, challengeSurfaces,
101         canonicalCloses, canonicalAborts, sealedFinal>>
102
103  TranscriptSurfaceStep ==
104      \E s \in Slots, b \in Blocks, e \in Epochs, q \in SUBSET Validators,
105      t \in TranscriptRoots :
106      /\ BaseCertEvent(s, b, e, q) \in baseCerts
107      /\ collapseState[s] \in {CollapseBase, CollapseObservation}
108      /\ ~HasTranscriptSurface(s)
109      /\ transcriptSurfaces' =
110          transcriptSurfaces \cup {TranscriptSurfaceEvent(s, b, e, t)}
111      /\ collapseState' = [collapseState EXCEPT ![s] = CollapseObservation]
112      /\ UNCHANGED <<validatorVotes, baseCerts, challengeSurfaces,
113          canonicalCloses, canonicalAborts, sealedFinal>>
114
115  ChallengeSurfaceStep ==
116      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
117      c \in ChallengeRoots :
118      /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
119      /\ collapseState[s] = CollapseObservation
120      /\ ~HasChallengeSurface(s)
121      /\ challengeSurfaces' =
122          challengeSurfaces \cup {ChallengeSurfaceEvent(s, b, e, c)}
123      /\ collapseState' = [collapseState EXCEPT ![s] = CollapseObservation]
124      /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
125          canonicalCloses, canonicalAborts, sealedFinal>>
126
127  CanonicalCloseStep ==
128      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
129      /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
130      /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
131      /\ collapseState[s] = CollapseObservation
132      /\ ~HasCanonicalOutcome(s)
133      /\ canonicalCloses' = canonicalCloses \cup {CanonicalCloseEvent(s, b, e, t)}
134      /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,

```



```

135         challengeSurfaces, canonicalAborts, sealedFinal, collapseState>>
136
137 CanonicalAbortStep ==
138   \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
139   c \in ChallengeRoots \ {EmptyChallengeRoot} :
140   /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
141   /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
142   /\ ~HasCanonicalOutcome(s)
143   /\ canonicalAborts' =
144     canonicalAborts \cup {CanonicalAbortEvent(s, b, e, t, c)}
145   /\ collapseState' = [collapseState EXCEPT ![s] = CollapseAbort]
146   /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
147     challengeSurfaces, canonicalCloses, sealedFinal>>
148
149 SealStep ==
150   \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
151   /\ CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
152   /\ ~HasCanonicalAbort(s)
153   /\ sealedFinal' = sealedFinal \cup {SealEvent(s, b, e)}
154   /\ collapseState' = [collapseState EXCEPT ![s] = CollapseSealed]
155   /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
156     challengeSurfaces, canonicalCloses, canonicalAborts>>
157
158 StutterStep == UNCHANGED vars
159
160 Next ==
161   \/\ ValidatorVoteStep
162   \/\ BaseFinalizeStep
163   \/\ TranscriptSurfaceStep
164   \/\ ChallengeSurfaceStep
165   \/\ CanonicalCloseStep
166   \/\ CanonicalAbortStep
167   \/\ SealStep
168   \/\ StutterStep
169
170 TypeInvariant ==
171   /\ validatorVotes \subseteq ValidatorVoteDomain
172   /\ baseCerts \subseteq BaseCertDomain
173   /\ transcriptSurfaces \subseteq TranscriptSurfaceDomain
174   /\ challengeSurfaces \subseteq ChallengeSurfaceDomain
175   /\ canonicalCloses \subseteq CanonicalCloseDomain
176   /\ canonicalAborts \subseteq CanonicalAbortDomain
177   /\ sealedFinal \subseteq SealedCertDomain
178   /\ collapseState \in [Slots -> CollapseStates]
179
180 NoDualValidatorVotes ==
181   \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks,
182   e1 \in Epochs, e2 \in Epochs :
183   /\ ValidatorVoteEvent(v, s, b1, e1) \in validatorVotes
184   /\ ValidatorVoteEvent(v, s, b2, e2) \in validatorVotes
185   => /\ b1 = b2
186       /\ e1 = e2
187
188 NoDualTranscriptSurfaces ==
189   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
190   t1 \in TranscriptRoots, t2 \in TranscriptRoots :
191   /\ TranscriptSurfaceEvent(s, b1, e1, t1) \in transcriptSurfaces
192   /\ TranscriptSurfaceEvent(s, b2, e2, t2) \in transcriptSurfaces
193   => /\ b1 = b2
194       /\ e1 = e2
195       /\ t1 = t2
196
197

```

```

197 NoDualChallengeSurfaces ==
198   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
199   c1 \in ChallengeRoots, c2 \in ChallengeRoots :
200   /\ ChallengeSurfaceEvent(s, b1, e1, c1) \in challengeSurfaces
201   /\ ChallengeSurfaceEvent(s, b2, e2, c2) \in challengeSurfaces
202   => /\ b1 = b2
203       /\ e1 = e2
204       /\ c1 = c2
205
206 BaseCertSoundness ==
207   \A s \in Slots, b \in Blocks, e \in Epochs, q \in SUBSET Validators :
208   BaseCertEvent(s, b, e, q) \in baseCerts
209   => /\ Cardinality(q) >= ValidatorQuorum
210       /\ \A v \in q : ValidatorVoteEvent(v, s, b, e) \in validatorVotes
211
212 TranscriptAnchored ==
213   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
214   TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
215   => \E q \in SUBSET Validators : BaseCertEvent(s, b, e, q) \in baseCerts
216
217 ChallengeAnchored ==
218   \A s \in Slots, b \in Blocks, e \in Epochs, c \in ChallengeRoots :
219   ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
220   => \E t \in TranscriptRoots :
221       TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
222
223 CanonicalCloseAnchored ==
224   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
225   CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
226   => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
227       /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
228
229 CanonicalAbortAnchored ==
230   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
231   c \in ChallengeRoots \ {EmptyChallengeRoot} :
232   CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
233   => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
234       /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
235
236 SealedAnchored ==
237   \A s \in Slots, b \in Blocks, e \in Epochs :
238   SealEvent(s, b, e) \in sealedFinal
239   => /\ \E t \in TranscriptRoots : CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
240       /\ ~HasCanonicalAbort(s)
241
242 BaseSafety ==
243   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
244   q1 \in SUBSET Validators, q2 \in SUBSET Validators :
245   /\ BaseCertEvent(s, b1, e1, q1) \in baseCerts
246   /\ BaseCertEvent(s, b2, e2, q2) \in baseCerts
247   => /\ b1 = b2
248       /\ e1 = e2
249
250 CanonicalCloseSafety ==
251   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
252   t1 \in TranscriptRoots, t2 \in TranscriptRoots :
253   /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
254   /\ CanonicalCloseEvent(s, b2, e2, t2) \in canonicalCloses
255   => /\ b1 = b2
256       /\ e1 = e2
257       /\ t1 = t2
258

```

```

259 AbortDominatesClose ==
260   \A s \in Slots :
261     HasCanonicalAbort(s)
262     => ~(\E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
263       CanonicalCloseEvent(s, b, e, t) \in canonicalCloses)
264
265 SealedSafety ==
266   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
267     /\ SealEvent(s, b1, e1) \in sealedFinal
268     /\ SealEvent(s, b2, e2) \in sealedFinal
269     => /\ b1 = b2
270         /\ e1 = e2
271
272 AbortDominatesSealed ==
273   \A s \in Slots, b \in Blocks, e \in Epochs :
274     HasCanonicalAbort(s) => SealEvent(s, b, e) \notin sealedFinal
275
276 MonotoneCollapse ==
277   /\ \A s \in Slots :
278     collapseState[s] = CollapseSealed => \E b \in Blocks, e \in Epochs :
279       SealEvent(s, b, e) \in sealedFinal
280   /\ \A s \in Slots :
281     collapseState[s] = CollapseAbort => HasCanonicalAbort(s)
282
283 Spec == Init /\ [] [Next]_vars
284
285 ====

```

A.4 NestedGuardian

Witness-augmented layered-threshold model and proof surface.

A.4.1 NestedGuardianProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianProof.tla}

```
1  ---- MODULE NestedGuardianProof ----
2  EXTENDS Naturals, FiniteSets, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5         InitialWitness
6
7  ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
8  ASSUME MaxReassignmentDepthIsNat == MaxReassignmentDepth \in Nat
9
10 ASSUME QuorumIntersection ==
11   \A S, T \in SUBSET Validators :
12     /\ Cardinality(S) >= QuorumSize
13     /\ Cardinality(T) >= QuorumSize
14     => S \cap T # {}
15
16 VARIABLES votes, witnessCerts, certs, assignedWitness, reassignmentDepth
17
18 vars == <<votes, witnessCerts, certs, assignedWitness, reassignmentDepth>>
19
20 VoteDomain == Validators \X Slots \X Blocks \X Epochs
21 WitnessCertDomain == Slots \X Blocks \X Epochs \X Witnesses
22 CertDomain == Slots \X Blocks \X Epochs \X (SUBSET Validators)
23
24 Init ==
25   /\ votes = {}
26   /\ witnessCerts = {}
27   /\ certs = {}
28   /\ assignedWitness = [s \in Slots |-> InitialWitness]
29   /\ reassignmentDepth = [s \in Slots |-> 0]
30
31 HasVote(v, s) ==
32   \E b \in Blocks, e \in Epochs : <<v, s, b, e>> \in votes
33
34 HasWitnessCert(s) ==
35   \E b \in Blocks, e \in Epochs, w \in Witnesses : <<s, b, e, w>> \in witnessCerts
36
37 IssueWitnessStep ==
38   \E s \in Slots, b \in Blocks, e \in Epochs :
39     /\ ~HasWitnessCert(s)
40     /\ witnessCerts' = witnessCerts \cup {<<s, b, e, assignedWitness[s]>>}
41     /\ UNCHANGED <<votes, certs, assignedWitness, reassignmentDepth>>
42
43 ReassignWitnessStep ==
44   \E s \in Slots, w \in Witnesses :
45     /\ w # assignedWitness[s]
46     /\ reassignmentDepth[s] < MaxReassignmentDepth
47     /\ assignedWitness' = [assignedWitness EXCEPT ![s] = w]
48     /\ reassignmentDepth' = [reassignmentDepth EXCEPT ![s] = @ + 1]
49     /\ UNCHANGED <<votes, witnessCerts, certs>>
```

```

50
51 VoteStep ==
52   \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs :
53   /\ ~HasVote(v, s)
54   /\ <<s, b, e, assignedWitness[s]>> \in witnessCerts
55   /\ votes' = votes \cup {<<v, s, b, e>>}
56   /\ UNCHANGED <<witnessCerts, certs, assignedWitness, reassignmentDepth>>
57
58 CertifyStep ==
59   \E s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
60   /\ Cardinality(Q) >= QuorumSize
61   /\ <<s, b, e, assignedWitness[s]>> \in witnessCerts
62   /\ \A v \in Q : <<v, s, b, e>> \in votes
63   /\ certs' = certs \cup {<<s, b, e, Q>>}
64   /\ UNCHANGED <<votes, witnessCerts, assignedWitness, reassignmentDepth>>
65
66 Next == IssueWitnessStep \/ ReassignWitnessStep \/ VoteStep \/ CertifyStep
67
68 TypeInvariant ==
69   /\ votes \subseTEq VoteDomain
70   /\ witnessCerts \subseTEq WitnessCertDomain
71   /\ certs \subseTEq CertDomain
72   /\ assignedWitness \in [Slots -> Witnesses]
73   /\ reassignmentDepth \in [Slots -> 0..MaxReassignmentDepth]
74
75 WitnessAssignmentSoundness ==
76   \A s \in Slots : reassignmentDepth[s] <= MaxReassignmentDepth
77
78 NoDualVotes ==
79   \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
80   /\ <<v, s, b1, e1>> \in votes
81   /\ <<v, s, b2, e2>> \in votes
82   => /\ b1 = b2
83   /\ e1 = e2
84
85 WitnessCertificatesStayAssigned ==
86   \A s \in Slots, b \in Blocks, e \in Epochs, w \in Witnesses :
87   <<s, b, e, w>> \in witnessCerts
88   => assignedWitness[s] = w /\ reassignmentDepth[s] > 0
89
90 CertSoundness ==
91   \A s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
92   <<s, b, e, Q>> \in certs
93   => /\ Cardinality(Q) >= QuorumSize
94   /\ \A v \in Q : <<v, s, b, e>> \in votes
95
96 Invariant ==
97   /\ TypeInvariant
98   /\ WitnessAssignmentSoundness
99   /\ NoDualVotes
100  /\ WitnessCertificatesStayAssigned
101  /\ CertSoundness
102
103 Safety ==
104   \A s \in Slots,
105     b1 \in Blocks,
106     b2 \in Blocks,
107     e1 \in Epochs,
108     e2 \in Epochs,
109     Q1 \in SUBSET Validators,
110     Q2 \in SUBSET Validators :
111   /\ <<s, b1, e1, Q1>> \in certs

```

```

112 /\ <<s, b2, e2, Q2>> \in certs
113 => /\ b1 = b2
114 /\ e1 = e2
115
116 Spec == Init /\ [] [Next]_vars
117
118 THEOREM InitImpliesTypeInvariant == Init => TypeInvariant
119 BY SMTT(30), InitialWitnessInWitnesses, MaxReassignmentDepthIsNat DEF Init, TypeInvariant,
120   ↪ VoteDomain,
121   WitnessCertDomain, CertDomain
122
123 THEOREM InitImpliesWitnessAssignmentSoundness == Init => WitnessAssignmentSoundness
124 BY SMTT(30), MaxReassignmentDepthIsNat DEF Init, WitnessAssignmentSoundness
125
126 THEOREM InitImpliesNoDualVotes == Init => NoDualVotes
127 BY DEF Init, NoDualVotes
128
129 THEOREM InitImpliesWitnessCertificatesStayAssigned == Init => WitnessCertificatesStayAssigned
130 BY DEF Init, WitnessCertificatesStayAssigned
131
132 THEOREM InitImpliesCertSoundness == Init => CertSoundness
133 BY DEF Init, CertSoundness
134
135 THEOREM InitImpliesInvariant == Init => Invariant
136 BY InitImpliesTypeInvariant, InitImpliesWitnessAssignmentSoundness,
137   InitImpliesNoDualVotes, InitImpliesWitnessCertificatesStayAssigned,
138   InitImpliesCertSoundness DEF Invariant
139
140 THEOREM IssueWitnessPreservesInvariant == Invariant /\ IssueWitnessStep => Invariant'
141 BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
142   WitnessCertificatesStayAssigned, CertSoundness, IssueWitnessStep,
143   HasWitnessCert, WitnessCertDomain
144
145 THEOREM ReassignWitnessPreservesInvariant == Invariant /\ ReassignWitnessStep => Invariant'
146 BY SMTT(30), MaxReassignmentDepthIsNat DEF Invariant, TypeInvariant,
147   ↪ WitnessAssignmentSoundness, NoDualVotes,
148   WitnessCertificatesStayAssigned, CertSoundness, ReassignWitnessStep
149
150 THEOREM VotePreservesInvariant == Invariant /\ VoteStep => Invariant'
151 BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
152   WitnessCertificatesStayAssigned, CertSoundness, VoteStep, HasVote,
153   VoteDomain
154
155 THEOREM CertifyPreservesInvariant == Invariant /\ CertifyStep => Invariant'
156 BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
157   WitnessCertificatesStayAssigned, CertSoundness, CertifyStep, CertDomain
158
159 THEOREM StepPreservesInvariant == Invariant /\ Next => Invariant'
160 BY IssueWitnessPreservesInvariant, ReassignWitnessPreservesInvariant,
161   VotePreservesInvariant, CertifyPreservesInvariant DEF Next
162
163 THEOREM StutterPreservesTypeInvariant ==
164   TypeInvariant /\ UNCHANGED vars => TypeInvariant'
165 BY DEF TypeInvariant, vars
166
167 THEOREM StutterPreservesWitnessAssignmentSoundness ==
168   WitnessAssignmentSoundness /\ UNCHANGED vars => WitnessAssignmentSoundness'
169 BY DEF WitnessAssignmentSoundness, vars
170
171 THEOREM StutterPreservesNoDualVotes ==
172   NoDualVotes /\ UNCHANGED vars => NoDualVotes'
173 BY DEF NoDualVotes, vars

```

```

172
173 THEOREM StutterPreservesWitnessCertificatesStayAssigned ==
174   WitnessCertificatesStayAssigned /\ UNCHANGED vars => WitnessCertificatesStayAssigned'
175   BY DEF WitnessCertificatesStayAssigned, vars
176
177 THEOREM StutterPreservesCertSoundness ==
178   CertSoundness /\ UNCHANGED vars => CertSoundness'
179   BY DEF CertSoundness, vars
180
181 THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
182   BY StutterPreservesTypeInvariant, StutterPreservesWitnessAssignmentSoundness,
183     StutterPreservesNoDualVotes, StutterPreservesWitnessCertificatesStayAssigned,
184     StutterPreservesCertSoundness DEF Invariant
185
186 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
187   BY StepPreservesInvariant, StutterPreservesInvariant DEF vars
188
189 THEOREM QuorumCertificatesIntersect ==
190   \A s \in Slots,
191     b1 \in Blocks,
192     b2 \in Blocks,
193     e1 \in Epochs,
194     e2 \in Epochs,
195     Q1 \in SUBSET Validators,
196     Q2 \in SUBSET Validators :
197     /\ Invariant
198     /\ <<s, b1, e1, Q1>> \in certs
199     /\ <<s, b2, e2, Q2>> \in certs
200     => Q1 \cap Q2 # {}
201   BY SMTT(60), QuorumIntersection DEF Invariant, CertSoundness
202
203 THEOREM InvariantImpliesSafety == Invariant => Safety
204   BY SMTT(60), QuorumCertificatesIntersect DEF Invariant, Safety, NoDualVotes,
205     CertSoundness
206
207 =====

```

A.4.2 NestedGuardian.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardian.tla}

```

1  ---- MODULE NestedGuardian ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5     InitialEpoch, InitialWitness
6
7  ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
8  ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
9
10 ASSUME QuorumIntersection ==
11   \A S, T \in SUBSET Validators :
12     /\ Cardinality(S) >= QuorumSize
13     /\ Cardinality(T) >= QuorumSize
14     => S \cap T # {}
15
16 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
   ↪ witnessOnline,

```

```

17     witnessCheckpoint, assignedWitness, reassignmentDepth
18
19 vars == <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
20     witnessOnline, witnessCheckpoint, assignedWitness, reassignmentDepth>>
21
22 CheckpointLevels == 0..2
23
24 VoteEvent(v, s, b, e) == <<v, s, b, e>>
25 WitnessEvent(w, s, b, e) == <<w, s, b, e>>
26 Finalization(s, b, e) == <<s, b, e>>
27
28 VoteDomain == {VoteEvent(v, s, b, e) : v \in Validators, s \in Slots, b \in Blocks, e \in Epochs}
29 WitnessDomain == {WitnessEvent(w, s, b, e) : w \in Witnesses, s \in Slots, b \in Blocks, e \in
    ↪ Epochs}
30 FinalizationDomain == {Finalization(s, b, e) : s \in Slots, b \in Blocks, e \in Epochs}
31
32 Init ==
33     /\ votes = {}
34     /\ witnessCerts = {}
35     /\ finalized = {}
36     /\ finalizerSets = [f \in FinalizationDomain |-> {}]
37     /\ registryEpoch = [v \in Validators |-> InitialEpoch]
38     /\ guardianReady = [v \in Validators |-> TRUE]
39     /\ witnessOnline = [w \in Witnesses |-> TRUE]
40     /\ witnessCheckpoint = [w \in Witnesses |-> 1]
41     /\ assignedWitness = [s \in Slots |-> InitialWitness]
42     /\ reassignmentDepth = [s \in Slots |-> 0]
43
44 HasVote(v, s) ==
45     \E b \in Blocks, e \in Epochs : VoteEvent(v, s, b, e) \in votes
46
47 HasWitnessCert(w, s) ==
48     \E b \in Blocks, e \in Epochs : WitnessEvent(w, s, b, e) \in witnessCerts
49
50 CanIssueWitness(w, s, b, e) ==
51     /\ w \in Witnesses
52     /\ s \in Slots
53     /\ b \in Blocks
54     /\ e \in Epochs
55     /\ witnessOnline[w]
56     /\ witnessCheckpoint[w] > 0
57     /\ assignedWitness[s] = w
58     /\ ~HasWitnessCert(w, s)
59
60 IssueWitness(w, s, b, e) ==
61     /\ CanIssueWitness(w, s, b, e)
62     /\ witnessCerts' = witnessCerts \cup {WitnessEvent(w, s, b, e)}
63     /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, guardianReady, witnessOnline,
64         witnessCheckpoint, assignedWitness, reassignmentDepth>>
65
66 CanReassign(s, w) ==
67     /\ s \in Slots
68     /\ w \in Witnesses
69     /\ w # assignedWitness[s]
70     /\ ~witnessOnline[assignedWitness[s]]
71     /\ reassignmentDepth[s] < MaxReassignmentDepth
72
73 ReassignWitness(s, w) ==
74     /\ CanReassign(s, w)
75     /\ assignedWitness' = [assignedWitness EXCEPT ![s] = w]
76     /\ reassignmentDepth' = [reassignmentDepth EXCEPT ![s] = @ + 1]
77     /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,

```



```

78         witnessOnline, witnessCheckpoint>>
79
80 CanVote(v, s, b, e) ==
81     /\ v \in Validators
82     /\ s \in Slots
83     /\ b \in Blocks
84     /\ e \in Epochs
85     /\ guardianReady[v]
86     /\ registryEpoch[v] = e
87     /\ ~HasVote(v, s)
88     /\ WitnessEvent(assignedWitness[s], s, b, e) \in witnessCerts
89
90 Vote(v, s, b, e) ==
91     /\ CanVote(v, s, b, e)
92     /\ votes' = votes \cup {VoteEvent(v, s, b, e)}
93     /\ UNCHANGED <<witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
94         ↪ witnessOnline,
95         witnessCheckpoint, assignedWitness, reassignmentDepth>>
96
97 EligibleVoters(s, b, e) ==
98     {v \in Validators :
99         /\ VoteEvent(v, s, b, e) \in votes
100        /\ guardianReady[v]
101        /\ registryEpoch[v] = e
102        /\ WitnessEvent(assignedWitness[s], s, b, e) \in witnessCerts}
103
104 CanFinalize(s, b, e) ==
105     /\ s \in Slots
106     /\ b \in Blocks
107     /\ e \in Epochs
108     /\ Cardinality(EligibleVoters(s, b, e)) >= QuorumSize
109
110 Finalize(s, b, e) ==
111     /\ CanFinalize(s, b, e)
112     /\ finalized' = finalized \cup {Finalization(s, b, e)}
113     /\ finalizerSets' = [finalizerSets EXCEPT ![Finalization(s, b, e)] = EligibleVoters(s, b, e)]
114     /\ UNCHANGED <<votes, witnessCerts, registryEpoch, guardianReady, witnessOnline,
115         witnessCheckpoint, assignedWitness, reassignmentDepth>>
116
117 AdoptEpoch(v, e) ==
118     /\ v \in Validators
119     /\ e \in Epochs
120     /\ e >= registryEpoch[v]
121     /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
122     /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, guardianReady, witnessOnline,
123         witnessCheckpoint, assignedWitness, reassignmentDepth>>
124
125 RegistryRollback(v, e) ==
126     /\ v \in Validators
127     /\ e \in Epochs
128     /\ e < registryEpoch[v]
129     /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
130     /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, guardianReady, witnessOnline,
131         witnessCheckpoint, assignedWitness, reassignmentDepth>>
132
133 GuardianOutage(v) ==
134     /\ v \in Validators
135     /\ guardianReady[v]
136     /\ guardianReady' = [guardianReady EXCEPT ![v] = FALSE]
137     /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, witnessOnline,
138         witnessCheckpoint, assignedWitness, reassignmentDepth>>

```

```

139 WitnessOutage(w) ==
140   /\ w \in Witnesses
141   /\ witnessOnline[w]
142   /\ witnessOnline' = [witnessOnline EXCEPT ![w] = FALSE]
143   /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
144     witnessCheckpoint, assignedWitness, reassignmentDepth>>
145
146 WitnessRecovery(w) ==
147   /\ w \in Witnesses
148   /\ ~witnessOnline[w]
149   /\ witnessOnline' = [witnessOnline EXCEPT ![w] = TRUE]
150   /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
151     witnessCheckpoint, assignedWitness, reassignmentDepth>>
152
153 AdvanceWitnessCheckpoint(w, level) ==
154   /\ w \in Witnesses
155   /\ level \in CheckpointLevels
156   /\ level > witnessCheckpoint[w]
157   /\ witnessCheckpoint' = [witnessCheckpoint EXCEPT ![w] = level]
158   /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
159     witnessOnline, assignedWitness, reassignmentDepth>>
160
161 WitnessCheckpointRollback(w, level) ==
162   /\ w \in Witnesses
163   /\ level \in CheckpointLevels
164   /\ level < witnessCheckpoint[w]
165   /\ witnessCheckpoint' = [witnessCheckpoint EXCEPT ![w] = level]
166   /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
167     witnessOnline, assignedWitness, reassignmentDepth>>
168
169 Next ==
170   /\ \E w \in Witnesses, s \in Slots, b \in Blocks, e \in Epochs : IssueWitness(w, s, b, e)
171   /\ \E s \in Slots, w \in Witnesses : ReassignWitness(s, w)
172   /\ \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs : Vote(v, s, b, e)
173   /\ \E s \in Slots, b \in Blocks, e \in Epochs : Finalize(s, b, e)
174   /\ \E v \in Validators, e \in Epochs : AdoptEpoch(v, e)
175   /\ \E v \in Validators, e \in Epochs : RegistryRollback(v, e)
176   /\ \E v \in Validators : GuardianOutage(v)
177   /\ \E w \in Witnesses : WitnessOutage(w)
178   /\ \E w \in Witnesses : WitnessRecovery(w)
179   /\ \E w \in Witnesses, level \in CheckpointLevels : AdvanceWitnessCheckpoint(w, level)
180   /\ \E w \in Witnesses, level \in CheckpointLevels : WitnessCheckpointRollback(w, level)
181
182 TypeInvariant ==
183   /\ votes \subseq VoteDomain
184   /\ witnessCerts \subseq WitnessDomain
185   /\ finalized \subseq FinalizationDomain
186   /\ finalizerSets \in [FinalizationDomain -> SUBSET Validators]
187   /\ registryEpoch \in [Validators -> Epochs]
188   /\ guardianReady \in [Validators -> BOOLEAN]
189   /\ witnessOnline \in [Witnesses -> BOOLEAN]
190   /\ witnessCheckpoint \in [Witnesses -> CheckpointLevels]
191   /\ assignedWitness \in [Slots -> Witnesses]
192   /\ reassignmentDepth \in [Slots -> 0..MaxReassignmentDepth]
193
194 WitnessAssignmentSoundness ==
195   \A s \in Slots :
196     reassignmentDepth[s] <= MaxReassignmentDepth
197
198 NoDualVotes ==
199   \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
200     /\ VoteEvent(v, s, b1, e1) \in votes

```

```

201     /\ VoteEvent(v, s, b2, e2) \in votes
202     => /\ b1 = b2
203     /\ e1 = e2
204
205 WitnessCertificatesStayAssigned ==
206   \A w \in Witnesses, s \in Slots, b \in Blocks, e \in Epochs :
207     WitnessEvent(w, s, b, e) \in witnessCerts
208     => assignedWitness[s] = w /\ reassignmentDepth[s] > 0
209
210 FinalizationWitnessSoundness ==
211   \A s \in Slots, b \in Blocks, e \in Epochs :
212     Finalization(s, b, e) \in finalized
213     => /\ Cardinality(finalizerSets[Finalization(s, b, e)]) >= QuorumSize
214     /\ \A v \in finalizerSets[Finalization(s, b, e)] :
215       VoteEvent(v, s, b, e) \in votes
216
217 Invariant ==
218   /\ TypeInvariant
219   /\ WitnessAssignmentSoundness
220   /\ NoDualVotes
221   /\ WitnessCertificatesStayAssigned
222   /\ FinalizationWitnessSoundness
223
224 Safety ==
225   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
226     /\ Finalization(s, b1, e1) \in finalized
227     /\ Finalization(s, b2, e2) \in finalized
228     => b1 = b2
229
230 Spec ==
231   Init /\ [] [Next]_vars
232
233 THEOREM OneInCheckpointLevels == 1 \in CheckpointLevels
234   BY SMT DEF CheckpointLevels
235
236 THEOREM InitImpliesInvariant == Init => Invariant
237 PROOF
238 <1>1. Init => TypeInvariant
239   BY SMT, InitialEpochInEpochs, InitialWitnessInWitnesses, OneInCheckpointLevels
240   DEF Init, TypeInvariant, VoteDomain, WitnessDomain, FinalizationDomain
241 <1>2. Init => WitnessAssignmentSoundness
242   BY DEF Init, WitnessAssignmentSoundness
243 <1>3. Init => NoDualVotes
244   BY DEF Init, NoDualVotes
245 <1>4. Init => WitnessCertificatesStayAssigned
246   BY DEF Init, WitnessCertificatesStayAssigned
247 <1>5. Init => FinalizationWitnessSoundness
248   BY DEF Init, FinalizationWitnessSoundness
249 <1>6. QED
250   BY <1>1, <1>2, <1>3, <1>4, <1>5 DEF Invariant
251
252 THEOREM StepPreservesInvariant == Invariant /\ Next => Invariant'
253   BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
254     WitnessCertificatesStayAssigned, FinalizationWitnessSoundness, Next,
255     IssueWitness, ReassignWitness, Vote, Finalize, AdoptEpoch,
256     RegistryRollback, GuardianOutage, WitnessOutage, WitnessRecovery,
257     AdvanceWitnessCheckpoint, WitnessCheckpointRollback, CanIssueWitness,
258     CanVote, HasVote, CanFinalize, EligibleVoters, VoteEvent,
259     WitnessEvent, Finalization, VoteDomain, WitnessDomain, FinalizationDomain
260
261 THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
262   BY SMT DEF Invariant, vars

```

```

263
264 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
265   BY SMT, StepPreservesInvariant, StutterPreservesInvariant DEF vars
266
267 THEOREM FinalizedWitnessesIntersect ==
268   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
269     /\ Invariant
270     /\ <<s, b1, e1>> \in finalized
271     /\ <<s, b2, e2>> \in finalized
272     => \E v \in Validators :
273       /\ <<v, s, b1, e1>> \in votes
274       /\ <<v, s, b2, e2>> \in votes
275 PROOF
276 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
277 <1>2. ASSUME Invariant,
278       <<s, b1, e1>> \in finalized,
279       <<s, b2, e2>> \in finalized
280   PROVE \E v \in Validators :
281     /\ <<v, s, b1, e1>> \in votes
282     /\ <<v, s, b2, e2>> \in votes
283 <2>1. finalizerSets[<<s, b1, e1>>] \subseq Validators
284   BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
285 <2>2. finalizerSets[<<s, b2, e2>>] \subseq Validators
286   BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
287 <2>3. Cardinality(finalizerSets[<<s, b1, e1>>]) >= QuorumSize
288   BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
289 <2>4. Cardinality(finalizerSets[<<s, b2, e2>>]) >= QuorumSize
290   BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
291 <2>5. finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] # {}
292   BY <2>1, <2>2, <2>3, <2>4, QuorumIntersection
293 <2>6. PICK v \in finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] : TRUE
294   BY <2>5
295 <2>7. <<v, s, b1, e1>> \in votes
296   BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
297 <2>8. <<v, s, b2, e2>> \in votes
298   BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
299 <2>9. QED
300   BY SMT, <2>6, <2>7, <2>8
301 <1>3. QED
302   BY <1>2
303
304 THEOREM InvariantImpliesSafety == Invariant => Safety
305 PROOF
306 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
307 <1>2. ASSUME Invariant,
308       <<s, b1, e1>> \in finalized,
309       <<s, b2, e2>> \in finalized
310   PROVE b1 = b2
311 <2>1. \E v \in Validators :
312     /\ <<v, s, b1, e1>> \in votes
313     /\ <<v, s, b2, e2>> \in votes
314   BY SMT, FinalizedWitnessesIntersect, <1>2
315 <2>2. PICK v \in Validators :
316     /\ <<v, s, b1, e1>> \in votes
317     /\ <<v, s, b2, e2>> \in votes
318   BY <2>1
319 <2>3. QED
320   BY SMT, <1>2, <2>2 DEF Invariant, NoDualVotes
321 <1>3. QED
322   BY <1>2 DEF Safety
323
324 ====

```

A.5 CanonicalOrdering Companion Witnesses

Executable witness artifacts for omission dominance and recursive continuity.

A.5.1 CanonicalOrderingOmissionTrace.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingOmissionTrace.tla}

```
1  ---- MODULE CanonicalOrderingOmissionTrace ----
2  EXTENDS CanonicalOrdering
3
4  TargetSlot == 1
5  TargetCert == {"tx1"}
6  TargetTx == "tx2"
7
8  WitnessState ==
9    /\ TargetSlot \in closedCutoffs
10   /\ bulletin[TargetSlot] = {"tx1", "tx2"}
11   /\ CertEvent(TargetSlot, TargetCert) \in candidateCerts
12   /\ OmissionEvent(TargetSlot, TargetCert, TargetTx) \in omissionProofs
13   /\ CertEvent(TargetSlot, TargetCert) \notin admittedCerts
14
15  NoOmissionDominanceWitness == ~WitnessState
16
17  =====
```

A.5.2 CanonicalOrderingOmissionTrace.cfg

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingOmissionTrace.cfg}

```
1  SPECIFICATION Spec
2
3  CONSTANTS
4    Slots = {1}
5    Transactions = {"tx1", "tx2"}
6
7  INVARIANTS
8    TypeInvariant
9    AdmittedSoundness
10   OmissionSoundness
11   RecoveredSoundness
12   AdmittedUniqueness
13   Recoverability
14   OmissionDominates
15   NoOmissionDominanceWitness
```

A.5.3 CanonicalCollapseRecursiveContinuity.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseRecursiveContinuity.tla}

```

1  ---- MODULE CanonicalCollapseRecursiveContinuity ----
2  EXTENDS Naturals, Sequences, TLC
3
4  CONSTANT MaxHeight
5
6  Slots == 1..MaxHeight
7  Genesis == 1
8  NonGenesisSlots == Slots \ {Genesis}
9  NullHash == 0
10
11  VARIABLES publishedCollapses, publishedProofs, publishedExtensionCertificates, admittedHeaders
12
13  vars ==
14    <<publishedCollapses, publishedProofs, publishedExtensionCertificates, admittedHeaders>>
15
16  RECURSIVE AccumulatorHash(_), ProofHash(_), ProofChain(_)
17
18  PayloadHash(s) == <<"payload", s>>
19
20  AccumulatorHash(s) ==
21    IF s = Genesis
22      THEN <<"accum", s, NullHash, PayloadHash(s)>>
23      ELSE <<"accum", s, AccumulatorHash(s - 1), PayloadHash(s)>>
24
25  Commitment(s) ==
26    [height |-> s,
27     continuityAccumulatorHash |-> AccumulatorHash(s),
28     resultingStateRootHash |-> <<"state", s>>]
29
30  CommitmentHash(s) == <<"commitment-hash", Commitment(s)>>
31
32  PreviousCommitmentHash(s) ==
33    IF s = Genesis THEN NullHash ELSE CommitmentHash(s - 1)
34
35  StatementHash(s) ==
36    <<"statement-hash", Commitment(s), PreviousCommitmentHash(s), PayloadHash(s)>>
37
38  PreviousProofHash(s) ==
39    IF s = Genesis THEN NullHash ELSE ProofHash(s - 1)
40
41  ProofBytes(s) ==
42    <<"hash-pcd-v1", StatementHash(s), PreviousProofHash(s)>>
43
44  ProofStep(s) ==
45    [commitment |-> Commitment(s),
46     previousCommitmentHash |-> PreviousCommitmentHash(s),
47     payloadHash |-> PayloadHash(s),
48     previousProofHash |-> PreviousProofHash(s),
49     proofBytes |-> ProofBytes(s)]
50
51  ProofHash(s) == <<"proof-hash", ProofStep(s)>>
52
53  ProofChain(s) ==
54    IF s = Genesis
55      THEN <<ProofStep(s)>>
56      ELSE Append(ProofChain(s - 1), ProofStep(s))
57
58  ExtensionCertificate(s) ==
59    [coveredHeight |-> s,
60     predecessorCommitment |-> Commitment(s - 1),
61     predecessorProofHash |-> ProofHash(s - 1)]

```

```

62
63 Last(seq) == seq[Len(seq)]
64
65 Init ==
66   /\ publishedCollapses = {}
67   /\ publishedProofs = {}
68   /\ publishedExtensionCertificates = {}
69   /\ admittedHeaders = {}
70
71 PublishCollapseStep ==
72   \E s \in Slots :
73     /\ s \notin publishedCollapses
74     /\ publishedCollapses' = publishedCollapses \cup {s}
75     /\ UNCHANGED <<publishedProofs, publishedExtensionCertificates, admittedHeaders>>
76
77 PublishRecursiveProofStep ==
78   \E s \in Slots :
79     /\ s \in publishedCollapses
80     /\ s \notin publishedProofs
81     /\ IF s = Genesis THEN TRUE ELSE s - 1 \in publishedProofs
82     /\ publishedProofs' = publishedProofs \cup {s}
83     /\ UNCHANGED <<publishedCollapses, publishedExtensionCertificates, admittedHeaders>>
84
85 PublishExtensionCertificateStep ==
86   \E s \in NonGenesisSlots :
87     /\ s \in publishedCollapses
88     /\ s - 1 \in publishedProofs
89     /\ s \notin publishedExtensionCertificates
90     /\ publishedExtensionCertificates' = publishedExtensionCertificates \cup {s}
91     /\ UNCHANGED <<publishedCollapses, publishedProofs, admittedHeaders>>
92
93 AdmitHeaderStep ==
94   \E s \in Slots :
95     /\ s \in publishedCollapses
96     /\ s \notin admittedHeaders
97     /\ IF s = Genesis THEN TRUE ELSE s \in publishedExtensionCertificates
98     /\ admittedHeaders' = admittedHeaders \cup {s}
99     /\ UNCHANGED <<publishedCollapses, publishedProofs, publishedExtensionCertificates>>
100
101 Next ==
102   \/ PublishCollapseStep
103   \/ PublishRecursiveProofStep
104   \/ PublishExtensionCertificateStep
105   \/ AdmitHeaderStep
106
107 TypeInvariant ==
108   /\ publishedCollapses \subseq Slots
109   /\ publishedProofs \subseq Slots
110   /\ publishedExtensionCertificates \subseq NonGenesisSlots
111   /\ admittedHeaders \subseq Slots
112
113 RecursiveProofSoundness ==
114   \A s \in publishedProofs :
115     /\ s \in publishedCollapses
116     /\ IF s = Genesis THEN TRUE ELSE s - 1 \in publishedProofs
117     /\ LET step == ProofStep(s) IN
118       /\ step.commitment = Commitment(s)
119       /\ step.previousCommitmentHash = PreviousCommitmentHash(s)
120       /\ step.payloadHash = PayloadHash(s)
121       /\ step.previousProofHash = PreviousProofHash(s)
122       /\ step.proofBytes = ProofBytes(s)
123       /\ step.commitment.continuityAccumulatorHash = AccumulatorHash(s)

```

```

124
125 RecursiveProofChainSoundness ==
126   \A s \in publishedProofs :
127     /\ Len(ProofChain(s)) = s
128     /\ Last(ProofChain(s)) = ProofStep(s)
129     /\ IF s = Genesis
130       THEN TRUE
131       ELSE /\ ProofChain(s)[Len(ProofChain(s)) - 1] = ProofStep(s - 1)
132
133 ExtensionCertificateSoundness ==
134   \A s \in publishedExtensionCertificates :
135     /\ s \in NonGenesisSlots
136     /\ s - 1 \in publishedProofs
137     /\ ExtensionCertificate(s).coveredHeight = s
138     /\ ExtensionCertificate(s).predecessorCommitment = Commitment(s - 1)
139     /\ ExtensionCertificate(s).predecessorProofHash = ProofHash(s - 1)
140
141 HeaderAdmissionSoundness ==
142   \A s \in admittedHeaders :
143     /\ s \in publishedCollapses
144     /\ IF s = Genesis
145       THEN TRUE
146       ELSE /\ s \in publishedExtensionCertificates
147         /\ s - 1 \in publishedProofs
148         /\ ExtensionCertificate(s).predecessorCommitment = Commitment(s - 1)
149         /\ ExtensionCertificate(s).predecessorProofHash = ProofHash(s - 1)
150
151 Invariant ==
152   /\ TypeInvariant
153   /\ RecursiveProofSoundness
154   /\ RecursiveProofChainSoundness
155   /\ ExtensionCertificateSoundness
156   /\ HeaderAdmissionSoundness
157
158 Spec == Init /\ [] [Next]_vars
159
160 =====

```

A.5.4 CanonicalCollapseRecursiveContinuity.cfg

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseRecursiveContinuity.cfg}`

```

1 SPECIFICATION Spec
2
3 CONSTANTS
4   MaxHeight = 3
5
6 INVARIANTS
7   TypeInvariant
8   RecursiveProofSoundness
9   RecursiveProofChainSoundness
10  ExtensionCertificateSoundness
11  HeaderAdmissionSoundness

```


A.6 Recovery and Classical-Agreement Bridge

Recurring recovery kernel, finite reduction, totality bridge, and final classical-agreement collapse wrapper.

A.6.1 NestedGuardianRecoveryRecurringInductionCore.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryRecurringInductionCore.tla}

```
1  ---- MODULE NestedGuardianRecoveryRecurringInductionCore ----
2  EXTENDS Naturals, FiniteSets, TLC
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5         InitialEpoch, InitialWitness, TotalCycles,
6         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7         SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8         SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpoch \in Epochs
11 ASSUME InitialWitness \in Witnesses
12 ASSUME QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCycles \in Nat
14 ASSUME TotalCycles >= 1
15 ASSUME \A c \in 1..TotalCycles :
16     /\ TargetSlotOf(c) \in Slots
17     /\ TargetBlockOf(c) \in Blocks
18     /\ StableEpochOf(c) \in Epochs
19
20 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
21     ↪ witnessOnline,
22     witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
23     continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
24     continuationPagePublished, continuationPageBoundary, continuationFetched,
25     shareReceipts, shareConflicts, windowClosed, missingShareClaims,
26     missingThresholdCertificates, recovered, recoveredSurfaces,
27     recoveryConflicts, aborted
28
29 Core ==
30     INSTANCE NestedGuardianRecoveryRecurringLivenessCore
31     WITH Validators <- Validators,
32         Witnesses <- Witnesses,
33         Blocks <- Blocks,
34         Slots <- Slots,
35         Epochs <- Epochs,
36         QuorumSize <- QuorumSize,
37         MaxReassignmentDepth <- MaxReassignmentDepth,
38         InitialEpoch <- InitialEpoch,
39         InitialWitness <- InitialWitness,
40         TotalCycles <- TotalCycles,
41         TargetSlotOf <- TargetSlotOf,
42         TargetBlockOf <- TargetBlockOf,
43         StableEpochOf <- StableEpochOf,
44         SmallCommitteeSlot <- SmallCommitteeSlot,
45         SmallRecoveryThreshold <- SmallRecoveryThreshold,
46         LargeRecoveryThreshold <- LargeRecoveryThreshold,
47         SmallRecoveryCommittee <- SmallRecoveryCommittee,
48         LargeRecoveryCommittee <- LargeRecoveryCommittee,
```

```

48     votes <- votes,
49     witnessCerts <- witnessCerts,
50     finalized <- finalized,
51     finalizerSets <- finalizerSets,
52     registryEpoch <- registryEpoch,
53     guardianReady <- guardianReady,
54     witnessOnline <- witnessOnline,
55     witnessCheckpoint <- witnessCheckpoint,
56     assignedWitness <- assignedWitness,
57     reassignmentDepth <- reassignmentDepth,
58     currentCycle <- currentCycle,
59     phase <- phase,
60     churnStage <- churnStage,
61     continuationBoundary <- continuationBoundary,
62     continuationAnchorPublished <- continuationAnchorPublished,
63     continuationAnchorBoundary <- continuationAnchorBoundary,
64     continuationPagePublished <- continuationPagePublished,
65     continuationPageBoundary <- continuationPageBoundary,
66     continuationFetched <- continuationFetched,
67     shareReceipts <- shareReceipts,
68     shareConflicts <- shareConflicts,
69     windowClosed <- windowClosed,
70     missingShareClaims <- missingShareClaims,
71     missingThresholdCertificates <- missingThresholdCertificates,
72     recovered <- recovered,
73     recoveredSurfaces <- recoveredSurfaces,
74     recoveryConflicts <- recoveryConflicts,
75     aborted <- aborted
76
77 vars == Core!vars
78 CycleRange == Core!CycleRange
79 PrefixAdvanceRange == Core!PrefixAdvanceRange
80 RecoveryClosedPrefix(c) == Core!RecoveryClosedPrefix(c)
81
82 PriorCycleRange(c) ==
83   IF c = 1 THEN {}
84   ELSE 1..(c - 1)
85
86 RecoveryInductionBase ==
87   <>RecoveryClosedPrefix(1)
88
89 RecoveryInductionStep(c) ==
90   /\ c \in PrefixAdvanceRange
91   /\ Core!RecoveryPrefixAdvanceContract(c)
92
93 RecoveryInductionPremisesUpTo(c) ==
94   /\ c \in CycleRange
95   /\ RecoveryInductionBase
96   /\ \A i \in PriorCycleRange(c) :
97     RecoveryInductionStep(i)
98
99 RecoveryRecurringInductionPremises ==
100   /\ RecoveryInductionBase
101   /\ \A i \in PrefixAdvanceRange :
102     RecoveryInductionStep(i)
103
104 RecoveryClosedPrefixInductionKernel(c) ==
105   /\ c \in CycleRange
106   /\ (RecoveryInductionPremisesUpTo(c)
107     => <>RecoveryClosedPrefix(c))
108
109 RecoveryRecurringInductionKernel ==

```

```

110 \A c \in CycleRange :
111     RecoveryClosedPrefixInductionKernel(c)
112
113 ====

```

A.6.2 NestedGuardianRecoveryRecurringProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryRecurringProof.tla}

```

1  ---- MODULE NestedGuardianRecoveryRecurringProof ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5             InitialEpoch, InitialWitness, TotalCycles,
6             TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7             SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8             SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
11 ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
12 ASSUME QuorumSizeInValidatorRange == QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCyclesIsNat == TotalCycles \in Nat
14 ASSUME TotalCyclesAtLeastOne == TotalCycles >= 1
15 ASSUME TargetCycleDomains ==
16     \A c \in 1..TotalCycles :
17     /\ TargetSlotOf(c) \in Slots
18     /\ TargetBlockOf(c) \in Blocks
19     /\ StableEpochOf(c) \in Epochs
20
21 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
22     ↪ witnessOnline,
23     witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
24     continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
25     continuationPagePublished, continuationPageBoundary, continuationFetched,
26     shareReceipts, shareConflicts, windowClosed, missingShareClaims,
27     missingThresholdCertificates, recovered, recoveredSurfaces,
28     recoveryConflicts, aborted
29
30 Core ==
31     INSTANCE NestedGuardianRecoveryRecurringInductionCore
32     WITH Validators <- Validators,
33          Witnesses <- Witnesses,
34          Blocks <- Blocks,
35          Slots <- Slots,
36          Epochs <- Epochs,
37          QuorumSize <- QuorumSize,
38          MaxReassignmentDepth <- MaxReassignmentDepth,
39          InitialEpoch <- InitialEpoch,
40          InitialWitness <- InitialWitness,
41          TotalCycles <- TotalCycles,
42          TargetSlotOf <- TargetSlotOf,
43          TargetBlockOf <- TargetBlockOf,
44          StableEpochOf <- StableEpochOf,
45          SmallCommitteeSlot <- SmallCommitteeSlot,
46          SmallRecoveryThreshold <- SmallRecoveryThreshold,
47          LargeRecoveryThreshold <- LargeRecoveryThreshold,
48          SmallRecoveryCommittee <- SmallRecoveryCommittee,
49          LargeRecoveryCommittee <- LargeRecoveryCommittee,

```

```

49     votes <- votes,
50     witnessCerts <- witnessCerts,
51     finalized <- finalized,
52     finalizerSets <- finalizerSets,
53     registryEpoch <- registryEpoch,
54     guardianReady <- guardianReady,
55     witnessOnline <- witnessOnline,
56     witnessCheckpoint <- witnessCheckpoint,
57     assignedWitness <- assignedWitness,
58     reassignmentDepth <- reassignmentDepth,
59     currentCycle <- currentCycle,
60     phase <- phase,
61     churnStage <- churnStage,
62     continuationBoundary <- continuationBoundary,
63     continuationAnchorPublished <- continuationAnchorPublished,
64     continuationAnchorBoundary <- continuationAnchorBoundary,
65     continuationPagePublished <- continuationPagePublished,
66     continuationPageBoundary <- continuationPageBoundary,
67     continuationFetched <- continuationFetched,
68     shareReceipts <- shareReceipts,
69     shareConflicts <- shareConflicts,
70     windowClosed <- windowClosed,
71     missingShareClaims <- missingShareClaims,
72     missingThresholdCertificates <- missingThresholdCertificates,
73     recovered <- recovered,
74     recoveredSurfaces <- recoveredSurfaces,
75     recoveryConflicts <- recoveryConflicts,
76     aborted <- aborted
77
78 RecoveryRecurringClosedPrefixes ==
79   \A c \in Core!CycleRange :
80     <>Core!RecoveryClosedPrefix(c)
81
82 RecoveryParametricRecurrenceArgument ==
83   /\ Core!RecoveryRecurringInductionPremises
84   /\ Core!RecoveryRecurringInductionKernel
85   => RecoveryRecurringClosedPrefixes
86
87 THEOREM PriorCycleRangeImpliesTotalCyclesNotOne ==
88   \A c \in Core!CycleRange :
89     \A i \in Core!PriorCycleRange(c) :
90       TotalCycles # 1
91   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
92   DEF Core!CycleRange, Core!Core!CycleRange, Core!PriorCycleRange
93
94 THEOREM PriorCycleRangeImpliesPrefixAdvanceInterval ==
95   \A c \in Core!CycleRange :
96     \A i \in Core!PriorCycleRange(c) :
97       i \in 1..(TotalCycles - 1)
98   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
99   DEF Core!CycleRange, Core!Core!CycleRange, Core!PriorCycleRange
100
101 THEOREM RecoveryInductionPremisesRestrict ==
102   \A c \in Core!CycleRange :
103     Core!RecoveryRecurringInductionPremises
104     => Core!RecoveryInductionPremisesUpTo(c)
105 PROOF
106   <1>1. TAKE c \in Core!CycleRange
107   <1>2. ASSUME Core!RecoveryRecurringInductionPremises
108     PROVE Core!RecoveryInductionPremisesUpTo(c)
109   <2>1. Core!RecoveryInductionBase
110     BY <1>2 DEF Core!RecoveryRecurringInductionPremises

```

```

111 <2>2. \A i \in Core!PriorCycleRange(c) :
112     Core!RecoveryInductionStep(i)
113 PROOF
114     <3>1. TAKE i \in Core!PriorCycleRange(c)
115     <3>2. i \in Core!PrefixAdvanceRange
116         PROOF
117             <4>1. TotalCycles # 1
118                 BY PriorCycleRangeImpliesTotalCyclesNotOne, <1>1, <3>1
119             <4>2. i \in 1..(TotalCycles - 1)
120                 BY PriorCycleRangeImpliesPrefixAdvanceInterval, <1>1, <3>1
121             <4>3. TotalCycles # 1
122                 BY <4>1
123             <4>6. QED
124                 BY <4>3, <4>2
125                 DEF Core!PrefixAdvanceRange, Core!Core!PrefixAdvanceRange
126             <3>3. Core!RecoveryInductionStep(i)
127                 BY <1>2, <3>2 DEF Core!RecoveryRecurringInductionPremises
128             <3>4. QED
129                 BY <3>3
130         <2>3. QED
131             BY <1>1, <2>1, <2>2
132             DEF Core!RecoveryInductionPremisesUpTo
133     <1>3. QED
134         BY <1>2
135
136 THEOREM RecoveryInductionKernelRestrict ==
137     \A c \in Core!CycleRange :
138         Core!RecoveryRecurringInductionKernel
139         => Core!RecoveryClosedPrefixInductionKernel(c)
140 BY DEF Core!RecoveryRecurringInductionKernel,
141     Core!RecoveryClosedPrefixInductionKernel,
142     Core!CycleRange
143
144 THEOREM RecoveryRecurringClosedPrefixFromKernel ==
145     \A c \in Core!CycleRange :
146         /\ Core!RecoveryRecurringInductionPremises
147         /\ Core!RecoveryRecurringInductionKernel
148         => <>Core!RecoveryClosedPrefix(c)
149 PROOF
150     <1>1. TAKE c \in Core!CycleRange
151     <1>2. ASSUME Core!RecoveryRecurringInductionPremises,
152             Core!RecoveryRecurringInductionKernel
153             PROVE <>Core!RecoveryClosedPrefix(c)
154     <2>1. Core!RecoveryInductionPremisesUpTo(c)
155         BY RecoveryInductionPremisesRestrict, <1>1, <1>2
156     <2>2. Core!RecoveryClosedPrefixInductionKernel(c)
157         BY RecoveryInductionKernelRestrict, <1>1, <1>2
158     <2>3. QED
159         BY <2>1, <2>2
160         DEF Core!RecoveryClosedPrefixInductionKernel
161     <1>3. QED
162         BY <1>2
163
164 THEOREM RecoveryParametricRecurrenceArgumentTheorem ==
165     RecoveryParametricRecurrenceArgument
166 BY RecoveryRecurringClosedPrefixFromKernel
167     DEF RecoveryParametricRecurrenceArgument,
168     RecoveryRecurringClosedPrefixes
169
170 ====

```

A.6.3 NestedGuardianRecoveryClassicalAgreementReduction.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryClassicalAgreementReduction.tla}

```
1  ---- MODULE NestedGuardianRecoveryClassicalAgreementReduction ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5         InitialEpoch, InitialWitness, TotalCycles,
6         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7         SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8         SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpoch \in Epochs
11 ASSUME InitialWitness \in Witnesses
12 ASSUME QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCycles \in Nat
14 ASSUME TotalCycles >= 1
15 ASSUME \A c \in 1..TotalCycles :
16     /\ TargetSlotOf(c) \in Slots
17     /\ TargetBlockOf(c) \in Blocks
18     /\ StableEpochOf(c) \in Epochs
19
20 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
21     ↪ witnessOnline,
22     witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
23     continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
24     continuationPagePublished, continuationPageBoundary, continuationFetched,
25     shareReceipts, shareConflicts, windowClosed, missingShareClaims,
26     missingThresholdCertificates, recovered, recoveredSurfaces,
27     recoveryConflicts, aborted
28
29 Proof ==
30     INSTANCE NestedGuardianRecoveryRecurringProof
31     WITH Validators <- Validators,
32         Witnesses <- Witnesses,
33         Blocks <- Blocks,
34         Slots <- Slots,
35         Epochs <- Epochs,
36         QuorumSize <- QuorumSize,
37         MaxReassignmentDepth <- MaxReassignmentDepth,
38         InitialEpoch <- InitialEpoch,
39         InitialWitness <- InitialWitness,
40         TotalCycles <- TotalCycles,
41         TargetSlotOf <- TargetSlotOf,
42         TargetBlockOf <- TargetBlockOf,
43         StableEpochOf <- StableEpochOf,
44         SmallCommitteeSlot <- SmallCommitteeSlot,
45         SmallRecoveryThreshold <- SmallRecoveryThreshold,
46         LargeRecoveryThreshold <- LargeRecoveryThreshold,
47         SmallRecoveryCommittee <- SmallRecoveryCommittee,
48         LargeRecoveryCommittee <- LargeRecoveryCommittee,
49         votes <- votes,
50         witnessCerts <- witnessCerts,
51         finalized <- finalized,
52         finalizerSets <- finalizerSets,
53         registryEpoch <- registryEpoch,
54         guardianReady <- guardianReady,
55         witnessOnline <- witnessOnline,
```

```

55     witnessCheckpoint <- witnessCheckpoint,
56     assignedWitness <- assignedWitness,
57     reassignmentDepth <- reassignmentDepth,
58     currentCycle <- currentCycle,
59     phase <- phase,
60     churnStage <- churnStage,
61     continuationBoundary <- continuationBoundary,
62     continuationAnchorPublished <- continuationAnchorPublished,
63     continuationAnchorBoundary <- continuationAnchorBoundary,
64     continuationPagePublished <- continuationPagePublished,
65     continuationPageBoundary <- continuationPageBoundary,
66     continuationFetched <- continuationFetched,
67     shareReceipts <- shareReceipts,
68     shareConflicts <- shareConflicts,
69     windowClosed <- windowClosed,
70     missingShareClaims <- missingShareClaims,
71     missingThresholdCertificates <- missingThresholdCertificates,
72     recovered <- recovered,
73     recoveredSurfaces <- recoveredSurfaces,
74     recoveryConflicts <- recoveryConflicts,
75     aborted <- aborted
76
77 CycleRange == 1..TotalCycles
78
79 ClassicalAgreementDecisionObject(c) ==
80   [cycle |-> c,
81    slot |-> TargetSlotOf(c),
82    block |-> TargetBlockOf(c),
83    epoch |-> StableEpochOf(c)]
84
85 ClassicalAgreementPrefixObject(c) ==
86   [i \in 1..c |-> ClassicalAgreementDecisionObject(i)]
87
88 FiniteClassicalAgreementPrefixRealized(c) ==
89   /\ c \in CycleRange
90   /\ <>Proof!Core!RecoveryClosedPrefix(c)
91
92 FiniteClassicalAgreementLiveness ==
93   \A c \in CycleRange :
94     FiniteClassicalAgreementPrefixRealized(c)
95
96 ClassicalAgreementFiniteReduction ==
97   Proof!RecoveryRecurringClosedPrefixes
98   => FiniteClassicalAgreementLiveness
99
100 THEOREM ProofCycleRangeMatchesCycleRange ==
101   Proof!Core!CycleRange = CycleRange
102   BY DEF Proof!Core!CycleRange, Proof!Core!Core!CycleRange, CycleRange
103
104 THEOREM FiniteClassicalAgreementPrefixFromRecurringPrefixes ==
105   \A c \in CycleRange :
106     Proof!RecoveryRecurringClosedPrefixes
107     => FiniteClassicalAgreementPrefixRealized(c)
108 PROOF
109   <1>1. TAKE c \in CycleRange
110   <1>2. ASSUME Proof!RecoveryRecurringClosedPrefixes
111     PROVE FiniteClassicalAgreementPrefixRealized(c)
112     <2>1. c \in Proof!Core!CycleRange
113       BY ProofCycleRangeMatchesCycleRange, <1>1
114     <2>2. <>Proof!Core!RecoveryClosedPrefix(c)
115       BY <1>2, <2>1 DEF Proof!RecoveryRecurringClosedPrefixes
116     <2>3. QED

```

```

117         BY <1>1, <2>2 DEF FiniteClassicalAgreementPrefixRealized
118     <1>3. QED
119     BY <1>2
120
121 THEOREM ClassicalAgreementFiniteReductionTheorem ==
122     ClassicalAgreementFiniteReduction
123     BY FiniteClassicalAgreementPrefixFromRecurringPrefixes
124     DEF ClassicalAgreementFiniteReduction,
125         FiniteClassicalAgreementLiveness
126
127 ====

```

A.6.4 NestedGuardianRecoveryClassicalAgreementTotality.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryClassicalAgreementTotality.tla}

```

1  ---- MODULE NestedGuardianRecoveryClassicalAgreementTotality ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5         InitialEpoch, InitialWitness, TotalCycles,
6         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7         SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8         SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
11 ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
12 ASSUME QuorumSizeInValidatorRange == QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCyclesIsNat == TotalCycles \in Nat
14 ASSUME TotalCyclesAtLeastOne == TotalCycles >= 1
15 ASSUME TargetCycleDomains ==
16     \A c \in 1..TotalCycles :
17     /\ TargetSlotOf(c) \in Slots
18     /\ TargetBlockOf(c) \in Blocks
19     /\ StableEpochOf(c) \in Epochs
20
21 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
22     ↪ witnessOnline,
23     witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
24     continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
25     continuationPagePublished, continuationPageBoundary, continuationFetched,
26     shareReceipts, shareConflicts, windowClosed, missingShareClaims,
27     missingThresholdCertificates, recovered, recoveredSurfaces,
28     recoveryConflicts, aborted
29
30 Reduction ==
31     INSTANCE NestedGuardianRecoveryClassicalAgreementReduction
32     WITH Validators <- Validators,
33         Witnesses <- Witnesses,
34         Blocks <- Blocks,
35         Slots <- Slots,
36         Epochs <- Epochs,
37         QuorumSize <- QuorumSize,
38         MaxReassignmentDepth <- MaxReassignmentDepth,
39         InitialEpoch <- InitialEpoch,
40         InitialWitness <- InitialWitness,
41         TotalCycles <- TotalCycles,
42         TargetSlotOf <- TargetSlotOf,

```



```

42     TargetBlockOf <- TargetBlockOf,
43     StableEpochOf <- StableEpochOf,
44     SmallCommitteeSlot <- SmallCommitteeSlot,
45     SmallRecoveryThreshold <- SmallRecoveryThreshold,
46     LargeRecoveryThreshold <- LargeRecoveryThreshold,
47     SmallRecoveryCommittee <- SmallRecoveryCommittee,
48     LargeRecoveryCommittee <- LargeRecoveryCommittee,
49     votes <- votes,
50     witnessCerts <- witnessCerts,
51     finalized <- finalized,
52     finalizerSets <- finalizerSets,
53     registryEpoch <- registryEpoch,
54     guardianReady <- guardianReady,
55     witnessOnline <- witnessOnline,
56     witnessCheckpoint <- witnessCheckpoint,
57     assignedWitness <- assignedWitness,
58     reassignmentDepth <- reassignmentDepth,
59     currentCycle <- currentCycle,
60     phase <- phase,
61     churnStage <- churnStage,
62     continuationBoundary <- continuationBoundary,
63     continuationAnchorPublished <- continuationAnchorPublished,
64     continuationAnchorBoundary <- continuationAnchorBoundary,
65     continuationPagePublished <- continuationPagePublished,
66     continuationPageBoundary <- continuationPageBoundary,
67     continuationFetched <- continuationFetched,
68     shareReceipts <- shareReceipts,
69     shareConflicts <- shareConflicts,
70     windowClosed <- windowClosed,
71     missingShareClaims <- missingShareClaims,
72     missingThresholdCertificates <- missingThresholdCertificates,
73     recovered <- recovered,
74     recoveredSurfaces <- recoveredSurfaces,
75     recoveryConflicts <- recoveryConflicts,
76     aborted <- aborted
77
78 CycleRange == 1..TotalCycles
79
80 RecoveryRecurringClosedPrefixes == Reduction!Proof!RecoveryRecurringClosedPrefixes
81 RecoveryRecurringInductionPremises == Reduction!Proof!Core!RecoveryRecurringInductionPremises
82 RecoveryRecurringInductionKernel == Reduction!Proof!Core!RecoveryRecurringInductionKernel
83 RecoveryClosedPrefix(c) == Reduction!Proof!Core!RecoveryClosedPrefix(c)
84
85 ClassicalAgreementTotalHistoryObject ==
86   [i \in CycleRange |> Reduction!ClassicalAgreementDecisionObject(i)]
87
88 ClassicalAgreementTotalHistoryExtendsFinitePrefixes ==
89   \A c \in CycleRange :
90     [i \in 1..c |> ClassicalAgreementTotalHistoryObject[i]]
91     = Reduction!ClassicalAgreementPrefixObject(c)
92
93 TotalClassicalAgreementHistoryRealized ==
94   /\ <>RecoveryClosedPrefix(TotalCycles)
95   /\ ClassicalAgreementTotalHistoryExtendsFinitePrefixes
96
97 TotalClassicalAgreementReduction ==
98   Reduction!FiniteClassicalAgreementLiveness
99   => TotalClassicalAgreementHistoryRealized
100
101 TotalClassicalAgreementFromRecurringPrefixes ==
102   RecoveryRecurringClosedPrefixes
103   => TotalClassicalAgreementHistoryRealized

```

```

104
105 TotalClassicalAgreementFromRecurrenceKernel ==
106   /\ RecoveryRecurringInductionPremises
107   /\ RecoveryRecurringInductionKernel
108   => TotalClassicalAgreementHistoryRealized
109
110 THEOREM TotalHistoryEntryMatchesDecisionObject ==
111   \A c \in CycleRange :
112     \A i \in 1..c :
113       ClassicalAgreementTotalHistoryObject[i]
114       = Reduction!ClassicalAgreementDecisionObject(i)
115   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
116   DEF CycleRange, ClassicalAgreementTotalHistoryObject
117
118 THEOREM ClassicalAgreementTotalHistoryExtendsFinitePrefixesTheorem ==
119   ClassicalAgreementTotalHistoryExtendsFinitePrefixes
120 PROOF
121   <1>1. \A c \in CycleRange :
122     [i \in 1..c |-> ClassicalAgreementTotalHistoryObject[i]]
123     = Reduction!ClassicalAgreementPrefixObject(c)
124   PROOF
125     <2>1. TAKE c \in CycleRange
126     <2>2. \A i \in 1..c :
127       ClassicalAgreementTotalHistoryObject[i]
128       = Reduction!ClassicalAgreementDecisionObject(i)
129     BY TotalHistoryEntryMatchesDecisionObject, <2>1
130     <2>3. QED
131     BY IsaWithSetExtensionality, <2>1, <2>2
132     DEF Reduction!ClassicalAgreementPrefixObject
133   <1>2. QED
134   BY <1>1 DEF ClassicalAgreementTotalHistoryExtendsFinitePrefixes
135
136 THEOREM TotalCyclesInCycleRange ==
137   TotalCycles \in CycleRange
138   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne DEF CycleRange
139
140 THEOREM TotalFinitePrefixFromFiniteLiveness ==
141   Reduction!FiniteClassicalAgreementLiveness
142   => Reduction!FiniteClassicalAgreementPrefixRealized(TotalCycles)
143 PROOF
144   <1>1. ASSUME Reduction!FiniteClassicalAgreementLiveness
145   PROVE Reduction!FiniteClassicalAgreementPrefixRealized(TotalCycles)
146   <2>1. TotalCycles \in Reduction!CycleRange
147   BY TotalCyclesInCycleRange DEF CycleRange, Reduction!CycleRange
148   <2>2. QED
149   BY <1>1, <2>1 DEF Reduction!FiniteClassicalAgreementLiveness
150   <1>2. QED
151   BY <1>1
152
153 THEOREM TotalRecoveryClosedPrefixFromFiniteLiveness ==
154   Reduction!FiniteClassicalAgreementLiveness
155   => <>RecoveryClosedPrefix(TotalCycles)
156   BY TotalFinitePrefixFromFiniteLiveness
157   DEF Reduction!FiniteClassicalAgreementPrefixRealized,
158     RecoveryClosedPrefix
159
160 THEOREM TotalClassicalAgreementReductionTheorem ==
161   TotalClassicalAgreementReduction
162   BY TotalRecoveryClosedPrefixFromFiniteLiveness,
163     ClassicalAgreementTotalHistoryExtendsFinitePrefixesTheorem
164   DEF TotalClassicalAgreementReduction,
165     TotalClassicalAgreementHistoryRealized

```

```

166
167 THEOREM ReductionLivenessFromRecurringPrefixes ==
168   RecoveryRecurringClosedPrefixes
169   => Reduction!FiniteClassicalAgreementLiveness
170 PROOF
171   <1>1. ASSUME RecoveryRecurringClosedPrefixes
172     PROVE Reduction!FiniteClassicalAgreementLiveness
173     <2>1. \A c \in Reduction!CycleRange :
174       Reduction!FiniteClassicalAgreementPrefixRealized(c)
175       PROOF
176         <3>1. TAKE c \in Reduction!CycleRange
177         <3>2. c \in Reduction!Proof!Core!CycleRange
178           BY <3>1
179             DEF Reduction!CycleRange,
180               Reduction!Proof!Core!CycleRange,
181               Reduction!Proof!Core!Core!CycleRange
182         <3>3. Reduction!Proof!RecoveryRecurringClosedPrefixes
183           BY <1>1 DEF RecoveryRecurringClosedPrefixes
184         <3>4. <>Reduction!Proof!Core!RecoveryClosedPrefix(c)
185           BY <3>2, <3>3 DEF Reduction!Proof!RecoveryRecurringClosedPrefixes
186         <3>5. QED
187           BY <3>1, <3>4 DEF Reduction!FiniteClassicalAgreementPrefixRealized
188     <2>2. QED
189       BY <2>1 DEF Reduction!FiniteClassicalAgreementLiveness
190   <1>2. QED
191     BY <1>1
192
193 THEOREM TotalClassicalAgreementFromRecurringPrefixesTheorem ==
194   TotalClassicalAgreementFromRecurringPrefixes
195 PROOF
196   <1>1. ASSUME RecoveryRecurringClosedPrefixes
197     PROVE TotalClassicalAgreementHistoryRealized
198     <2>1. Reduction!FiniteClassicalAgreementLiveness
199       BY ReductionLivenessFromRecurringPrefixes, <1>1
200     <2>2. Reduction!FiniteClassicalAgreementLiveness
201       => TotalClassicalAgreementHistoryRealized
202       BY TotalClassicalAgreementReductionTheorem
203       DEF TotalClassicalAgreementReduction
204     <2>3. QED
205       BY <2>1, <2>2
206   <1>2. QED
207     BY <1>1 DEF TotalClassicalAgreementFromRecurringPrefixes
208
209 THEOREM RecoveryRecurringClosedPrefixesFromRecurrenceKernel ==
210   /\ RecoveryRecurringInductionPremises
211   /\ RecoveryRecurringInductionKernel
212   => RecoveryRecurringClosedPrefixes
213 PROOF
214   <1>1. ASSUME RecoveryRecurringInductionPremises,
215     RecoveryRecurringInductionKernel
216     PROVE RecoveryRecurringClosedPrefixes
217     <2>1. \A c \in Reduction!Proof!Core!CycleRange :
218       <>Reduction!Proof!Core!RecoveryClosedPrefix(c)
219     PROOF
220       <3>1. TAKE c \in Reduction!Proof!Core!CycleRange
221       <3>2. Reduction!Proof!Core!RecoveryInductionPremisesUpTo(c)
222         PROOF
223           <4>1. Reduction!Proof!Core!RecoveryInductionBase
224             BY <1>1
225             DEF RecoveryRecurringInductionPremises,
226               Reduction!Proof!Core!RecoveryRecurringInductionPremises
227       <4>2. \A i \in Reduction!Proof!Core!PriorCycleRange(c) :

```

```

228         Reduction!Proof!Core!RecoveryInductionStep(i)
229     PROOF
230         <5>1. TAKE i \in Reduction!Proof!Core!PriorCycleRange(c)
231         <5>2. TotalCycles # 1
232             BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne, <3>1, <5>1
233             DEF Reduction!Proof!Core!CycleRange,
234                 Reduction!Proof!Core!Core!CycleRange,
235                 Reduction!Proof!Core!PriorCycleRange
236         <5>3. i \in 1..(TotalCycles - 1)
237             BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne, <3>1, <5>1
238             DEF Reduction!Proof!Core!CycleRange,
239                 Reduction!Proof!Core!Core!CycleRange,
240                 Reduction!Proof!Core!PriorCycleRange
241         <5>4. i \in Reduction!Proof!Core!PrefixAdvanceRange
242             BY <5>2, <5>3
243             DEF Reduction!Proof!Core!PrefixAdvanceRange,
244                 Reduction!Proof!Core!Core!PrefixAdvanceRange
245         <5>5. QED
246             BY <1>1, <5>4
247             DEF RecoveryRecurringInductionPremises,
248                 Reduction!Proof!Core!RecoveryRecurringInductionPremises
249         <4>3. QED
250             BY <3>1, <4>1, <4>2
251             DEF Reduction!Proof!Core!RecoveryInductionPremisesUpTo
252         <3>3. Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel(c)
253             BY <1>1, <3>1
254             DEF RecoveryRecurringInductionKernel,
255                 Reduction!Proof!Core!RecoveryRecurringInductionKernel
256         <3>4. QED
257             BY <3>2, <3>3
258             DEF Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel
259         <2>2. QED
260             BY <2>1 DEF RecoveryRecurringClosedPrefixes,
261                 Reduction!Proof!RecoveryRecurringClosedPrefixes
262         <1>2. QED
263             BY <1>1
264             DEF RecoveryRecurringClosedPrefixes,
265                 RecoveryRecurringInductionPremises,
266                 RecoveryRecurringInductionKernel,
267                 Reduction!Proof!RecoveryRecurringClosedPrefixes
268
269     THEOREM TotalClassicalAgreementFromRecurrenceKernelTheorem ==
270         TotalClassicalAgreementFromRecurrenceKernel
271     PROOF
272         <1>1. ASSUME RecoveryRecurringInductionPremises,
273             RecoveryRecurringInductionKernel
274             PROVE TotalClassicalAgreementHistoryRealized
275         <2>1. RecoveryRecurringClosedPrefixes
276             BY RecoveryRecurringClosedPrefixesFromRecurrenceKernel, <1>1
277         <2>2. TotalClassicalAgreementFromRecurringPrefixes
278             BY TotalClassicalAgreementFromRecurringPrefixesTheorem
279         <2>3. QED
280             BY <2>1, <2>2 DEF TotalClassicalAgreementFromRecurringPrefixes
281         <1>2. QED
282             BY <1>1 DEF TotalClassicalAgreementFromRecurrenceKernel
283
284     =====

```

A.6.5 NestedGuardianRecoveryClassicalAgreementCollapse.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryClassicalAgreementCollapse.tla}

```
1  ---- MODULE NestedGuardianRecoveryClassicalAgreementCollapse ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5         InitialEpoch, InitialWitness, TotalCycles,
6         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7         SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8         SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
11 ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
12 ASSUME QuorumSizeInValidatorRange == QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCyclesIsNat == TotalCycles \in Nat
14 ASSUME TotalCyclesAtLeastOne == TotalCycles >= 1
15 ASSUME TargetCycleDomains ==
16     \A c \in 1..TotalCycles :
17     /\ TargetSlotOf(c) \in Slots
18     /\ TargetBlockOf(c) \in Blocks
19     /\ StableEpochOf(c) \in Epochs
20
21 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
22     ↪ witnessOnline,
23     witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
24     continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
25     continuationPagePublished, continuationPageBoundary, continuationFetched,
26     shareReceipts, shareConflicts, windowClosed, missingShareClaims,
27     missingThresholdCertificates, recovered, recoveredSurfaces,
28     recoveryConflicts, aborted
29
30 Totality ==
31     INSTANCE NestedGuardianRecoveryClassicalAgreementTotality
32     WITH Validators <- Validators,
33         Witnesses <- Witnesses,
34         Blocks <- Blocks,
35         Slots <- Slots,
36         Epochs <- Epochs,
37         QuorumSize <- QuorumSize,
38         MaxReassignmentDepth <- MaxReassignmentDepth,
39         InitialEpoch <- InitialEpoch,
40         InitialWitness <- InitialWitness,
41         TotalCycles <- TotalCycles,
42         TargetSlotOf <- TargetSlotOf,
43         TargetBlockOf <- TargetBlockOf,
44         StableEpochOf <- StableEpochOf,
45         SmallCommitteeSlot <- SmallCommitteeSlot,
46         SmallRecoveryThreshold <- SmallRecoveryThreshold,
47         LargeRecoveryThreshold <- LargeRecoveryThreshold,
48         SmallRecoveryCommittee <- SmallRecoveryCommittee,
49         LargeRecoveryCommittee <- LargeRecoveryCommittee,
50         votes <- votes,
51         witnessCerts <- witnessCerts,
52         finalized <- finalized,
53         finalizerSets <- finalizerSets,
54         registryEpoch <- registryEpoch,
55         guardianReady <- guardianReady,
```

```

55     witnessOnline <- witnessOnline,
56     witnessCheckpoint <- witnessCheckpoint,
57     assignedWitness <- assignedWitness,
58     reassignmentDepth <- reassignmentDepth,
59     currentCycle <- currentCycle,
60     phase <- phase,
61     churnStage <- churnStage,
62     continuationBoundary <- continuationBoundary,
63     continuationAnchorPublished <- continuationAnchorPublished,
64     continuationAnchorBoundary <- continuationAnchorBoundary,
65     continuationPagePublished <- continuationPagePublished,
66     continuationPageBoundary <- continuationPageBoundary,
67     continuationFetched <- continuationFetched,
68     shareReceipts <- shareReceipts,
69     shareConflicts <- shareConflicts,
70     windowClosed <- windowClosed,
71     missingShareClaims <- missingShareClaims,
72     missingThresholdCertificates <- missingThresholdCertificates,
73     recovered <- recovered,
74     recoveredSurfaces <- recoveredSurfaces,
75     recoveryConflicts <- recoveryConflicts,
76     aborted <- aborted
77
78 CycleRange == Totality!CycleRange
79
80 ClassicalAgreementDecision(c) ==
81   [cycle |-> c,
82    slot |-> TargetSlotOf(c),
83    block |-> TargetBlockOf(c),
84    epoch |-> StableEpochOf(c)]
85
86 ClassicalAgreementHistory(history) ==
87   \A i \in CycleRange :
88     history[i] = ClassicalAgreementDecision(i)
89
90 OrdinaryClassicalAgreementHistory(history) ==
91   /\ ClassicalAgreementHistory(history)
92   /\ <>Totality!RecoveryClosedPrefix(TotalCycles)
93
94 UnconditionalClassicalAgreementSentence ==
95   OrdinaryClassicalAgreementHistory(
96     Totality!ClassicalAgreementTotalHistoryObject
97   )
98
99 THEOREM TotalHistoryMatchesCollapsedClassicalHistory ==
100   ClassicalAgreementHistory(Totality!ClassicalAgreementTotalHistoryObject)
101 PROOF
102   <1>1. \A i \in CycleRange :
103     Totality!ClassicalAgreementTotalHistoryObject[i]
104       = ClassicalAgreementDecision(i)
105   PROOF
106     <2>1. TAKE i \in CycleRange
107     <2>2. QED
108     BY <2>1
109       DEF ClassicalAgreementDecision,
110         CycleRange,
111         Totality!ClassicalAgreementTotalHistoryObject,
112         Totality!Reduction!ClassicalAgreementDecisionObject
113   <1>2. QED
114     BY <1>1 DEF ClassicalAgreementHistory
115
116 THEOREM TotalityCyclePrefixEntriesStayInCycleRange ==

```

```

117 \A c \in Totality!CycleRange :
118   \A i \in 1..c :
119     i \in Totality!CycleRange
120 BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
121 DEF Totality!CycleRange
122
123 THEOREM TotalityReductionCorePriorCycleRangeImpliesPrefixAdvanceRange ==
124 \A c \in Totality!Reduction!Proof!Core!CycleRange :
125   \A i \in Totality!Reduction!Proof!Core!PriorCycleRange(c) :
126     i \in Totality!Reduction!Proof!Core!PrefixAdvanceRange
127 BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
128 DEF Totality!Reduction!Proof!Core!CycleRange,
129   Totality!Reduction!Proof!Core!Core!CycleRange,
130   Totality!Reduction!Proof!Core!PriorCycleRange,
131   Totality!Reduction!Proof!Core!PrefixAdvanceRange,
132   Totality!Reduction!Proof!Core!Core!PrefixAdvanceRange
133
134 THEOREM LocalReductionLivenessFromRecurringPrefixesTheorem ==
135   Totality!RecoveryRecurringClosedPrefixes
136 => Totality!Reduction!FiniteClassicalAgreementLiveness
137 PROOF
138 <1>1. ASSUME Totality!RecoveryRecurringClosedPrefixes
139   PROVE Totality!Reduction!FiniteClassicalAgreementLiveness
140 <2>1. \A c \in Totality!Reduction!CycleRange :
141   Totality!Reduction!FiniteClassicalAgreementPrefixRealized(c)
142   PROOF
143   <3>1. TAKE c \in Totality!Reduction!CycleRange
144   <3>2. c \in Totality!Reduction!Proof!Core!CycleRange
145   BY <3>1
146   DEF Totality!Reduction!CycleRange,
147   Totality!Reduction!Proof!Core!CycleRange,
148   Totality!Reduction!Proof!Core!Core!CycleRange
149   <3>3. <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
150   BY <1>1, <3>2
151   DEF Totality!RecoveryRecurringClosedPrefixes,
152   Totality!Reduction!Proof!RecoveryRecurringClosedPrefixes
153   <3>4. QED
154   BY <3>1, <3>3
155   DEF Totality!Reduction!FiniteClassicalAgreementPrefixRealized
156   <2>2. QED
157   BY <2>1 DEF Totality!Reduction!FiniteClassicalAgreementLiveness
158 <1>2. QED
159 BY <1>1
160
161 THEOREM LocalTotalClassicalAgreementHistoryFromFiniteLivenessTheorem ==
162   Totality!Reduction!FiniteClassicalAgreementLiveness
163 => Totality!TotalClassicalAgreementHistoryRealized
164 PROOF
165 <1>1. ASSUME Totality!Reduction!FiniteClassicalAgreementLiveness
166   PROVE Totality!TotalClassicalAgreementHistoryRealized
167 <2>1. TotalCycles \in Totality!Reduction!CycleRange
168   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
169   DEF Totality!Reduction!CycleRange
170 <2>2. Totality!Reduction!FiniteClassicalAgreementPrefixRealized(TotalCycles)
171   BY <1>1, <2>1 DEF Totality!Reduction!FiniteClassicalAgreementLiveness
172 <2>3. <>Totality!RecoveryClosedPrefix(TotalCycles)
173   BY <2>2
174   DEF Totality!Reduction!FiniteClassicalAgreementPrefixRealized,
175   Totality!RecoveryClosedPrefix
176 <2>4. Totality!ClassicalAgreementTotalHistoryExtendsFinitePrefixes
177   PROOF
178   <3>1. \A c \in Totality!CycleRange :

```

```

179         [i \in 1..c |-> Totality!ClassicalAgreementTotalHistoryObject[i]]
180         = Totality!Reduction!ClassicalAgreementPrefixObject(c)
181     PROOF
182         <4>1. TAKE c \in Totality!CycleRange
183         <4>2. \A i \in 1..c :
184             Totality!ClassicalAgreementTotalHistoryObject[i]
185             = Totality!Reduction!ClassicalAgreementDecisionObject(i)
186     PROOF
187         <5>1. TAKE i \in 1..c
188         <5>2. c \in 1..TotalCycles
189             BY <4>1 DEF Totality!CycleRange
190         <5>3. i \in Totality!CycleRange
191             BY TotalityCyclePrefixEntriesStayInCycleRange, <4>1, <5>1
192         <5>4. QED
193             BY <5>3
194             DEF Totality!ClassicalAgreementTotalHistoryObject
195         <4>3. QED
196             BY IsaWithSetExtensionality, <4>1, <4>2
197             DEF Totality!Reduction!ClassicalAgreementPrefixObject
198         <3>2. QED
199             BY <3>1 DEF Totality!ClassicalAgreementTotalHistoryExtendsFinitePrefixes
200         <2>5. QED
201             BY <2>3, <2>4 DEF Totality!TotalClassicalAgreementHistoryRealized
202     <1>2. QED
203         BY <1>1
204
205 THEOREM LocalRecurringPrefixesFromRecurrenceKernelTheorem ==
206     /\ Totality!RecoveryRecurringInductionPremises
207     /\ Totality!RecoveryRecurringInductionKernel
208     => Totality!RecoveryRecurringClosedPrefixes
209 PROOF
210     <1>1. ASSUME Totality!RecoveryRecurringInductionPremises,
211             Totality!RecoveryRecurringInductionKernel
212     PROVE Totality!RecoveryRecurringClosedPrefixes
213     <2>1. \A c \in Totality!Reduction!Proof!Core!CycleRange :
214         <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
215     PROOF
216         <3>1. TAKE c \in Totality!Reduction!Proof!Core!CycleRange
217         <3>2. /\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
218             /\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionKernel
219             BY <1>1
220             DEF Totality!RecoveryRecurringInductionPremises,
221             Totality!RecoveryRecurringInductionKernel
222         <3>3. Totality!Reduction!Proof!Core!RecoveryInductionPremisesUpTo(c)
223     PROOF
224         <4>1. Totality!Reduction!Proof!Core!RecoveryInductionBase
225             BY <3>2
226             DEF Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
227         <4>2. \A i \in Totality!Reduction!Proof!Core!PriorCycleRange(c) :
228             Totality!Reduction!Proof!Core!RecoveryInductionStep(i)
229     PROOF
230         <5>1. TAKE i \in Totality!Reduction!Proof!Core!PriorCycleRange(c)
231         <5>2. i \in Totality!Reduction!Proof!Core!PrefixAdvanceRange
232             BY TotalityReductionCorePriorCycleRangeImpliesPrefixAdvanceRange,
233             <3>1, <5>1
234         <5>3. QED
235             BY <3>2, <5>2
236             DEF Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
237         <4>3. QED
238             BY <3>1, <4>1, <4>2
239             DEF Totality!Reduction!Proof!Core!RecoveryInductionPremisesUpTo
240         <3>4. Totality!Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel(c)

```



```

241         BY <3>1, <3>2
242         DEF Totality!Reduction!Proof!Core!RecoveryRecurringInductionKernel
243     <3>5. <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
244         BY <3>3, <3>4
245         DEF Totality!Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel
246     <3>6. (& Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
247         /\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionKernel)
248         => <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
249         BY <3>5
250     <3>7. QED
251         BY <3>5
252 <2>2. QED
253         BY <2>1
254         DEF Totality!RecoveryRecurringClosedPrefixes,
255             Totality!Reduction!Proof!RecoveryRecurringClosedPrefixes
256 <1>2. QED
257         BY <1>1
258
259 THEOREM UnconditionalClassicalAgreementFromTotalHistoryTheorem ==
260     Totality!TotalClassicalAgreementHistoryRealized
261     => UnconditionalClassicalAgreementSentence
262 PROOF
263     <1>1. ASSUME Totality!TotalClassicalAgreementHistoryRealized
264         PROVE UnconditionalClassicalAgreementSentence
265     <2>1. ClassicalAgreementHistory(Totality!ClassicalAgreementTotalHistoryObject)
266         BY TotalHistoryMatchesCollapsedClassicalHistory
267     <2>2. <>Totality!RecoveryClosedPrefix(TotalCycles)
268         BY <1>1 DEF Totality!TotalClassicalAgreementHistoryRealized
269     <2>3. QED
270         BY <2>1, <2>2
271         DEF UnconditionalClassicalAgreementSentence,
272             OrdinaryClassicalAgreementHistory
273 <1>2. QED
274         BY <1>1
275
276 THEOREM UnconditionalClassicalAgreementFromRecurringPrefixesTheorem ==
277     Totality!RecoveryRecurringClosedPrefixes
278     => UnconditionalClassicalAgreementSentence
279 PROOF
280     <1>1. ASSUME Totality!RecoveryRecurringClosedPrefixes
281         PROVE UnconditionalClassicalAgreementSentence
282     <2>1. Totality!Reduction!FiniteClassicalAgreementLiveness
283         BY LocalReductionLivenessFromRecurringPrefixesTheorem, <1>1
284     <2>2. Totality!TotalClassicalAgreementHistoryRealized
285         BY LocalTotalClassicalAgreementHistoryFromFiniteLivenessTheorem, <2>1
286     <2>3. QED
287         BY UnconditionalClassicalAgreementFromTotalHistoryTheorem, <2>2
288 <1>2. QED
289         BY <1>1
290
291 THEOREM UnconditionalClassicalAgreementFromRecurrenceKernelTheorem ==
292     /\ Totality!RecoveryRecurringInductionPremises
293     /\ Totality!RecoveryRecurringInductionKernel
294     => UnconditionalClassicalAgreementSentence
295 PROOF
296     <1>1. ASSUME Totality!RecoveryRecurringInductionPremises,
297         Totality!RecoveryRecurringInductionKernel
298         PROVE UnconditionalClassicalAgreementSentence
299     <2>1. Totality!RecoveryRecurringClosedPrefixes
300         BY LocalRecurringPrefixesFromRecurrenceKernelTheorem, <1>1
301     <2>2. Totality!Reduction!FiniteClassicalAgreementLiveness
302         BY LocalReductionLivenessFromRecurringPrefixesTheorem, <2>1

```

```

303 <2>3. Totality!TotalClassicalAgreementHistoryRealized
304     BY LocalTotalClassicalAgreementHistoryFromFiniteLivenessTheorem, <2>2
305 <2>4. QED
306     BY UnconditionalClassicalAgreementFromTotalHistoryTheorem, <2>3
307 <1>3. QED
308     BY <1>1
309
310 ====

```

References

- [1] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan-Gueta, and Ittai Abraham. *HotStuff: BFT Consensus in the Lens of Blockchain*. arXiv:1803.05069. <https://arxiv.org/abs/1803.05069>
- [2] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. *Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus*. arXiv:2105.11827. <https://arxiv.org/abs/2105.11827>
- [3] Ittai Abraham, Dahlia Malkhi, Kartik Nayak, Ling Ren, and Alexander Spiegelman. *Bullshark: DAG BFT Protocols Made Practical*. arXiv:2201.05677. <https://arxiv.org/abs/2201.05677>
- [4] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. *The Honey Badger of BFT Protocols*. Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. <https://eprint.iacr.org/2016/199.pdf>
- [5] Bingyong Guo, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. *Dumbo: Faster Asynchronous BFT Protocols*. Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. <https://dblp.org/rec/conf/ccs/GuoL0XZ20>
- [6] Alistair Stewart and Eleftherios Kokoris-Kogias. *GRANDPA: a Byzantine Finality Gadget*. arXiv:2007.01560. <https://arxiv.org/abs/2007.01560>
- [7] Jelena Doseovic, Roger Wattenhofer, and others. *Accountable Safety Implies Finality*. arXiv:2308.16902. <https://arxiv.org/abs/2308.16902>
- [8] Chainlink Labs. *Chainlink CCIP's Defense-In-Depth Security and the Risk Management Network*. Chainlink blog, 2023. <https://blog.chain.link/ccip-risk-management-network/>
- [9] John P. Conley. *Proof of Honesty: Coalition-Proof Blockchain Validation without Proof of Work or Stake*. Geeq technical paper, version 2.0, 2019. <https://geeq.io/wp-content/uploads/GEEQ-OFFICIAL-TECHNICAL-PAPER.pdf>
- [10] Sean Bowe, Jack Grigg, and Daira Hopwood. *Halo: Recursive Proof Composition without a Trusted Setup*. IACR ePrint 2019/1021. <https://eprint.iacr.org/2019/1021>
- [11] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. *Proof-Carrying Data from Accumulation Schemes*. IACR ePrint 2020/499. <https://eprint.iacr.org/2020/499>